SQX: bloques, plantillas y proyectos

AlgoProject — manual de uso. Actualizado el 2026-10-12 con `tools/manual.py`.

1. 40. sqx-lab — instalar y refrescar el kit de autoría sin `/plugin`
2. 111. Snippets propios de SQX — instalarlos en un worker
3. 36. Taxonomía y paletas de bloques — qué bloque pega con qué tipo de estrategia
4. 22. Plantillas — de una idea a una plantilla que construye
5. 28. Builder — de una plantilla a un proyecto que arranca, en un comando
6. 47. El proyecto del workflow — todas las pruebas de una estrategia en un solo custom project
7. 6. Quitar estrategias descartadas — aplicar un veredicto dentro de SQX
8. 27. Curar una databank — escribir el veredicto con un filtro o con un nombre
9. 17. Pipeline — mete cien estrategias, vuelve al cabo de unos días y lee los veredictos
10. 80. Autopiloto — el workflow de un proyecto de punta a punta, sin pararse
11. 105. Workflow de un comando — del paso 5 al 21 con una sola línea
12. 106. La campaña — un bucle sin supervisión que recorre plantilla × activo × temporalidad
13. 55. Retirar proyectos — quitar de SQX los custom projects que ya respondieron
14. 62. Metadatos de una estrategia — qué hace y con qué costes se probó, leído del fichero
15. 82. El perfil de mercado — qué comportamiento tiene cada activo, y si paga sus costes
16. 81. Memoria de resultados — qué se probó, dónde murió y por qué, y qué ideas se gastaron
17. 83. Tablero de investigación — en qué celda investigar ahora, y el director que lo lee
18. 90. Explorador de bibliografía — papers que entran solos, reglas ancladas a su cita
19. 97. Inspeccionar proyectos y archivos de SQX — salud, mapa, índice y copia de logs
20. 91. Potencia del perfil — qué tamaño de edge es capaz de ver el perfil de mercado
21. 92. Perfil agrupado — las medidas del perfil juzgadas por grupo de activos

40. sqx-lab — instalar y refrescar el kit de autoría sin `/plugin`

Qué pregunta responde

«¿Por qué las skills que escriben bloques, grupos, plantillas y proyectos de SQX no se instalan con `/plugin` como dice su web, y cómo me aseguro de que están bien?»

`sqx-lab` es un plugin de Claude Code del autor de StrategyQuant. Su instalación oficial son dos comandos:

```
/plugin marketplace add /home/sergioguslw/Desktop/AlgoProject/tools/sqx-lab
/plugin install sqx-lab@sqx-lab
```

y **en este entorno no existen**. Medido el 2026-09-24: la extensión de VSCode responde `/plugin isn't available in this environment`. Así que el kit se instala a mano, y este script es esa mano.

Lo único que pierdes respecto a una instalación de plugin de verdad es la actualización automática. Las cuatro skills y los dos comandos se comportan igual una vez enchufados.

Cuándo lo usas, y cuándo no

- **Sí**, la primera vez en una máquina nueva.
- **Sí**, después de bajar una versión nueva del zip y descomprimirla sobre `tools/sqx-lab/`.
- **Sí**, cada vez que cambien los bloques o los grupos del install que construye — los catálogos se quedan rancios y **un catálogo rancio no falla: inventa átomos** que no están en el install, y la plantilla sale silenciosamente mal.
- **No** para arreglar un fallo de una plantilla concreta. Para eso, `/sqx-doctor` primero.

Antes de empezar

Sólo que exista el install cuyos catálogos quieres construir. El script lo comprueba y se niega si no está. No hace falta parar SQX: sólo lee `customBlocks.xml`, `blockGroups.xml`, `user/projects` y `user/settings/StrategyTemplates`.

Cómo se ejecuta

```
bin/sqx-lab-install.sh
bin/sqx-lab-install.sh --role custodian     # construir los catálogos de otro install
```

flag	obligatorio	qué hace
`--role`	no	de qué install salen los catálogos. Por defecto **conductor**, que es el carril de autoría

Es **idempotente**: se puede correr todas las veces que quieras. Tarda unos segundos, salvo los catálogos, que son medio minuto.

Qué produce

ruta	qué es
`~/.claude/skills/sqx-*`	**cuatro symlinks** a las skills del plugin. Enlaces, no copias: así una versión nueva se ve sin volver a copiar nada
`~/.claude/commands/sqx-setup.md`, `sqx-doctor.md`	**copias**, y aquí está el detalle que importa
`tools/sqx-lab/.../catalog.json` y `catalog.md`	los catálogos, uno por skill, junto a su propia skill
`~/.sqx-lab/sqx-install.txt`	el install que las cuatro comparten. Lo escribe el bootstrap

⚠ **Por qué los comandos son copias y no enlaces.** Los del fabricante llaman a `"${CLAUDE_PLUGIN_ROOT}/doctor.py"`, y esa variable sólo la define una instalación de plugin de verdad. El script las copia sustituyendo la variable por la ruta real. **Ésa es la razón por la que una actualización obliga a volver a correr esto**: los enlaces se actualizan solos, las copias no.

⚠ **Se enchufan tres, no cinco.** `sqx-strategy-project` está **retirada** (dueño, 2026-09-25): clona cualquier proyecto de la instalación y despliega en ella, contra la regla dura 10, y seis de sus pruebas fallan aquí. Los proyectos los crea `sqx.projects.builder` desde el donante congelado. `sqx-spp` es local, no viene del zip, y duplica el `/spp` de este proyecto. El script quita el enlace de las dos si existe (`retired`).

⚠ **Las tres llevan encima las reglas de este proyecto.** El script inserta `tools/sqx-lab/overlays/<skill>.md` al principio de cada `SKILL.md` del fabricante, entre marcas (`overlay`), y a `sqx-strategy-template` le cambia la descripción para que tus defaults manden: tu condición fija más un aleatorio libre. Las capas se editan en `overlays/`, nunca dentro del fichero del fabricante. Hay además dos parches al código del fabricante, listados en `tools/sqx-lab/LOCAL_PATCHES.md` ; si una versión nueva se los lleva, el script lo avisa con `⚠ falta el parche local`. Descomprimir una versión nueva encima de `tools/sqx-lab/` se lleva también `sqx-spp` por delante.

Cómo se lee el resultado

Salida real del 2026-09-25:

```
sqx-lab 1.2.0  ->  conductor (/home/sergioguslw/Desktop/SQX_w1)
  skill   sqx-custom-block
  skill   sqx-random-group
  skill   sqx-strategy-template
  overlay sqx-custom-block
  overlay sqx-random-group
  overlay sqx-strategy-template
  command /sqx-setup
  command /sqx-doctor
catálogos:
    note: 74 talib_* atoms flagged UNUSABLE in single-symbol builds
    .../sqx-random-group/catalog.md
wrote .../sqx-strategy-template/engine/catalog.json
-> .../sqx-strategy-project/engine/catalog.json (8 projects, 8 templates)



================================================================
All green — the full block -> group -> template -> project chain is usable.
```

La última línea es la que se mira. Cualquier cosa que no sea `All green` la explica el propio doctor con la orden que lo arregla. Lo que decía antes de correr esto, el 2026-09-24:

```
[ WARN ] sqx-custom-block  —  built 20d ago from a DIFFERENT install: /home/sergioguslw/Desktop/SQX
```

Veinte días rancio y construido contra el **maestro** en vez del conductor. Eso es el fallo que hay que reconocer: las skills diseñaban contra bloques de otro install.

Un ejemplo completo — actualizar a una versión nueva

```
cd /tmp && curl -sSLO https://strategyquant.com/wp-content/uploads/2026/08/sqx-lab-1.2.0.zip
unzip -q sqx-lab-1.2.0.zip -d nuevo

# mira qué cambia antes de sobrescribir; los catalog.json y __pycache__ salen siempre
diff -rq nuevo/sqx-lab ~/Desktop/AlgoProject/tools/sqx-lab | grep -v catalog -e __pycache__

# guarda la skill local, que el copiado se la lleva
cp -r ~/Desktop/AlgoProject/tools/sqx-lab/plugins/sqx-lab/skills/sqx-spp /tmp/
cp -r nuevo/sqx-lab/. ~/Desktop/AlgoProject/tools/sqx-lab/
cp -r /tmp/sqx-spp ~/Desktop/AlgoProject/tools/sqx-lab/plugins/sqx-lab/skills/

cd ~/Desktop/AlgoProject && bin/sqx-lab-install.sh
```

Qué NO te dice

- **No dice que una plantilla vaya a construir.** Sólo que las skills ven el vocabulario correcto. Lo que construye o no lo dice el motor: `sqx/inspect/template_check.py` después del build.
- **No compara los tres installs entre sí.** Un bloque puede estar en el conductor y no en el custodio, y entonces el build ignora la plantilla sin un solo error. Eso es `python3 -m sqx.inspect.vocabulary --diff custodian`.
- **No instala nada en SQX.** Los bloques que una skill escribe se instalan con `python3 -m sqx.blocks.install`, con el install parado.

Si algo falla

- **`no está el plugin en …`** — `tools/sqx-lab/` no existe o está a medio descomprimir. Baja el zip otra vez.
- **`el install <rol> no existe`** — el rol no está en `config/machine.yaml`.
- **Un `[ WARN ] STALE catalog` justo después de correrlo** — el install cambió entre el bootstrap y el doctor, o construiste los catálogos contra un rol distinto del que mira el doctor. Ese rol sale de `~/.sqx-lab/sqx-install.txt`.

111. Snippets propios de SQX — instalarlos en un worker

Qué pregunta responde

¿Tiene este worker compilado el código Java propio del proyecto, y si no, cómo se le pone? StrategyQuant X deja ampliarlo con *snippets*: ficheros Java que él mismo compila al arrancar. El proyecto tiene dos:

snippet	qué hace
`AlgoTrades`	escribe las operaciones de todas las estrategias de un databank desde la memoria de SQX, idénticas byte a byte a las de `orderstocsv`: **4 segundos** por cada 1.000 estrategias, contra 274
`Seeded Random Entry`	un bloque de entrada al azar **con semilla**: la misma estrategia retesteada dos veces da las mismas operaciones (el bloque «Random Entry» del master no: cada retest es un sorteo nuevo)

Cuándo lo usas, y cuándo no

- **Sí:** una vez por worker (conductor y custodio), y otra vez cada vez que cambie un `.java` de `sqx/snippets/`.
- **No** para el master: es tu instalación y este comando no lo admite.
- **No** hace falta para lanzar un workflow: si un worker no los tiene, la exportación usa `orderstocsv` como siempre, sólo que más lenta.

Antes de empezar

El worker tiene que estar **parado** y sin otra sesión usándolo. El comando se niega si está arrancado.

Cómo se ejecuta

```
python3 -m sqx.snippets.install --role conductor --check   # sólo mirar: no toca nada
python3 -m sqx.snippets.install --role conductor           # instalar
python3 -m sqx.snippets.install --role custodian
```

Instalar tarda cerca de un minuto: copia los ficheros a `user/extend/` del worker, lo arranca una vez para que SQX los compile, lo para y comprueba que la clase compilada está en su sitio. También crea, si falta, el proyecto `Trade_AlgoTools`: un proyecto de una sola tarea («Export trades») desde el que se ejecuta `AlgoTrades` sobre el databank de cualquier otro proyecto.

Qué produce

- En el worker: los `.java` en `user/extend/Snippets/SQ/…`, sus clases en `internal/libs/Snippets.jar` y el proyecto `user/projects/Trade_AlgoTools/`.
- Nada en `AlgoData` ni en el repositorio.

Cómo se lee el resultado

```
$ python3 -m sqx.snippets.install --role conductor --check
conductor: todos los snippets compilados

$ python3 -m sqx.snippets.install --role custodian --check
custodian: faltan ['Code/PseudoCode/blocks/SeededRandomEntry.tpl', 'Snippets/SQ/Blocks/RandomEntry/SeededRandomEntry.java', 'Snippets/SQ/CustomAnalys…
```

- `todos los snippets compilados`: ese worker ya los usa.
- `faltan [...]`: los ficheros que ese worker todavía no ejecuta — sin copiar, cambiados después, o copiados pero sin compilar.

- `N snippet(s) compilados; el siguiente arranque ya los carga`: instalación correcta.

Un ejemplo completo

Tras instalar, no hay nada más que hacer a mano. Al terminar un workflow lanzado desde la ventana con sesión viva, el registro dice:

```
operaciones desde memoria: Results 1000, OOS 1000
operaciones escritas desde la memoria de la sesión viva (AlgoTrades), sin orderstocsv
```

Medido el 2026-10-12 sobre 1.000 estrategias de XAUUSD H1: exportar las operaciones de `Results` y `OOS` pasó de 274 + 136 segundos a 3,9 + 1,4.

El bloque con semilla aparece en el vocabulario como «(RAND) Seeded Random Entry», con dos parámetros: `Probability` (probabilidad de entrar en cada barra, en %) y `Seed` (la semilla: otra semilla, otro sorteo; la misma, el mismo).

Qué NO te dice

- No dice que una estrategia sea buena: sólo instala herramientas.
- `AlgoTrades` cubre las operaciones del resultado principal. Las métricas, y las exportaciones de otros mercados y de WFM, siguen saliendo por el camino de siempre.
- El bloque con semilla no tiene código de MetaTrader: es para medir contra el azar dentro de SQX, no para operar.

Si algo falla

- **`SQX no compiló — ningún snippet se actualizó`**: hay un error en algún `.java`. SQX, cuando uno falla, no actualiza **ninguno** y sigue usando los antiguos sin avisar; el comando lo detecta y enseña el error del compilador.
- **`está arrancado`**: para el worker (`bin/sqx-worker.sh --role <rol> stop`) o espera a que termine quien lo usa.
- **Un snippet recién copiado «no existe»**: SQX compila al arrancar pero carga lo que compiló el arranque anterior. Por eso el comando arranca y para una vez.

36. Taxonomía y paletas de bloques — qué bloque pega con qué tipo de estrategia

Qué pregunta responde

SQX conoce 1.020 bloques. Con tantos, el builder puede combinar un filtro de reversión a la media con un disparador de ruptura y sacar algo que parece rentable y no significa nada. Este módulo prepara el fichero donde se decide, bloque a bloque, **con qué familia de estrategia pega cada uno**.

No etiqueta nada: monta la tabla vacía, con todo lo que se puede saber leyendo la instalación, para que la etiqueta la ponga alguien que sepa —tú o un agente— y no se pierda en un chat.

Cuándo lo usas, y cuándo no

Se corre **cuando cambia el vocabulario de la instalación**: después de autorar bloques propios con `/sqx-strategy-template`, después de importar un paquete de bloques, o cuando llega una instalación nueva. Fuera de eso no hay motivo: el fichero ya está y las etiquetas no las pone este comando.

No sirve para saber si un bloque es bueno, ni si funciona en tu mercado. Solo dice qué bloques existen, qué papel puede jugar cada uno y qué grupos aleatorios lo agrupan.

⚠ **Y no vale para estrechar un hueco atado a un grupo.** Las etiquetas alimentan una paleta, y una paleta solo alcanza los huecos **libres** de una plantilla. Un `RandomCondition` atado a un grupo sortea ese grupo y se salta los interruptores — medido el 2026-09-24, `knowhow/authoring/builder-block-switches.md`. Para esos, la palanca es el grupo.

Antes de empezar

Nada. Lee tres ficheros XML de la instalación y una tarea Build del donante congelado. **No arranca SQX, no escribe en ninguna instalación y se puede correr con la GUI del maestro abierta.**

Lo único que tiene que existir es el donante, que ya está: `~/Desktop/AlgoData/projectsBackup/XAUUSD_base_2026-09-21/project.cfx`.

Cómo se ejecuta

```
python3 -m sqx.blocks.taxonomy                      # refresca desde el maestro
python3 -m sqx.blocks.taxonomy --role custodian     # leer otra instalación
```

flag	obligatorio	qué hace
`--role`	no	`master`, `conductor` o `custodian`. Por defecto **el maestro**, que es el que lleva el vocabulario de referencia
`--donor`	no	el `project.cfx` cuya tarea Build lista los papeles. Por defecto el donante congelado de XAUUSD. **Solo lectura**
`--out`	no	dónde escribir. Por defecto `sqx/blocks/taxonomy.yaml`

Tarda menos de dos segundos.

Qué produce

Un único fichero, **en el repositorio** porque es configuración a mano y tiene que estar versionada:

```
sqx/blocks/taxonomy.yaml        (~148 KB, 767 bloques en 83 categorías)
```

Lo sobrescribe entero en cada corrida, pero conservando lo único que no es derivado: los `archetypes` que ya tuvieran valor. Todo lo demás —los papeles, los grupos, la forma escrita— se vuelve a leer de la instalación y se pisa.

Cómo se lee el resultado

La salida real de hoy:

```
/home/sergioguslw/Desktop/AlgoProject/sqx/blocks/taxonomy.yaml: 767 bloques en 83 categorías, leídos de
SQX
  papeles   señal 588 · indicador 171 · nivel 85
  etiquetas 0 puestas, 767 por poner
  nuevos (767): ADX, ADXChangesDown, ADXChangesUp, ADXCrossDown, ADXCrossUp, ADXFalling, ADXHigher,
ADXLower …
```

Tres cosas que leer ahí:

- **767 de 1.020.** Los otros 253 existen y el builder no los ve: las acciones, las funciones matemáticas, los 158 `talib_*`. Etiquetarlos sería etiquetar algo que ninguna paleta puede tocar.
- **Los tres papeles.** Un mismo bloque puede jugar dos: el `ATR` es indicador (un valor que comparar) y nivel (el precio de una orden stop). Son dos interruptores distintos sobre él.
- **`etiquetas 0 puestas, 767 por poner`.** Ese es el trabajo que falta. Mientras esté en 0, ninguna paleta se puede construir.

Una fila del fichero:

```
BookTriggers_user:
  CBlock_BBBreakoutUp:
    roles: [signal]
    origin: own
    form: BBBreakoutUp
    groups: [BookTriggers]
    archetypes: {}
```

`archetypes` es lo que hay que rellenar: **un peso por familia**, `breakout`, `mean_reversion` y `trend`. `0` excluye el bloque de esa familia, `1` es neutro, `2` y `3` lo prefieren. Vacío = sin etiquetar, y un bloque sin etiquetar no entra en ninguna paleta.

`groups` es el aviso: ese bloque está en `BookTriggers`, así que una plantilla que ate un hueco a `BookTriggers` lo puede sacar **aunque la paleta lo apague**.

Un ejemplo completo

Etiquetar los bloques de ruptura propios y comprobar que sobreviven a un refresco.

```
$ python3 -m sqx.blocks.taxonomy
sqx/blocks/taxonomy.yaml: 767 bloques en 83 categorías, leídos de SQX
  papeles   señal 588 · indicador 171 · nivel 85
  etiquetas 0 puestas, 767 por poner
```

Se edita el fichero a mano y se pone una etiqueta:

```
  CBlock_BBBreakoutUp:
    roles: [signal]
    origin: own
    form: BBBreakoutUp
    groups: [BookTriggers]
    archetypes: {breakout: 3, mean_reversion: 0, trend: 1}
```

Se vuelve a correr, por ejemplo después de autorar un bloque nuevo:

```
$ python3 -m sqx.blocks.taxonomy
sqx/blocks/taxonomy.yaml: 768 bloques en 83 categorías, leídos de SQX
  papeles   señal 589 · indicador 171 · nivel 85
  etiquetas 1 puestas, 767 por poner
  nuevos (1): CBlock_DonchianBreakUp
```

La etiqueta sigue ahí y el bloque nuevo aparece vacío, esperando la suya.

Qué NO te dice

- **No dice si la etiqueta es correcta.** Es una opinión escrita en un fichero. La evidencia de que un bloque pega con una familia sale de las corridas (`runs.csv`), no de aquí.
- **No construye ninguna paleta ni toca ningún proyecto.** Escribe un YAML y nada más.
- **No alcanza los huecos atados a un grupo.** Repetido porque es el error que va a costar caro: una paleta perfecta no cambia nada en una plantilla cuyos huecos apuntan a grupos.
- **No es el catálogo completo del vocabulario.** Para eso, `sqx/inspect/vocabulary.py` y la página `01-empezar.pdf › 21-vocabulario`.

La librería de paletas

La taxonomía dice **qué es** cada bloque. Una **paleta** dice **qué entra en una construcción concreta**, y de esas hay muchas: tres por defecto que vienen con el repositorio, una por familia, y todas las que clones o crees encima.

```
sqx/blocks/palettes/ruptura_base.yaml        ★ por defecto, familia Ruptura
sqx/blocks/palettes/reversion_base.yaml      ★ por defecto, familia Reversión a la media
sqx/blocks/palettes/tendencia_base.yaml      ★ por defecto, familia Tendencia
sqx/blocks/palettes/<lo_que_tú_guardes>.yaml
```

Una paleta son cuatro decisiones y nada más:

campo	qué decide
`family`	de qué columna de la taxonomía hereda los pesos: `breakout` , `mean_reversion` o `trend`
`unlabelled`	qué pasa con un bloque sin etiquetar — `neutral` entra a peso 1, `off` lo deja fuera
`overrides`	los pesos puestos a mano, que ganan sobre la taxonomía. `0` apaga el bloque
`label` · `note` · `origin` · `based_on`	cómo se llama, para qué es, y de dónde salió

`unlabelled: off` más una lista de `overrides` hace que la paleta SEA esa lista: no entra nada más. Es la forma de tener una paleta curada —«para ruptura yo usaría estos treinta bloques»— **sin esperar a que la taxonomía esté etiquetada**. Con `neutral` la paleta hereda lo que diga la taxonomía y solo corrige lo que tú le digas.

El comando

```
python3 -m sqx.blocks.palette                    # la librería entera
python3 -m sqx.blocks.palette ruptura_base       # una paleta: qué deja pasar
python3 -m sqx.blocks.palette ruptura_base --json  # lo mismo, para un agente
```

flag	obligatorio	qué hace
`NAME`	no	el slug de una paleta. Sin él, lista la librería
`--json`	no	emite la paleta y su resolución bloque a bloque, para que un agente la consuma

Salida real de hoy, con las tres por defecto y ninguna curada todavía:

```
ruptura_base      Ruptura               default  condiciones   64  ⚠ FUERA DE 90-170  Ruptura — base
reversion_base    Reversión a la media  default  condiciones   83  ⚠ FUERA DE 90-170  Reversión a la
media — base
tendencia_base    Tendencia             default  condiciones   98  ok                 Tendencia — base
```

El número que importa es **condiciones**, y tiene banda: el dueño pide **entre 90 y 170 por build** (2026-09-24). Por debajo el builder no tiene de dónde combinar; por encima vuelve el sobreajuste que esto viene a evitar. El comando marca `⚠ FUERA DE 90-170` cuando una paleta se sale, y la ventana pinta el recuento en rojo. No es una orientación: es el criterio de aceptación.

Clonar, editar, guardar

Se hace en la ventana (`02-la-ventana.pdf › 35-app-plantillas`, zona **Paletas**): abres una, pulsas **Clonar**, le das nombre, y la copia es tuya para editar. Las tres por defecto **no se pueden borrar** desde la ventana — son de las que parte todo lo demás.

Si algo falla

- **`KeyError: 'custodian'`** — esa instalación no está en `config/machine.yaml`. Es válido tener solo maestro y conductor.
- **`FileNotFoundError` del donante** — falta `AlgoData/projectsBackup/XAUUSD_base_2026-09-21/project.cfx`. Es el donante congelado del que clona todo; si no está, hay que recuperarlo antes de seguir.
- **`StopIteration` en `builder_roles`** — el `.cfx` que le has pasado con `--donor` no tiene tarea Build, y los papeles solo viven ahí.
- **`'<slug>' already exists`** al clonar — ya hay una paleta con ese nombre de fichero. Dale otro nombre; no se sobrescribe una paleta por accidente.
- **`FileNotFoundError` de un `.yaml` de paleta** — le has pedido una paleta que no está en la librería. `python3 -m sqx.blocks.palette` sin argumentos dice cuáles hay.

22. Plantillas — de una idea a una plantilla que construye

Qué pregunta responde

Tienes una idea de entrada —"que el cierre cruce por encima de la banda superior de Keltner"— y quieres una plantilla de AlgoWizard que la construya. Estas tres herramientas hacen el camino entero sin abrir la GUI: autoran el bloque que falte, lo meten en la instalación, montan la plantilla y anotan lo que se ha probado y dónde.

Cuándo lo usas, y cuándo no

Cuando la lógica esté decidida: qué condición exactamente, en qué dirección, qué se fija y qué se deja aleatorio. Eso lo decide el brief, y el brief lo aprueba el dueño.

No sirve para decidir si la idea es buena, ni sustituye a una construcción real: lo que produce es una plantilla que SQX acepta. Si la estrategia gana dinero es otra pregunta y otras herramientas.

Antes de empezar

1. **La instalación de destino tiene que estar parada.** `customBlocks.xml` es el almacén de verdad, y una GUI abierta lo reescribe al salir: tu escritura se pierde en silencio. `sqx/blocks/install.py` se niega a escribir si el puerto de esa instalación contesta.
2. **Regla dura 5**: `python3 -m core.assets <SÍMBOLO>` antes de tocar cualquier plantilla o tarea. Si sale distinto de cero, se para y se pregunta.
3. **El bloque tiene que estar en las dos instalaciones** si autoras en el conductor y construyes en el custodio. `python3 -m sqx.inspect.vocabulary --diff custodian` te lo dice en un segundo.

Cómo se ejecuta

```
# 1. meter los bloques autorados en una instalación parada
python3 -m sqx.blocks.install ~/Desktop/AlgoData/templates/library/<nombre>/deps/blocks.xml --role
conductor
python3 -m sqx.blocks.install ~/Desktop/AlgoData/templates/library/<nombre>/deps/blocks.xml --role
custodian

# 2. montar la plantilla fijando un bloque en el esqueleto
python3 -m sqx.templates.build <nombre> <deps/blocks.xml> <CBlock_clave> <salida.sqx> [--shape
market_long] [--install custodian]

# 3. anotar lo que existe y lo que se ha probado
python3 -m sqx.templates.registry --set name=<nombre> --set archetype=breakout ...
python3 -m sqx.templates.registry --run --set template=<nombre> --set symbol=XAUUSD --set timeframe=H1
...
```

flag	obligatorio	qué hace
`install.py --role`	no	`conductor` (por defecto) o `custodian`. El maestro está prohibido
`build.py --shape`	no	esqueleto a transplantar; por defecto `market_long`
`build.py --install ROLE`	no	además copia el `.sqx` a `StrategyTemplates/` de esa instalación
`build.py --param '#Type#=1'`	no	fija un parámetro que nombraste (una EMA: `#Type#=1`); el resto de números —periodos, desviaciones— los **sortea el builder** en cada estrategia. Repetible

Los periodos son aleatorios, siempre salvo que des un valor (dueño, 2026-09-26). SQX no deja sortear los parámetros de un bloque fijo, así que `build.py` mete tu condición en un **grupo de un solo ítem** (`<nombre>Signal`) y ata a él el primer hueco: todas las estrategias llevan tu condición, cada una con su periodo. El grupo se escribe también en `deps/groups.xml`, y hay que instalarlo en los dos workers con `python3 -m sqx.blocks.install deps/groups.xml --role conductor` (y `--role custodian`). Comprobado 2026-09-26 en USDJPY H1: 15 periodos distintos de la EMA en 20 estrategias.

Un bloque nativo (no autorado) se fija igual, pasando en lugar de `deps/blocks.xml` el `config.xml` de AlgoWizard del conductor. Salida real, 2026-09-25, que reproduce la `emaCloseAbove` que se hizo a mano y ya está confirmada en build:

```
$ python3 -m sqx.templates.build emaCloseAbove \
    ~/Desktop/SQX_w1/internal/web/SQWIZARD/branding/global/config.xml MABarClosesAbove \
    template.sqx --param '#Type#=1'
template.sqx  (5343 bytes, shape market_long, fixed MABarClosesAbove)
```

| `build.py --set` | no | subcarpeta usada al instalar; por defecto `authored` | | `registry.py --run` | no | escribe en `runs.csv` en vez de en `registry.csv` | | `registry.py --set COL=VALOR` | sí | un valor por columna, repetido tantas veces como columnas |

Los tres son instantáneos. Ninguno arranca SQX; `install.py` exige que esté parado.

Qué produce

```
<instalación>/user/settings/customBlocks.xml           modificado, con copia fechada al lado
<instalación>/user/settings/StrategyTemplates/<set>/   la plantilla, si usas --install
AlgoData/templates/library/<nombre>/template.sqx       la plantilla
AlgoData/templates/registry.csv                        una fila por plantilla
AlgoData/templates/runs.csv                            una fila por plantilla × símbolo × timeframe
```

`install.py` **reemplaza** un bloque cuya clave ya exista en vez de añadirlo dos veces: SQX lee la primera coincidencia y un duplicado no se ve hasta que una construcción usa el equivocado. Antes de escribir deja una copia fechada en `customBlocks-backups/`, igual que hace la GUI.

Cómo se lee el resultado

```
$ python3 -m sqx.blocks.install .../deps/blocks.xml --role conductor
SQX_w1: added ['CBlock_CloseCrossesAboveKCUpper', 'CBlock_CloseCrossesBelowKCLower'], replaced []
  store now holds 173 blocks; previous copy kept at .../customBlocks-backups/1790090019825.xml

$ python3 -m sqx.inspect.vocabulary --diff custodian
SQX_w2 lacks, against SQX_w1:
  custom blocks (2): CBlock_CloseCrossesAboveKCUpper, CBlock_CloseCrossesBelowKCLower
```

Esa segunda salida es la que hay que mirar siempre: el bloque estaba en el conductor y **no** en el custodio, que es donde se construye. Una plantilla que referencia un bloque ausente no da error — se queda coja y desaparece del builder.

```
$ python3 -m sqx.templates.build keltnerUpperCrossUp deps/blocks.xml CBlock_CloseCrossesAboveKCUpper
template.sqx
.../template.sqx  (5267 bytes, shape market_long, fixed CBlock_CloseCrossesAboveKCUpper)
```

`build.py` aborta si el bloque no aterriza en la señal, si queda más de un hueco aleatorio, o si el XML resultante no parsea. Los tres fallaron durante el desarrollo, así que los tres se comprueban.

Configurar el proyecto desde `assets/`

Un proyecto se clona de un donante, y **el donante lleva los ajustes vivos del maestro**, que no son la política declarada. En XAUUSD el donante cobra `SizeBased 8` donde `assets/` dice `PercentageBased 0.00174`, y cero slippage donde `assets/` dice cinco puntos. Si nadie lo pisa, la construcción corre a un coste que no ha elegido nadie, y no salta ningún error.

```
python3 -m sqx.projects.configure <ruta/project.cfx> XAUUSD
```

Sin `--segment`, **cada tarea toma el suyo de su tipo**: `Build` → `build`, todo lo demás → `oos1`, según `assets/_policy.yaml`. Eso es lo que hace útil declarar `spread_is` y `spread_oos` por separado; forzar un solo tramo en toda la cadena valora los retests con el spread de construcción.

```
cfgtest.cfx  XAUUSD
  build: spread 10.0, slippage 5, PercentageBased 0.00174, swap percent -7/-7,
         ventana 1199145600000 a 1514764800000
    1 tarea(s): Build-Task3.xml
  oos1:  spread 10.0, slippage 5, PercentageBased 0.00174, swap percent -7/-7,
         ventana 1514764800000 a 1672531200000
    14 tarea(s): Retest-Task1.xml, ... Retest-Task14.xml
  ⚠ PROVISIONAL: spread_is, spread_oos, commission, slippage, swap_long, swap_short
  ⚠ rangos MC Retest sin decidir: spread, slippage
```

Se niega ante tres cosas: `oos2` (sólo si lo pide un agente autónomo, `ALGO_AUTONOMOUS=1`), un coste con `use: null`, y un `.cfx` que tenga abierto una instalación viva, porque lo reescribe al salir.

Un ejemplo completo

La cadena entera del `keltnerUpperCrossUp`, tal como se corrió el 2026-09-22:

```
python3 -m core.assets XAUUSD                                    # regla dura 5 → exit 0
python3 -m sqx.blocks.install .../deps/blocks.xml --role conductor
python3 -m sqx.blocks.install .../deps/blocks.xml --role custodian
python3 -m sqx.templates.build keltnerUpperCrossUp .../deps/blocks.xml \
        CBlock_CloseCrossesAboveKCUpper .../template.sqx
# proyecto de una sola tarea sobre el donante, con la plantilla apuntada
bin/sqx-worker.sh --role custodian start
python3 -c "from core import worker; worker.call('-project action=start
name=smoke_keltnerUpperCrossUp','custodian')"
```

Resultado, leído del `-project action=status`:

```
Strategies generated                        13639
Rejected                                  99.48 %
Running time so far                         29 s.
In databank                                    30
```

Y la comprobación que decide si la plantilla sirve de algo — que las estrategias construidas lleven **de verdad** el bloque fijo:

```
estrategias: 30   llevan el bloque fijo: 30
companeros que eligio el builder: CSSAMarketRegimeAboveLevel ×15, VWAP ×5,
                                  BollingerBands ×5, IsFalling ×4, HighD ×3, UlcerIndex ×2, ...
```

30 de 30 con el Keltner fijo, cada una con un compañero aleatorio distinto. Eso es exactamente lo que se pidió: una condición fija y una libre.

Qué NO te dice

- **No dice si la estrategia es rentable.** Una construcción de humo prueba la cadena, no la idea. 30 estrategias con tope de 30 no son una muestra de nada.
- **No prueba que el bloque calcule lo que su nombre dice.** Que SQX lo acepte y construya con él no significa que el cruce esté bien definido. Eso solo lo dice leer sus trades.
- **No valida los costes.** El spread, la comisión y el point value de XAUUSD son a día de hoy los defaults de SQX, provisionales, no cifras acordadas con el bróker.
- **`registry.csv` no se lee solo.** Anota lo que le digas; si no anotas una corrida, la respuesta a "¿ya lo probé?" será que no.

Si algo falla

`SQX_w1 is running on port 5060` — la instalación está levantada y reescribiría tu cambio al salir. Párala: `bin/sqx-worker.sh --role conductor stop`.

`CBlock_X did not land in the signal` — la clave no está en el XML de bloques que le pasaste, o el esqueleto no tiene dos huecos aleatorios.

La construcción termina y el databank está vacío en disco. No es un fallo: el databank está en `Auto-sync never` y SQX guarda en memoria. Se arregla parando la instalación, poniendo ese databank en `Auto-sync every 1 hour` dentro del `project.cfx` —nunca con una instancia abierta, regla dura 4— y reconstruyendo. Está contado en `knowhow/databanks/memory-vs-disk-exporter.md`.

El proyecto arranca y no testea nada. Estás usando `startOnlyTask`, que informa de éxito y no hace nada. Usa `action=start`, y un `action=stop` antes de cada segundo arranque.

28. Builder — de una plantilla a un proyecto que arranca, en un comando

Qué pregunta responde

Tienes una plantilla en la librería y quieres que SQX genere estrategias con ella sobre un activo. Esto monta el proyecto entero: clona el donante, se queda solo con la tarea de construcción, apunta la plantilla, pone los topes, deja los databanks escribiendo a disco, fecha la ventana desde `assets/` y lo instala. **Hasta el 2026-09-23 esos cinco pasos se editaban a mano en el XML de la tarea**, que es lo que impedía meter la cadena dentro de una aplicación.

Cuándo lo usas, y cuándo no

Cuando la plantilla ya existe y está registrada. No sirve para crear la plantilla —eso es `/sqx-strategy-template` — ni para lanzar la construcción: esto deja el proyecto listo, y arrancarlo es una decisión aparte porque cuesta CPU.

Antes de empezar

1. **La instalación de destino tiene que estar parada.** Reescribe el `project.cfx` al salir y se comería el proyecto recién escrito (regla dura 4). El comando se niega si el puerto contesta.
2. **Regla dura 5**: el activo tiene que tener sus costes pactados. Si alguno sigue en `use: null`, el comando para y te dice cuál.
3. **El bloque de la plantilla tiene que estar en esa instalación**: `python3 -m sqx.inspect.vocabulary --diff custodian`.

Cómo se ejecuta

```
python3 -m sqx.projects.builder Test_chat_v2 --purpose "probar la plantilla keltner" \
    --template ~/Desktop/AlgoData/templates/library/keltnerUpperCrossUp/template.sqx \
    --symbol XAUUSD --role custodian --max-strategies 25 --minutes 8
```

flag	obligatorio	qué hace
`name`	sí	`Test_<...>` para una prueba de funcionamiento que se tira al acabar, `Trade_<...>` para un proyecto que tiene que dar fruto de verdad, `Research_<...>` para los que construye la cola de investigación (se quedan hasta que tú digas); **solo guiones bajos** — la API parte por espacios. Cualquier otro nombre se rechaza
`--purpose`	sí	una frase: para qué es el proyecto. Queda en `AlgoData/projects/registry.csv`, y es lo que lee quien encuentre el proyecto semanas después
`--template`	sí, o `--simple`	el `.sqx` de la librería. Se rechaza si abre en las dos direcciones (regla 13) o si pasa del tope de condiciones (regla 14): dos huecos aleatorios libres en un lado, o condiciones fijas más un hueco por encima de 2
`--simple long|short`	sí, o `--template`	**sin plantilla**: una Simple Strategy en esa única dirección. Ver «Sin plantilla: Simple Strategy» más abajo
`--random`	no	generación aleatoria en vez de genética: el «gemelo aleatorio» de una población. Solo cambia eso; el resto de tu bloque genético queda igual
`--symbol`	sí	activo de `assets/`; de ahí salen la ventana, la sesión, el feed y los costes
`--role`	no	instalación destino; `custodian` por defecto
`--max-strategies`	no	tope de estrategias, 30 por defecto
`--minutes`	no	tope de reloj, 10 por defecto
`--donor`	no	proyecto del que clona; el XAUUSD congelado por defecto
`--session-from`	no	un `project.cfx` que fuerce de dónde tomar la sesión y el feed del activo, si no quieres el que el comando encontraría solo. **Sólo lectura**, nunca se escribe en él
`--acceptance`	no	qué filtros de aceptación lleva la tarea Build: `study` (por defecto, los de `assets/_study.yaml`) o `calibration` (población sin seleccionar: operaciones ≥ 100 y beneficio > 0). Ver «Los filtros de aceptación» más abajo
`--json`	no	emite el resultado como JSON y nada más — **esto es lo que consume un chat**

Instantáneo: mueve ficheros y reescribe XML, no arranca SQX.

Encadena los databanks solo. El input de cada tarea pasa a ser el output de la anterior, en el orden que lleva `config.xml`. Hace falta porque al quedarte con un subconjunto de tareas (`--only`) las supervivientes siguen apuntando a donde apuntaban en el donante: sin esto, la tarea de mercados adicionales leía **su propio output** y el cross-check corría sobre un databank vacío. `--json` lo devuelve en `chain`, con lo que cada tarea leía antes. La primera tarea se queda con el input del donante a propósito — no hay nada anterior que darle, y un Build genético se siembra de `Initial population`, no de su `Input`.

Otros activos además de XAUUSD — arreglado el 2026-09-27

Hasta el 2026-09-23 `--symbol` no cambiaba el mercado: aplicaba la doctrina, el timeframe, la sesión y la ventana del activo que nombraras, pero el feed seguía siendo el del donante (oro por defecto), así que un `--symbol USDJPY` construía en `XAUUSD_DukasM1_Infinox` al spread del oro, con el horario de USDJPY y sin que `USDJPY_DukasM1_the5ers` apareciera en ninguna tarea.

Desde el 2026-09-27 (`sqx/projects/source.py`) el comando **también sustituye el feed**: busca, en cualquier instalación, el `project.cfx` más reciente que defina a la vez la sesión y el `<Symbol>` del activo, y copia los dos — nunca solo la sesión, que era la trampa original (`knowhow/costs/sessions-per-asset.md`)—. El resumen dice de qué proyecto los tomó, por ejemplo:

```
feed      XAUUSD_DukasM1_Infinox → USDJPY_DukasM1_the5ers en 18 tarea(s), traído de
          .../Test_USDJPY_mcrRanges/project.cfx
sesión USDJPY_ftmo añadida a 17 tarea(s) que la nombraban sin definirla
```

Comprobado de punta a punta el 2026-09-27 (`Test_USDJPY_donchianUpperCrossUp_M30`, sin `--session-from`): 200/200 estrategias construidas sobre `USDJPY_DukasM1_the5ers`, y el workflow completo — puerta, mercados adicionales, crossTF, MC Retest, SPP, variantes, WFC y estructura — leyó ese mismo feed en cada paso.

Si el activo no tiene ningún proyecto que defina su sesión y su feed (medido: EURUSD, AUDUSD, USDCAD, GBPUSD hoy), el comando se niega en vez de inventar nada — regístralo en SQX primero, o pasa `--session-from` a mano si sabes de qué proyecto tomarlo.

Antes de fiarte de un proyecto para un activo nuevo, míralo:

```
python3 -c "
import zipfile, re
z = zipfile.ZipFile('<instalación>/user/projects/<nombre>/project.cfx')
for n in sorted(z.namelist()):
    if n == 'config.xml': continue
    t = z.read(n).decode('utf-8', 'replace')
    print(n, sorted(set(re.findall(r'<Chart symbol=\"([^\"]+)\"', t))))
"
```

Si el feed de tu activo no sale ahí, el proyecto no es de tu activo. `OPEN.md` issue 35 tiene la causa y las dos vías de arreglo.

Qué produce

```
<instalación>/user/projects/<nombre>/project.cfx           el proyecto
<instalación>/user/settings/StrategyTemplates/authored/    la plantilla, con su propio nombre
```

La plantilla se instala con el nombre de **su carpeta de librería**, no con el del fichero: todas las carpetas guardan un `template.sqx`, así que instalarlas por el nombre del fichero haría que dos plantillas distintas se pisaran en silencio.

Cómo se lee el resultado

```
chat_v2 on SQX_w2
  template  .../StrategyTemplates/authored/keltnerUpperCrossUp.sqx
  tasks     Build   caps 25 strategies / 8 min
  segments  Build-Task3.xml=build
  to disk   Results, Last generation, Initial population, Strategies to improve
  costes    NO van en la tarea: una InstrumentInfo que no coincida con el registro
            de SQX deja el proyecto irresoluble. Aplícalos con el install vivo:
            -instrument action=edit instrument=XAUUSD_Infinox defaultspread=10.0 ...
  ⚠ PROVISIONAL: spread_is, spread_oos, commission, slippage, swap_long, swap_short
```

Cuatro cosas que decide esa salida:

- **`to disk`** — los databanks que estaban en `Auto-sync never` y ahora escriben. Sin eso, tras generar el directorio queda vacío y `/curate` no tiene ficheros que mover.
- **`segments`** — de qué tramo sale la ventana de cada tarea. `build` es la única muestra que ve el generador.

- `costes` — no se escriben en la tarea, y el motivo está medido: ver `knowhow/costs/per-task-costs.md`.
- **Los rangos del MC Retest sí se escriben**, porque viven en los `<Method type="Randomize*">` de la propia tarea y no en la `InstrumentInfo`. Salen de `mc_retest:` del activo; los que sigan en `null` se saltan en vez de inventarse, y la preflight lo avisa.
- `PROVISIONAL` — cifras de trabajo del dueño, no pactadas con el bróker. Marcan todo resultado.

Si la plantilla fuese a ignorarse — `type="simple"` con un `templateFile` puesto— **aborta** en vez de construir. Esa comprobación es gratis: lee un atributo, mientras que demostrar lo mismo después cuesta la construcción entera.

Sin plantilla: Simple Strategy (`--simple`)

Cuando no hay una idea fija no se escribe ninguna plantilla (regla 14: una plantilla de solo huecos aleatorios es una Simple Strategy). El comando deja la tarea Build así:

- **StrategyType simple**, sin plantilla.
- **Una sola dirección**: el interruptor «Long / Short» de «What to build» (`MarketSides`) en la que pidas, con las simetrías apagadas. Largos y cortos son proyectos distintos (regla 13).
- **«What to build»: entrada 1..2 y salida 1..2** — aquí la cuenta es por estrategia. Con plantilla es por cada hueco aleatorio, y el comando escribe `2 − condiciones fijas` (1 fija + 1 hueco → 0..1); la salida con plantilla sigue siendo optativa (0..2).
- Tus opciones genéticas, tal cual (`sqx/projects/buildmode_model.xml`), órdenes solo a mercado.

```
# genética
python3 -m sqx.projects.builder Test_Calib_USDJPY_H1_rupturaL --purpose "población de calibración,
ruptura, largos" \
    --simple long --symbol USDJPY --timeframe H1 --workflow --acceptance calibration \
    --minutes 6 --max-strategies 12000
# su gemelo aleatorio
python3 -m sqx.projects.builder Test_Calib_USDJPY_H1_rupturaL_rnd --purpose "gemelo aleatorio de la
anterior" \
    --simple long --random --symbol USDJPY --timeframe H1 --workflow --acceptance calibration \
    --minutes 6 --max-strategies 12000
```

Después **siempre** la paleta de la familia, y contar lo que quedó (sin paleta el proyecto hereda las 400 condiciones del donante):

```
python3 -m sqx.projects.buildingblocks --project Test_Calib_USDJPY_H1_rupturaL --palette ruptura_base_v2
--count
```

Lo que responde `--count` (también solo, sin `--palette`, sobre cualquier proyecto):

```
Build-Task3.xml: simple (sin plantilla)
  dirección      long  (simetría entrada false, salida false)
  condiciones    entrada 1..2 · salida 1..2 · lookback 1..1
  órdenes        EnterAtMarket
  salidas        ExitAfterBars.ExitAfterBars (100 %), _ExitRule_ (50 %)
  bloques ON     condiciones 6/575 · indicadores/valores 0/161 · niveles stop/limit 0/85 ·
reloj/calendario 0/23
  generación     genetic-evolution · fitness ReturnDDRatio
  genética       población 250 × 20 islas · generaciones 25 · cruce 89 · mutación 20 · …
  aceptación     NumberOfTrades >= 100 @sampleType 10 & NetProfit > 0 @sampleType 10 · MC manipulation
false
  topes          MaxStrategies 12000 · stop {'type': 'time-limit', … 'minutes': '6'}
  condiciones ON (6): CBlock_BBBreakoutDown, CBlock_BBBreakoutUp, CBlock_BreakoutDonchianLong, …
```

Cada línea es algo que la tarea **lleva escrito**, leído del fichero: no se supone nada. «niveles stop/limit» y «reloj/calendario» tienen que estar a 0 (regla 15) — el comando de la paleta no los enciende nunca, los pida la paleta o no, y dice cuáles ha dejado apagados.

Un ejemplo completo

```
python3 -m sqx.projects.builder Test_chat_v2 --purpose "humo" --template
.../keltnerUpperCrossUp/template.sqx \
    --symbol XAUUSD --role custodian --max-strategies 25 --minutes 8
bin/sqx-worker.sh --role custodian start
python3 -c "from core import worker; print(worker.call('-project action=start
name=chat_v2','custodian'))"
```

Resultado real del 2026-09-23: el proyecto arranca, genera **25 estrategias**, quedan **25 en disco**, y la puerta de la plantilla da **25/25 llevan el bloque fijo** (`python3 -m sqx.inspect.template_check --role custodian chat_v2`).

Qué NO te dice

- **No dice si la estrategia sirve.** Un proyecto que arranca y genera no es una estrategia buena.
- **No aplica los costes.** Los imprime como comando; aplicarlos en el registro de SQX es un paso aparte y **global a la instalación**.
- **No lanza la construcción.** A propósito: eso cuesta horas y es una decisión del dueño.

Todo el workflow en un solo proyecto: `--workflow`

Desde el 2026-09-25 (decisión del dueño), cuando una plantilla entra en el workflow el proyecto se crea con `--workflow` : lleva **todas** las tareas de los pasos siguientes —mercados, crossTF, MC Retest, SPP, WFC y WFM— apagadas salvo `CONSTRUCCION` y `OOS` , e ignora `--tasks` y `--only` . Cada paso enciende sólo las suyas antes de lanzar. Todo el detalle en `04-sqx-plantillas-y-proyectos.pdf › 47-proyecto-workflow` .

Si algo falla

`Project has unresolved resources` al arrancar — la `InstrumentInfo` de alguna tarea no coincide con el registro de SQX. No lo arregla este comando; ver `knowhow/costs/per-task-costs.md` .

`the custodian is running and rewrites a project.cfx on exit` — párala primero.

`'<nombre>': project names are underscores only` — la API HTTP parte el comando por espacios.

Los filtros de aceptación del build (`assets/_study.yaml`)

Mientras el build genera, SQX sólo guarda en el databank las estrategias que cumplen las condiciones de «Ranking» de la tarea. Hasta el 2026-10-02 esas condiciones eran las que traía el proyecto donante (más de 500 operaciones, Sortino > 0,6, estancamiento < 540 días) y nadie las había decidido. Ahora el builder las **sustituye enteras** por las de `assets/_study.yaml`, en cada proyecto nuevo:

filtro	valor por defecto	cómo se escribe en SQX
operaciones por año, suelo	M15, M30 y H1: 35 · H4: 20 — sin techo desde el 2026-10-09	`NumberOfTrades >= suelo`, en totales: por año × años del tramo `build` del activo (XAUUSD H1, 10 años: 350). Un `max:` escrito en un timeframe añade `<= techo`
profit factor mínimo	1,20 (índices 1,25: su build es más corto)	`ProfitFactor >= 1.2`
beneficio neto	mayor que 0	`NetProfit > 0`

Todas se miden **sólo sobre el IS** (la ventana `build`; `sampleType` 10 de SQX): un filtro del build que leyese el OOS lo gastaría. Las cifras son una **propuesta provisional** hasta que las apruebes; se cambian editando el YAML, sin tocar código. Si al activo o al timeframe le falta un valor, el builder **se niega** a crear el proyecto — no hay filtro por omisión.

La salida del builder lo dice en una línea, y lo mismo queda en `AlgoData/projects/registry.csv` (columna `acceptance`) y en el `--json` (`acceptance`):

```
  aceptación  study @IS: NumberOfTrades >= 350 & ProfitFactor >= 1.2 & NetProfit > 0
(assets/_study.yaml, build 10.0 años)
```

Modo calibración. `--acceptance calibration` escribe el juego relajado del bloque `calibration:` del mismo fichero (operaciones ≥ 100 y beneficio > 0) y apaga el crosscheck de Monte Carlo del build, que no filtra y gasta CPU. Sirve para construir poblaciones sin seleccionar con las que calibrar los propios filtros; nunca para un proyecto `Trade_` ni `Research_`.

Cambiar los filtros de un proyecto que ya existe (con su instalación parada — si está en marcha el comando se niega, porque SQX reescribe el fichero al salir):

```
python3 -m sqx.projects.rankings ~/Desktop/SQX_w2/user/projects/Test_x/project.cfx XAUUSD --timeframe H1
```

```
.../Test_x/project.cfx: Build-Task3.xml
  aceptación  study @IS: NumberOfTrades >= 350 & ProfitFactor >= 1.2 & NetProfit > 0  (build 10.0 años)
```

`--acceptance calibration` hace lo mismo con el juego relajado, y `--study-file otro.yaml` lee los umbrales de otro fichero — para comparar dos juegos de filtros en un A/B sin tocar el oficial.

`MonteCarloManipulation` no se usa nunca (dueño, 2026-10-11): `set_crosschecks` y la comprobación final del builder lo rechazan. Para apagarlo en un proyecto que ya lo trae encendido (con su instalación parada):

```
python3 -m sqx.projects.rankings ~/Desktop/SQX_w2/user/projects/Test_x/project.cfx --mc-off
```

Imprime en cuántas tareas lo ha apagado; no cambia nada más del fichero.

La doctrina de construcción (`assets/_build.yaml`)

Desde el 2026-09-23 el builder no hereda nada del donante en lo que a la forma de la estrategia se refiere. Lo escribe todo desde `assets/_build.yaml`, y **en todas las tareas del proyecto**:

regla	valor por defecto
condiciones de entrada / de salida	como máximo **2 y 2**
global lookback	**1 barra** (`minShift = maxShift = 1`)
tipos de orden	**sólo a mercado**
salidas	por barras o por condición; **nada** de SL, TP, trailing ni break-even
barras en mercado	**2 a 24 horas**, convertidas a barras según el timeframe
money management	`ATRRiskBasedSizingFixedRisk` , ATR(20) × **4**, riesgo 1000 sobre 100.000
salir el viernes	sí, a las **21:00**
motor	**MetaTrader5 (hedged)**
databank	10.000 estrategias, parada por databank lleno
crosschecks	**sólo** el de alta precisión, **al mismo spread que el test principal**
genético	copia verbatim de `XAUUSD_Breakout_H1`

El timeframe es obligatorio y va en `--timeframe`. Todas las tareas lo comparten: un retest en otro timeframe no está probando la estrategia que se construyó.

```
python3 -m sqx.projects.builder Test_XAU_doctrina --purpose "ver la doctrina aplicada" --timeframe M30 \
    --template ~/Desktop/AlgoData/templates/library/keltnerUpperCrossUp/template.sqx \
    --symbol XAUUSD --role custodian --tasks Build,Retest
```

```
  doctrina  M30 en todas las tareas, sesión XAUUSD_the5ers, salida por barras 4–48
  sesión XAUUSD_the5ers añadida a 14 tarea(s) que la nombraban sin definirla
```

Comprobado sobre las 20 estrategias que generó `algo_XAU_doctrina_smoke` : **20 de 20** con exactamente 2 condiciones de entrada, 1 de salida, `EnterAtMarket` y nada más, shift 1 en los 31 sitios donde aparece, y cero stop loss, profit target o trailing.

Quedarse con una sola tarea de retest

El donante lleva catorce. `--tasks Retest` las conserva todas, que es la cadena entera; para un proyecto de dos pasos —construir y cruzar la puerta del OOS— se nombra la que se quiere:

```
python3 -m sqx.projects.builder Test_XAU_ISOOS_ejemplo --purpose "ejemplo IS/OOS" --timeframe M30 --
symbol XAUUSD \
    --template <plantilla> --role custodian \
    --tasks Build,Retest --only Build-Task3.xml,Retest-Task1.xml
```

`Retest-Task1.xml` es la que lee de `Results` y escribe en `OOS` . Las demás son la cadena de Monte Carlo, los SPP y la WFM, cada una encadenada al databank de la anterior.

Hay un ejemplo construido y corrido, `XAU_ISOOS_ejemplo` en el custodio: cómo se hizo y qué tiene en `knowhow/locations/xau-isoos-example-project.md` .

47. El proyecto del workflow — todas las pruebas de una estrategia en un solo custom project

Qué pregunta responde

¿Dónde corre cada prueba de SQX de una población, del build a la Walk Forward Matrix? En **un único custom project**, creado en el paso 5 con todas las tareas del workflow dentro. Cada paso enciende sólo las suyas antes de lanzar, y SQX se salta las apagadas. Dos comandos: `builder --workflow` lo crea y `stage` elige qué paso corre en el próximo `start`.

Decisión del dueño, 2026-09-25: "que se creen todas las tasks dentro del mismo custom project".

Cuándo lo usas, y cuándo no

- **Sí:** siempre que una plantilla entra en el workflow. `/template-run` lo crea así, y el resto de skills (`/crossmarket`, `/crosstf`, `/mcretest`, `/spp`, `/variants`, `/wfm`) configuran y corren su tarea dentro de ese mismo proyecto.
- **No:** para una prueba suelta sobre un proyecto hecho sólo para ella (por ejemplo el antiguo `USDJPY_variantes`). Ahí se sigue usando `builder --tasks … --only …`. `stage` se niega sobre un proyecto que no lleva las tareas del workflow, y los configuradores lo avisan con un ⚠.

Antes de empezar

- `python3 -m core.assets <SIMBOLO>` con salida 0 (regla dura 5).
- **La instalación donde vive el proyecto, parada.** SQX reescribe el `.cfx` al salir y se perdería el cambio (regla dura 4). Los dos comandos se niegan si la instalación está arriba.
- Una plantilla ya en la librería (`~/Desktop/AlgoData/templates/library/<nombre>/template.sqx`).

Cómo se ejecuta

Crear el proyecto (paso 5):

```
python3 -m sqx.projects.builder Trade_XAUUSD_keltner_H1 --purpose "workflow entero de la keltner en H1"
\
    --template ~/Desktop/AlgoData/templates/library/keltnerUpperCrossUp/template.sqx \
    --symbol XAUUSD --timeframe H1 --role custodian --workflow --max-strategies 2000 --minutes 90
```

Elegir qué corre en el próximo `start`, a mano:

```
python3 -m sqx.projects.stage --cfx ~/Desktop/SQX_w2/user/projects/XAUUSD_keltner_H1/project.cfx \
    --step spp
```

flag	obligatorio	qué hace
`--workflow` (builder)	no	mete todas las tareas del workflow en el proyecto; sin él, el builder se queda con `--tasks` / `--only` como antes
`--cfx` (stage)	sí	el `project.cfx` del proyecto
`--step` (stage)	sí	uno o varios pasos separados por comas: `build`, `oos`, `crossmarket`, `crosstf`, `mcretest`, `spp`, `wfm`, `wfc`

Normalmente no hace falta llamar a `stage`: **cada configurador lo hace solo para su paso** al terminar (`sqx.projects.crossmarket`, `crosstf`, `mcretest`, `spp`, `wfc`, `wfm`). Tardan menos de un segundo y no arrancan SQX.

Qué produce

`<instalación>/user/projects/<nombre>/project.cfx` con 18 tareas:

paso	tarea(s)	lee → escribe
build	`CONSTRUCCION`	— → `Results`
oos	`OOS`	`Results` → `OOS`
crossmarket	`Retest Markets - Family`	`OOS` → `Retest Markets - Family`
mcretest	`MCR 1 Bar` … `MCR 8 Stress`	lo que diga `--input`
spp	`SPP IS`, `SPP OOS`	lo que diga `--input`
wfm	`WFM`	lo que diga `--input`
crosstf	`CrossTF` (añadida)	`CrossTF_Input` → `CrossTF`, ventana `build..oos1`
wfc	`WFC 1 IS`, `WFC 2 OOS1`, `WFC 3 OOS2` (añadidas)	`WFC_Variants` → `WFC_Build`, `WFC_OOS1`, `WFC_OOS2`

`MC Trades` y las tareas auxiliares del donante (`Clear databanks`, `Go To Task`) no entran: ningún paso las corre, y un `Go To Task` activo hace que `action=start` no acabe nunca. **Sobrescribe** un proyecto que ya tenga ese nombre en esa instalación.

Cómo se lee el resultado

Salida real del builder, 2026-09-25 (la línea nueva es `workflow`):

```
test_workflow on SQX_w1
  template  ~/Desktop/SQX_w1/user/settings/StrategyTemplates/authored/keltnerUpperCrossUp.sqx
  workflow  + CrossTF, WFC 1 IS, WFC 2 OOS1, WFC 3 OOS2: todas las tareas del workflow, activas sólo
CONSTRUCCION y OOS. CrossTF va a build..oos1; las WFC se precian con sqx.projects.wfc
  doctrina  H1 en todas las tareas, sesión XAUUSD_ftmo, salida por barras 2–24
  to disk   Results, Last generation, Initial population, Strategies to improve
  ⚠ PROVISIONAL: spread_is, spread_oos, commission, slippage_is, slippage_oos, swap_long, swap_short
```

Salida real de `stage --step spp` sobre ese proyecto. `●` corre en el próximo `start`, `·` se salta:

```
test_workflow: el proximo `action=start` corre spp
  · CONSTRUCCION
  · OOS
  · Retest Markets - Family
  · MCR 1 Bar
  ...
  · MCR 8 Stress
  ● SPP IS
  ● SPP OOS
  · WFM
  · CrossTF
  · WFC 1 IS
  · WFC 2 OOS1
  · WFC 3 OOS2
```

Lo que dice el log de SQX al lanzar el paso `wfc` (□ 2026-09-25, conductor, 3 madres):

```
CONSTRUCCION : SKIPPED, inactive task
...
CrossTF : SKIPPED, inactive task
WFC 1 IS : Task finished in 16.52 s.
WFC 2 OOS1 : Task finished in 7.03 s.
WFC 3 OOS2 : Task finished in 5.60 s.
Project finished
```

Lo que hay que mirar: que las `●` sean justo las del paso que vas a correr. Si `CONSTRUCCION` sale con `●` cuando no toca, el `start` vuelve a construir desde cero y pisa `Results`.

Un ejemplo completo

Un paso cualquiera, el SPP, sobre el proyecto del workflow en el custodio:

```
python3 -m core.assets XAUUSD
python3 -m sqx.projects.spp XAUUSD --cfx ~/Desktop/SQX_w2/user/projects/XAUUSD_keltner_H1/project.cfx \
    --input OOS                          # el databank con los supervivientes que llegan aquí
# … imprime `el proximo `action=start` de XAUUSD_keltner_H1 corre solo `spp``
bin/sqx-worker.sh --role custodian start
python3 -c "from core import worker; worker.call('-project action=stop
name=XAUUSD_keltner_H1','custodian')"
python3 -c "from core import worker; worker.call('-project action=start
name=XAUUSD_keltner_H1','custodian')"
# mientras corre, sólo `-project action=status`; el final es `Project finished` en el log de SQX
bin/sqx-worker.sh --role custodian stop
```

El `stop` antes del `start` es obligatorio: en un proyecto que ya corrió, un segundo `start` sin él no hace nada, y no avisa.

Qué NO te dice

- Que la tarea esté bien configurada: eso lo dice cada configurador en su salida. `stage` sólo enciende y apaga.

- Qué databank lee cada paso de la mitad del embudo: las MCR, el SPP y la WFM leen lo que les pases con `--input`, y eso lo decides tú según lo que haya sobrevivido.
- Nada sobre el `oos2`: la WFM y el WFC lo leen igual en un proyecto que en otro.

Si algo falla

- `<proyecto> no lleva la(s) tarea(s) …` — no es un proyecto de workflow. Créalo con `--workflow` o usa el proyecto suelto sabiendo qué tareas tiene activas.
- `el custodian tiene este proyecto abierto…` — para la instalación con `bin/sqx-worker.sh --role custodian stop` antes de tocar el `.cfx`.
- `⚠ <proyecto> no es un proyecto de workflow` (en un configurador) — ha escrito su tarea pero no ha tocado qué está activo. Míralo antes de lanzar.

6. Quitar estrategias descartadas — aplicar un veredicto dentro de SQX

> Para **escribir** el veredicto con un filtro o con un nombre suelto: `04-sqx-plantillas-y-proyectos.pdf › 27-curar`. Esta página es el paso que lo aplica.

Qué pregunta responde

Ninguna. Este no analiza nada: **ejecuta una decisión que ya tomaste**. Coge el `verdict.csv` que produjo un análisis y saca de SQX las estrategias marcadas como `DESCARTAR`, para que la databank de trabajo quede sólo con las que siguen vivas. **Las descartadas se borran**; antes queda anotado qué había, con la identidad y las métricas de cada una. Con `--into` en vez de borrarlas van a otra databank del mismo proyecto, y entonces SQX las vuelve a cargar al arrancar: cuestan RAM.

Es el único comando del proyecto que **cambia lo que contiene una databank**. Cada `.sqx` pesa unos 5 MB; guardar 8.000 descartadas de 10.000 serían 40 GB por una estrategia que nadie va a recuperar. Se anota y se borra (decisión del dueño, 2026-09-23).

Cuándo lo usas, y cuándo no

Lo usas cuando ya has leído el informe, estás de acuerdo con el veredicto, y quieres que la databank refleje esa decisión sin ir estrategia por estrategia en la interfaz.

No lo usas para "limpiar" una databank por criterios que no estén en un `verdict.csv`. El fichero es la traza de por qué se movió cada una; sin él no queda constancia de nada.

No lo usas con la interfaz de SQX abierta. Se niega a funcionar, y hace bien: las bases de datos del master están bloqueadas mientras la interfaz las tiene, y un cambio hecho por debajo de una instancia en marcha se pierde en silencio en la siguiente sincronización.

Antes de empezar

1. **Para la instalación.** `--role master` exige la interfaz cerrada; `--role conductor` o `--role custodian`, el worker parado (`bin/sqx-worker.sh --role custodian stop`). Se niega si está viva: SQX tiene los registros en memoria y reescribe los ficheros desde ella, así que un movimiento hecho por debajo se deshace en la siguiente sincronización.
2. **La databank de destino no hace falta crearla a mano.** Se crea sola si no existe, y el arranque siguiente la recoge — verificado en el custodio el 2026-09-23. La nota anterior, que decía que había que crearla desde la interfaz, no se sostiene.
3. Ten a mano la ruta del `verdict.csv`. Si trae la columna `identity` (la escribe `verdict.py` y cualquier módulo que pueda calcularla), el comando comprueba que el fichero que va a mover es la estrategia que se juzgó, y no otra que SQX haya puesto bajo el mismo nombre. Esa comprobación se hace ya en seco, con la instalación viva, porque sólo lee.

> **Actualizado el 2026-09-23.** Antes esto llamaba a `-databank action=move` con la lista de nombres. **Ese selector no funciona**: por la API HTTP el nombre se corta en su primer espacio, y un `sqcli` de un disparo ni siquiera carga los registros — los dos informan de éxito y no hacen nada, y `move` sin selector se lleva la databank entera. Ahora mueve los ficheros `.sqx` con la instalación parada, y es el arranque siguiente el que hace que la memoria coincida. Medido

> y escrito en `knowhow/databanks/`.

Cómo se ejecuta

Siempre dos veces: primero en seco, y sólo después de leer lo que sale, en serio.

```
# En seco. No toca absolutamente nada. Puedes lanzarlo con SQX abierto
python3 -m sqx.curate.apply_verdict --project XAUUSD --databank RetestMarkets \
    --verdict ~/Desktop/AlgoData/reports/XAUUSD/RetestMarkets/2026-09-08/randomentry/verdict.csv \
    --role custodian

# En serio. Con la instalación parada
python3 -m sqx.curate.apply_verdict --project XAUUSD --databank RetestMarkets \
    --verdict ~/Desktop/AlgoData/reports/XAUUSD/RetestMarkets/2026-09-08/randomentry/verdict.csv \
    --role custodian --apply
```

flag	obligatorio	qué hace
`--project`	sí	proyecto en esa instalación
`--databank`	sí	databank de la que salen
`--verdict`	sí	ruta del `verdict.csv`
`--into`	no	databank del mismo proyecto a la que moverlas **en vez de borrarlas**. SQX la carga al arrancar, así que cuestan RAM
`--role`	sí	la instalación que tiene el proyecto: `master`, `conductor` o `custodian`. **No tiene valor por defecto**: sin él el comando se niega, para que nunca se toque el maestro por olvido
`--apply`	no	**sin él no se toca nada.** Con él, se borra

Instantáneo: mueve ficheros, no arranca SQX. Lo que tarda es el arranque posterior, que es el que hace que la memoria coincida con lo que quedó en disco.

Y ese arranque es parte del procedimiento, no un detalle. Hasta que no lo hagas, SQX sigue teniendo en memoria lo que había antes:

```
bin/sqx-worker.sh --role custodian start
python3 -c "from core import worker; print(worker.call('-databank action=list project=
<P>','custodian'))"
```

El recuento que imprime ahí es la única verificación que vale: es lo que va a leer la tarea siguiente.

Qué produce

Tres cosas, en este orden:

1. **El registro**, en `~/Desktop/AlgoData/reports/<proyecto>/<databank>/<fecha>/curate/`, junto al veredicto: `before-HHMMSS.csv` con nombre, identidad, tamaño, veredicto **y el motivo** de cada estrategia que había en disco, y `rejected-HHMMSS.csv` con la fila de métricas de cada descartada, con el motivo como primera columna, si la databank tiene métricas exportadas. El motivo es la columna `reason` del veredicto: el filtro que no pasó

(`failed: sharpe_ratio_is > 0.93`), `dropped by name`, o el nombre del fichero de veredicto cuando el módulo que juzgó no escribió motivo — entonces ese fichero es la prueba. Kilobytes. Copiar los `.sqx` costaría 50 GB por corte con 10.000 estrategias.
2. **El borrado** de los `.sqx` descartados (o el movimiento, con `--into`).
3. **Un** `manifest.json` junto al registro, diciendo qué veredicto lo produjo y cuántas se quitaron.

Cómo se lee el resultado

En seco, te dice cuántas se quedan, cuántas se mueven, y en cuántas ha podido comprobar la identidad. Salida real del 2026-09-23 en el conductor:

```
SQX_w1 · Retester/Results: 66 strategies on disk
  the verdict drops 32, keeping 34
  identity checked on 32 of 32

dry run. Re-run with --apply, with the conductor stopped.
```

Si un fichero no es la estrategia juzgada, se para ahí y no mueve nada. Probado cambiando un fichero por otro bajo el mismo nombre:

```
  ✗ 1 carry a different strategy than the verdict judged: Strategy 11.10.85(1). The databank
  changed under that name since the verdict was written; re-export and judge again. Nothing was moved.
```

En serio, las líneas que importan son estas:

```
  metrics of the dropped kept in /home/sergioguslw/Desktop/AlgoData/reports/Retester/Results/2026-09-23/curate/rejected-073842.csv
  what was here is listed in /home/sergioguslw/Desktop/AlgoData/reports/Retester/Results/2026-09-23/curate/before-073842.csv
Results: 34 → 33   deleted 1
start the conductor and the sync from files makes memory match: the next task reads 33.
```

Y tras arrancar, SQX contó **33**. Ese es el número que lee la tarea siguiente.

Los números se cuentan **volviendo a mirar los ficheros en disco**, no creyéndose la respuesta de SQX. Si no cuadran con lo que decía el veredicto, el comando se para y te dice contra qué lista comparar el directorio.

Qué NO te dice

- **No te dice que la decisión fuera buena.** Aplica un fichero. La calidad del veredicto es la del análisis que lo escribió.
- **No conserva las operaciones de las descartadas.** Con el fichero se van sus trades y su XML. Lo que queda de la población completa son sus métricas en `rejected-HHMMSS.csv`, que es lo que leen los estudios de población; un estudio sobre trades de una estrategia descartada ya no es posible. Es la contrapartida de no guardar 40 GB.

Si algo falla

- **"the master is running"** — la interfaz de SQX sigue abierta. Ciérrala del todo y repite.
- **Los números no cuadran al final** — el comando se para solo y no ejecuta nada más. Compara el directorio con `before-HHMMSS.csv` antes de tocar nada.

- **"N had no file"** con los números cuadrando — el veredicto nombraba una estrategia que ya no estaba, porque las métricas se exportaron antes de un corte anterior. Lo demás se aplicó bien; vuelve a exportar las métricas antes del siguiente veredicto. En seco ya lo avisa: `⚠ 1 named by the verdict are not here`.
- **"carry a different strategy than the verdict judged"** — la databank cambió bajo ese nombre desde que se escribió el veredicto. No se movió nada: vuelve a exportar las métricas, vuelve a juzgar, y aplica el veredicto nuevo.

27. Curar una databank — escribir el veredicto con un filtro o con un nombre

Qué pregunta responde

Ninguna: **ejecuta lo que tú decides.** Tienes una databank con miles de estrategias y un criterio en Python, o una estrategia concreta que has visto en tu tabla y quieres fuera. Esto convierte esa decisión en un `verdict.csv`, que `04-sqx-plantillas-y-proyectos.pdf › 06-mover-estrategias` aplica dentro de SQX.

Dos modos: `--keep` con un filtro sobre las métricas exportadas, y `--drop` con uno o varios nombres.

Cuándo lo usas, y cuándo no

Lo usas entre dos tareas de SQX: la generación acabó, exportaste las métricas, y quieres que la tarea siguiente lea sólo las que pasan tu corte. O cuando en tu tabla ves una que no quieres y la apartas por su nombre. Se puede repetir tantas veces como cortes quieras: cada corte deja su propio `verdict-HHMMSS.csv`, y al aplicarlo, su propio registro de lo que quitó.

No lo usas como sustituto de un análisis. Un filtro sobre métricas es un corte, no una prueba de robustez; para eso están `05-cribado-oos.pdf › 02-filtros`, `08-montecarlo.pdf › 07-montecarlo` y compañía, que escriben su propio veredicto.

Antes de empezar

1. **Las métricas exportadas.** `verdict.py` juzga `~/Desktop/AlgoData/metrics/<proyecto>/<databank>/metrics.csv` y nada más: `python3 -m sqx.export.export_metrics --project P --databank Results`. Si el CSV es viejo, el veredicto es viejo.
2. **Los ficheros `.sqx` en disco** en la instalación que nombras con `--role`: de ahí lee la identidad de cada estrategia. Una databank en `Auto-sync never` tiene el directorio vacío; el comando lo avisa (`identity from file on 0 of N`) y el paso siguiente se negará.
3. **No hace falta parar nada**: sólo lee. Para aplicar el veredicto sí (`04-sqx-plantillas-y-proyectos.pdf › 06-mover-estrategias`).

Cómo se ejecuta

```
# qué columnas puedes usar en el filtro
python3 -m sqx.curate.verdict --project Retester --databank Results --role conductor --columns

# un filtro: lo que NO lo cumple se marca DESCARTAR
python3 -m sqx.curate.verdict --project Retester --databank Results --role conductor \
    --keep "sharpe_ratio_is > 0.9"

# una estrategia concreta, por su nombre tal y como lo muestra SQX
python3 -m sqx.curate.verdict --project Retester --databank Results --role conductor \
    --drop "Strategy 10.16.68"
```

flag	obligatorio	qué hace
`--project`	sí	proyecto tal y como aparece en SQX
`--databank`	sí	databank que se juzga
`--role`	no	instalación que tiene los ficheros: `master` (por defecto), `conductor` o `custodian`
`--keep`	uno de los tres	expresión de pandas sobre las columnas de `--columns`. `and`, `or`, `>`, `<=`, `==`
`--drop`	uno de los tres	uno o más nombres, entre comillas porque llevan espacio
`--columns`	uno de los tres	imprime los nombres de columna y no escribe nada

Los nombres de columna son los de la exportación en minúsculas y con guiones bajos: `Net profit (OOS)` es `net_profit_oos`, `# of trades (IS)` es `n_of_trades_is`. Instantáneo; no toca SQX.

Qué produce

`~/Desktop/AlgoData/reports/<proyecto>/<databank>/<fecha>/curate/verdict-HHMMSS.csv`, uno por ejecución, nunca sobrescribe. Columnas `strategy`, `verdict` (`MANTENER` o `DESCARTAR`), `reason` (el filtro que falló, o `dropped by name`) e `identity` (SHA-256 de la definición de la estrategia, leído del fichero).

Cómo se lee el resultado

Salida real del 2026-09-23 sobre el conductor:

```
$ python3 -m sqx.curate.verdict --project Retester --databank Results --role conductor --keep
"sharpe_ratio_is > 0.9"
Retester/Results: 66 judged, 34 kept, 32 DESCARTAR
  identity from file on 66 of 66
(/home/sergioguslw/Desktop/SQX_w1/user/projects/Retester/databanks/Results)
  /home/sergioguslw/Desktop/AlgoData/reports/Retester/Results/2026-09-23/curate/verdict-072117.csv
apply it: python3 -m sqx.curate.apply_verdict --project Retester --databank Results --verdict ... --role
conductor   # then --apply, stopped
```

La primera línea son los números; la segunda te dice si la identidad se pudo leer de todos los ficheros (si no es `N of N`, la databank no está en disco); la última es el comando que sigue.

Desde el 2026-09-28, `apply_verdict` exige `--role`. Antes, si no lo ponías, apuntaba al **maestro**: el único comando que borra estrategias podía tocar tu install por olvidar un flag. Ahora falla con `the following arguments are required: --role` y hay que decir `conductor`, `custodian` o, a propósito, `master`. El `--role` de `verdict.py` (que solo lee) no cambia.

Desde la ventana no hace falta nada de esto: filtras el databank y pulsas «Continuar workflow» (capítulo 69), que escribe el veredicto, copia antes los `.sqx` a `AlgoData/projects/discards/`, llama al mismo `apply_verdict` con el worker del proyecto y lanza la tarea siguiente. Nunca sobre el maestro.

Un ejemplo completo

El ciclo entero, tal y como se corrió el 2026-09-23 sobre `Retester/Results` en el conductor:

```
$ python3 -m sqx.curate.verdict --project Retester --databank Results --role conductor --drop "Strategy
10.16.68"
Retester/Results: 34 judged, 33 kept, 1 DESCARTAR

$ bin/sqx-worker.sh --role conductor stop
$ python3 -m sqx.curate.apply_verdict --project Retester --databank Results --verdict .../verdict-
073842.csv --role conductor --apply
SQX_w1 · Retester/Results: 34 strategies on disk
  the verdict drops 1, keeping 33
  identity checked on 1 of 1
  metrics of the dropped kept in .../reports/Retester/Results/2026-09-23/curate/rejected-073842.csv
  what was here is listed in .../reports/Retester/Results/2026-09-23/curate/before-073842.csv
Results: 34 → 33   deleted 1
start the conductor and the sync from files makes memory match: the next task reads 33.

$ bin/sqx-worker.sh --role conductor start      # SQX cargó 33 estrategias en Results
```

Qué NO te dice

- **No te dice que el corte sea bueno.** Un filtro sobre métricas dentro de muestra selecciona lo que mejor se ajustó al pasado; si eso sobrevive fuera de muestra lo responden los análisis, no esto.
- **No sabe nada de RAM ni de SQX.** El veredicto es un CSV. Quien saca las estrategias de la memoria de SQX es `apply_verdict` con la instalación parada y el arranque siguiente.

Si algo falla

- **`no metrics export at ...`** — no has exportado las métricas de esa databank, o la exportaste con otro nombre. `sqx.export.export_metrics` primero.
- **`not in the export: Strategy X`** — ese nombre no está en el CSV. Mira si lleva `(1)` al final: SQX renombra así cuando dos estrategias colisionan.
- **`identity from file on 0 of N`** — la databank no está en disco. `04-sqx-plantillas-y-proyectos.pdf › 06-mover-estrategias`, sección «Antes de empezar», explica el `Auto-sync never`.
- **Un error de pandas al evaluar `--keep`** — una columna mal escrita. `--columns` te da los nombres exactos.

17. Pipeline — mete cien estrategias, vuelve al cabo de unos días y lee los veredictos

Qué pregunta responde

El estudio de robustez de una estrategia son siete pasos encadenados: leer su SPP, diseñar las variantes, fabricarlas, correrlas en SQX, recoger los resultados, medir si la superficie aguanta fuera de muestra, y dictar un veredicto. Hacer eso a mano para **una** estrategia es una tarde. Para las ~100 madres de un databank son días de máquina.

Este comando lo hace solo. Le dices el proyecto, te vas, y cuando vuelves cada estrategia tiene su veredicto escrito, y los gigas de variantes que hicieron falta para llegar a él ya no ocupan disco.

Y mientras corre **te deja mirar**: hay un fichero por estrategia que dice, en todo momento, en qué paso va, por qué porcentaje y qué está haciendo. No tienes que adivinar si se ha colgado.

Cuándo lo usas, y cuándo no

Úsalo cuando ya has corrido los **SPP en SQX** y los tienes exportados: ése es el punto de partida de la cadena, y es lo único que tienes que hacer tú a mano.

No lo uses para una sola estrategia que estás mirando con lupa. Para eso corre cada etapa por separado — están pensadas para poder lanzarse sueltas, y cada una te enseña su informe. El pipeline es para cuando no quieres mirar.

No es un sustituto de las etapas. No calcula nada por su cuenta: las llama en orden, apunta lo que pasa y se niega a seguir cuando falta algo. Si borras `pipeline/run.py`, todo lo demás sigue funcionando a mano.

⚠ **Hoy solo la primera etapa es de verdad.** Las otras cinco (diseño, fabricación, ejecución, recogida y walk forward correlation) están siendo programadas por otros; mientras tanto, la cadena corre con **marcadores de posición** que escriben cifras inventadas. Sirven para probar que el encadenado funciona, **no para decidir nada**. Todo lo que escriben lleva dentro la marca `"placeholder": true`. Cuando cada módulo real esté listo, se cambia **una línea** en `pipeline/recipe.yaml` y esa etapa pasa a ser la de verdad.

Antes de empezar

- **Los SPP, corridos y exportados.** Tiene que existir `~/Desktop/AlgoData/raw/<proyecto>/<databank>/<fecha>/spp/`. Si no: `bash python3 -m sqx.export.export_spp --project XAUUSD --databank "SPP IS"`
- **Sitio en disco.** Antes de cada estrategia el pipeline mira lo que pesa `AlgoData` y **se para si algo está por encima de su presupuesto**. Es aposta: lo que tiene que frenar una corrida de días es el presupuesto, no el disco lleno — cuando el disco se llena, la etapa que estaba escribiendo ya ha dejado medio fichero a medias. Para ver cómo estás: `bash python3 -m perf.disk.report` Los techos están en `perf/config.yaml`, en `disk.budget_gb`.
- **No toca SQX directamente.** Las etapas que sí lo harán todavía no existen. Hoy puedes lanzarlo con la GUI abierta.

Cómo se ejecuta

```
python3 -m pipeline.run --project XAUUSD --databank SPP_IS
```

flag	obligatorio	qué hace
`--project`	sí	nombre del proyecto tal y como aparece en SQX
`--databank`	no	carpeta del export, **con guiones bajos en vez de espacios**. Por defecto `SPP_IS`
`--strategy`	no	se puede repetir. Por defecto, **todas** las estrategias del export
`--day`	no	fecha del export a leer. Por defecto, la más reciente
`--report-day`	no	carpeta de informes en la que escribe sppUltra. Por defecto, hoy. Si reanudas una corrida al día siguiente, **pon aquí el día en que empezó**, o no encontrará los informes que ya hizo
`--asset`	no	nombre en `assets/`. Por defecto, el del proyecto. De ahí sale el sello de costes provisionales
`--sweep`	no	borra los datos de variantes al terminar cada madre. **Sin él no se borra nada**

Puedes pararlo cuando quieras con Ctrl-C, o apagar el ordenador. Al volver a lanzarlo **continúa donde estaba**: se salta lo que ya estaba hecho y retoma la etapa que quedó a medias.

Qué produce

Una carpeta por estrategia en `~/Desktop/AlgoData/pipeline/<proyecto>/<estrategia>/`:

- **`state.json`** — el registro. En qué paso va, con qué porcentaje, qué dijo cada etapa y cuándo terminó. Pesa kilobytes y **no se borra nunca**, ni siquiera cuando se borran los datos que describe. Es lo que convierte el borrado en algo auditable en vez de en un agujero.
- **`state.json`** guarda además, de cada etapa, lo que esa etapa haya declarado: cuántas variantes salieron, cuántos canarios fallaron, y la **huella del diseño** (`brief_hash`) con la que se fabricaron. Esa huella es la que permite darse cuenta de que el diseño cambió por el camino.
- **`verdict.json`** — el veredicto, con los números en los que se apoya y, si sale `reject`, qué umbral falló y por qué existe ese umbral.
- Los ficheros que va dejando cada etapa.

El informe de la primera etapa va donde siempre: `~/Desktop/AlgoData/reports/<proyecto>/<databank>/<fecha>/`.

Cómo se lee el resultado

Esto es una corrida real sobre `Strategy 17.9.39`:

```
1 estrategias madre en XAUUSD/SPP_IS

[1/1] Strategy 17.9.39
  sppultra   python3 -m studies.breakage.spp.report --project XAUUSD --databank SPP_IS --strategy
Strategy 17.9.39
   | Strategy 17.9.39         proceed  n_eff= 3,381  max 14.64 vs nulo 5.11  vivos=7 congelados=1
   | -> /home/sergioguslw/Desktop/AlgoData/reports/XAUUSD/SPP_IS/2026-09-21
  design     python3 -m pipeline.stubs.placeholder --stage design --work .../Strategy_17-9-39
   | PROGRESS 20 paso 1 de 5 (marcador de posición de design)
   | PROGRESS 40 paso 2 de 5 (marcador de posición de design)
   ...
  verdict    python3 -m pipeline.stages.verdict --work .../Strategy_17-9-39 --brief
.../design_brief_Strategy_17-9-39.json
   | PROGRESS 20 leyendo los números de 4 umbrales
   | PROGRESS 60 13 cifras reunidas
   | PROGRESS 100 proceed
  -> verdict
```

Cada línea con sangría es **una etapa**, y debajo va lo que esa etapa imprime. Las líneas `PROGRESS` son el porcentaje: van a parar al registro mientras la etapa corre, no al acabar.

Si quieres mirar desde otra terminal cómo va, sin tocar nada:

```
cat ~/Desktop/AlgoData/pipeline/XAUUSD/Strategy_17-9-39/state.json
```

```
{ "strategy": "Strategy 17.9.39", "stage": "verdict",
  "stages": {
    "sppultra": { "progress": 100, "status": "-> .../reports/XAUUSD/SPP_IS/2026-09-21",
                  "done_at": "2026-09-21T18:31:13" },
    "build":    { "progress": 100, "status": "paso 5 de 5", "n": 5000, "bytes": 56500000,
                  "done_at": "2026-09-21T18:31:15" } },
  "costs_provisional": true }
```

`progress` nunca baja. Si lo ves parado, la etapa está tardando; si ves que retrocede, es un fallo y el programa se para solo.

`costs_provisional: true` quiere decir que los costes con los que se ha calculado todo son los que trae SQX por defecto, no los reales de tu bróker. Sale de `assets/symbols/XAUUSD.yaml`, y dejará de salir cuando pongas ahí las cifras buenas.

Y el veredicto, cuando falla:

```
{ "verdict": "reject",
  "failed": [ { "number": "wfc.rho", "value": 0.41, "limit": 0.9, "bound": "min",
                "why": "below this the in-sample surface does not order the out-of-sample one" } ] }
```

Los umbrales son tuyos, y cambiarlos es gratis

Están en `pipeline/config.yaml`, bajo `verdict.rules`. Cada regla es un número, un límite y la razón por la que ese límite existe.

No todo número con un límite es un umbral. Los canarios, por ejemplo, no están ahí: un canario fallido no quiere decir que la estrategia sea mala, quiere decir que la medición no vale. Eso es una **puerta**, vive en `pipeline/recipe.yaml` y para la corrida, en vez de producir un veredicto que nadie debería leer. La diferencia es siempre la misma pregunta: ¿el número dice *esta estrategia no da la talla*, o dice *esta medición no significa nada*?

Cambiar un umbral no obliga a repetir nada. El pipeline guarda los números, no los juicios: al volver a lanzarlo, las seis primeras etapas se saltan y solo se vuelve a dictar el veredicto. Así se ve:

```
  sppultra   hecho, se salta
  design     hecho, se salta
  build      hecho, se salta
  ran        hecho, se salta
  collected  hecho, se salta
  wfc        hecho, se salta
  verdict    python3 -m pipeline.stages.verdict ...
   | PROGRESS 100 reject, falla wfc.rho
```

Puedes dictar el veredicto suelto, sin pasar por el pipeline:

```
python3 -m pipeline.stages.verdict \
  --work ~/Desktop/AlgoData/pipeline/XAUUSD/Strategy_17-9-39 \
  --brief ~/Desktop/AlgoData/reports/XAUUSD/SPP_IS/2026-09-21/design_brief_Strategy_17-9-39.json
```

Borrar las variantes, con red

Las 5.000 variantes de una estrategia pesan gigas y no sirven para nada una vez medidas. El pipeline las borra si se lo pides con `--sweep`, o después, a mano:

```
python3 -m pipeline.cleanup --project XAUUSD --strategy "Strategy 17.9.39"
```

```
liberaría      1.204.531.200 B  pipeline/XAUUSD/Strategy_17-9-39/variants
1.204.531.200 B en 1 rutas  (nada borrado: falta --apply)
```

Sin `--apply` no borra nada, solo te dice cuánto liberaría. Y antes de borrar comprueba tres cosas, y si falla cualquiera se niega:

1. Que la etapa de recogida haya **terminado**.
2. Que cada fichero exportado **siga siendo exactamente el que se exportó** — lo vuelve a hashear. Si no coincide: `Strategy 0.0.1: el export ya no coincide con su hash: .../metrics_placeholder.json`
3. Que lo que va a borrar esté **dentro de la carpeta de esa estrategia**. Si el registro propone otra cosa, se para: `Strategy 0.0.1: el registro propone borrar fuera de su carpeta: .../raw/XAUUSD/SPP_IS`

⚠ **La estrategia madre no se borra nunca.** No es una promesa: la madre vive en el databank de SQX y en `raw/`, y el barrido no puede salir de `pipeline/<proyecto>/<estrategia>/`. No hay ruta por la que pueda llegar hasta ella.

Lo borrado queda apuntado en el registro, con su tamaño y la hora. El registro sobrevive; los datos no. Y `python3 -m perf.disk.report` lee esos registros para decirte qué más se puede soltar.

Un ejemplo completo

```
# 1. Mira cómo está el disco antes de empezar
python3 -m perf.disk.report

# 2. Lanza las cinco madres del databank, borrando según termina cada una
python3 -m pipeline.run --project XAUUSD --databank SPP_IS --sweep

# 3. Al día siguiente, mira los veredictos
grep -h '"verdict"' ~/Desktop/AlgoData/pipeline/XAUUSD/*/verdict.json
```

Si se corta a mitad, se relanza el mismo comando y sigue donde estaba.

Qué NO te dice

- **No te dice que la estrategia sea buena.** Dice que pasó unos umbrales que has puesto tú. Los números de los que salen esos umbrales están en `verdict.json`, y son lo que hay que mirar.
- **Hoy no te dice casi nada**, porque cinco de las siete etapas son marcadores de posición con cifras inventadas. Un `proceed` de hoy no significa nada; lo que hoy está probado es el encadenado.
- **`costs_provisional: true` invalida cualquier cifra de dinero.** Mientras esté ahí, los costes son los de por defecto de SQX.
- **No vigila SQX.** Si SQX se queda colgado durante una etapa, el pipeline se queda esperando; el registro seguirá diciendo el último porcentaje que le llegó.

Si algo falla

`AlgoData 7.2 GB de 60 GB. snapshots 4.8/2 GB. Libera espacio o sube el presupuesto` El pipeline se ha negado a empezar porque una rama de `AlgoData` está por encima de su techo. Mira qué se puede soltar con `python3 -m perf.disk.report`, o sube el número en `perf/config.yaml`. Es la puerta que existe para que una corrida de días no acabe con el disco lleno.

`sppultra: falta la entrada raw/XAUUSD/SPP_IS` No está el export. Córrelo (ver *Antes de empezar*) y relanza.

`design: terminó sin escribir pipeline/XAUUSD/.../design.json` La etapa acabó sin error pero no dejó lo que tenía que dejar. El pipeline no la da por buena: un código de salida cero no es prueba de nada.

`build: progress went 80 -> 40, it is monotonic` Un fallo de la etapa, no tuyo. Apúntalo y dilo.

`sppultra: design_brief_….json ha cambiado desde que se registró (brief_hash d081df20… -> e6d1be4a…)` El diseño de las variantes ha cambiado **después** de haberlas fabricado. Todo lo que viene detrás estaría midiendo un diseño contra los números de otro, y eso no da error: da resultados que parecen buenos. Por eso se para. O borras la carpeta de esa estrategia en `~/Desktop/AlgoData/pipeline/` para rehacerla entera, o recuperas el brief anterior.

`ran: canaries_failed = 3. un canario fallido significa que las variantes que corrió SQX no son las que se escribieron…` No es que la estrategia sea mala: es que el experimento no vale. Las variantes que volvieron de SQX no son las que se le dieron — renombrado en colisión, un databank que se sincronizó por medio, algo así. Se para ahí aposta, para no gastar horas midiendo algo que no significa nada.

`assets/symbols/XAUUSD.yaml: spread, commission sin valor acordado` El activo no tiene costes decididos y no se arranca sin ellos. Corre `python3 -m core.assets XAUUSD` para ver qué falta. Ojo: un valor **provisional** no bloquea — es una decisión tuya, y el registro la sella con `costs_provisional: true`.

Comprobar que el pipeline sigue siendo un pipeline

```
python3 -m pipeline.verify.selftest
```

Monta una estrategia de mentira, la corre entera, y comprueba dos cosas: que el progreso se escribe **mientras** la etapa corre y nunca baja, y que si matas el proceso a mitad el registro no se corrompe y al relanzar continúa donde estaba. Tarda unos 15 segundos y termina en `0 problemas`.

80. Autopiloto — el workflow de un proyecto de punta a punta, sin pararse

> **Desde el 2026-10-12** (encargo 65): el 18.5 está apagado (ni se corre ni se juzga) y los pasos 14 y 18 son columnas, sin regla: no cortan.
>
> **Desde el 2026-10-11** la cadena va del 6 al 21 y el 26 (cuando se dan sus cuatro datos): los pasos 22, 23, 24 y 25 ya no corren. Ningún paso se renumera.

Qué pregunta responde

¿Puede correr el workflow entero solo, sin que nadie (ni tú ni Claude) tenga que mirar entre paso y paso? El autopiloto lanza cada tarea de SQX y cada análisis de Python en el orden de `WORKFLOW.md`, y en cada paso que juzga (8, 10, 12, 14, 16) **decide él**: lee todos los números que han escrito los estudios, aplica tus criterios, borra en SQX lo que no pasa y sigue.

Cada estrategia sale de un paso con una de tres notas: **Pasa**, **Limbo** o **No pasa**. Las del limbo siguen, marcadas. Un paso sin criterios deja pasar a todas.

Cuándo lo usas, y cuándo no

Úsalo cuando quieras que el proyecto avance del paso 6 al 16.5 sin intervenir. En el paso 8, de las que pasan las reglas, solo siguen las **medoides de los 4 grupos de estructura mayores** (la estrategia de cada grupo cuyas barras de entrada más se parecen a las del resto del grupo); el resto se borra del databank OOS con la razón «no es la medoide de su grupo».

La regla que decide la ventaja en el paso 8 es **una sola z** (dueño, 2026-10-12): la z de la estrategia frente a su mono con horas tiene que llegar a $\Phi^{-1}(1 - 0{,}05/B)$, con B los lotes cribados en oos1 en la celda de tu proyecto contando el actual (1,645 con B = 1; 1,960 con 2; 2,241 con 4). El umbral no está escrito en `criteria.yaml` (solo `alpha: 0.05`): lo calcula el juez y el ledger apunta B y el umbral usado. Las medoides se eligen después de esa z. R3, la t de R5c, R4a, R5, R5b, R8 y R9b ya no juzgan (las cinco últimas siguen como columnas del scorecard). El scorecard añade `p_fdr` (probabilidad de no ser azar según la nula empírica del lote) y ordena por ella; todavía no corta (fila 9 del encargo 65).

No hace los pasos 1-5 (idea, plantilla, proyecto). Desde el 2026-10-01 **sí hace el 16.5**: fabrica y retestea las variantes de cada superviviente, una detrás de otra. Se para antes del 17: el 17, el 18, el 18.5 y el 19 leen el `oos2` o escriben en el ledger, y esos los lanzas tú.

Desde el 2026-10-04 el autopilot arranca con `ALGO_AUTONOMOUS=1` (decisión del dueño): la puerta del `oos2` le queda cerrada a todo paso que la política no nombre (el 8, por ejemplo); el 17, 18, 18.5, 19 sí pasan. El paso 20 no está en la política: su mitad SPA no se lee.

Si un corte deja **cero supervivientes** (lo normal en una población de entrada al azar), la corrida acaba en `FIN` y el resumen dice en qué paso; no es un `FALLO`. Si SQX aborta (línea de error en su log) o el proceso muere, la espera falla con un mensaje en vez de colgarse. Cada corte del juez deja una fila en el ledger con el hash de `criteria.yaml`.

Antes del autopiloto: los tres expertos

Los pasos 1 a 6 son los que más pesan: la idea, la plantilla y los building blocks. Los hace un experto en cada uno, sin escatimar tokens (dueño, 2026-10-01):

paso	experto	qué entrega
1 · idea	`ideaExpert`	`AlgoData/ideas/<activo>/<fecha>-<nombre>.md` : tres ideas exactas, con su mecanismo, su coste y su evidencia **solo en el tramo de construcción**; tú eliges una
2-3 · bloques y plantilla	`templateArchitect`	la plantilla en `AlgoData/templates/library/<nombre>/` , comprobada bloque a bloque contra la idea
6 · building blocks	`buildingBlocksExpert`	una paleta en `sqx/blocks/palettes/<nombre>.yaml` : qué condiciones puede combinar el builder con la idea, y por qué

La paleta se escribe en la tarea Build del proyecto con el worker parado:

```
python3 -m sqx.projects.buildingblocks --project Test_XAUUSD_ejemplo_H1 --palette ruptura_base --role custodian
```

Responde cuántos bloques quedan encendidos y si las condiciones están dentro de tu banda (90-170). Dos paletas separadas por coma (`--palette tendencia_base,reversion_base`) se aplican como su **unión**: un bloque entra si está en cualquiera de las dos, con el peso mayor. Los niveles stop/limit y los bloques de reloj o calendario no se encienden nunca, los pida la paleta o no (regla 15), y el comando dice cuáles dejó apagados. Con `--count` imprime además el recuento completo de la tarea Build (cap. 28, «Sin plantilla: Simple Strategy»). Sin este paso, el hueco aleatorio sortea entre las ~436 condiciones del donante.

Antes de empezar

- El proyecto existe, creado con `sqx.projects.builder --workflow` , en un worker (nunca en el maestro).
- El worker está **parado y libre**: el autopiloto se niega si está encendido o lo tiene otra sesión.
- Los criterios están en `pipeline/autopilot/criteria.yaml` . El paso 8 declara `medoids: 4` .

Cómo se ejecuta

Mira primero qué haría, sin correr nada:

```
python3 -m pipeline.autopilot.run --project Test_USDJPY_donchianUpperCrossUp_H1 --plan
```

Y luego en serio:

```
python3 -m pipeline.autopilot.run --project Test_USDJPY_donchianUpperCrossUp_H1
```

flag	obligatorio	qué hace
`--project`	sí	el proyecto, tal como sale en `registry.csv`
`--plan`	no	solo enseña los pasos que correría y dónde se pararía
`--restart`	no	vuelta a la ruta antigua: SQX arranca y para en cada paso. Sin esta opción, **una sola sesión de SQX para todos los pasos, sin reiniciar entre ellos** (abajo)
`--smoke`	no	**prueba funcional de la cadena**, solo en proyectos `Test_` (en otros se rechaza). Solo juzgan las reglas sobre hechos legibles (que haya dato, que sea sensato, operaciones por año), así que el corte por medoides siempre deja a alguien vivo; `criteria.yaml` no se toca. Sirve para ver que todos los pasos enlazan, no para filtrar. Por ejemplo, 1.000 estrategias simples en NIKKEI225 M30 no dejaron ninguna viva en el paso 8 con las reglas reales y la prueba acababa ahí. Con `--smoke` el MC Retest del paso 13 tolera más celdas dudosas (con diez estrategias, 5 de 70 celdas lo paraban); las celdas se excluyen igual y quedan anotadas
`--own-log`	no	el log reciente del worker es tuyo (acabas de usarlo): no esperes sus 15 minutos

Toca SQX. Con `--restart`, arranca y para el worker en cada paso de SQX: ~27 s al arrancar y 90-230 s al parar y exportar, en cada paso. Sin ella (lo normal), una vez al principio y una al final. Medido en `Test_NIKKEI225_M30_simpleL` (pasos 6 al 25 de entonces, 2026-10-05): 230,2 min al principio, 214,5 min con la ruta de reinicios mejorada y 160,1 min con una sola sesión.

Desde Claude: `/autopilot`. Lo lanza en segundo plano y solo lee el resumen, nunca el log de SQX. Por eso gasta pocos tokens.

Sin reinicios (lo normal)

```
python3 -m pipeline.autopilot.run --project Test_XAUUSD_donchianUpperCrossUp_H1
```

Abre el custodio **una vez**, en modo ventana pero sin pantalla (`bin/sqx-worker.sh --role custodian --gui start` : SQX dibuja su ventana en una pantalla invisible, nadie la ve ni la toca). Desde ahí habla con SQX por la misma puerta que usa su propia ventana: lanza solo las tareas de cada paso, cambia la configuración de una tarea, corta un databank y lo guarda en disco, todo con SQX abierto. Entre una tarea y la siguiente pasan segundos, no minutos.

lo que hace	cómo, con SQX abierto
elegir qué tareas corren	manda la configuración del proyecto con solo esas activas al arrancar
configurar una tarea	el configurador escribe una **copia** del proyecto y se le pasa a SQX la tarea cambiada (el `.cfx` de SQX no se toca a mano: regla 4)
cortar un databank (el juicio)	SQX quita las estrategias de su memoria y luego se guarda en disco: memoria y disco quedan iguales
exportar	se guarda en disco y se exporta de los ficheros, como tras una parada

Salida real del 2026-10-01 (`Test_XAUUSD_donchianUpperCrossUp_H1` , 128 construidas → 5):

```
| - | arranque | 29 | sesión GUI del custodio |
| 11 | sqx | 271 | CrossTF: 0 → 20 |
| 13 | sqx | 1565 | MCR 1 Bar: 0 → 5 · … · MCR 8 Stress: 0 → 5 |
| 15 | sqx | 587 | SPP IS: 0 → 5 · SPP OOS: 0 → 5 |
| - | parada | 46 | parada y export final |
```

El OOS de 128 estrategias tardó **11 s**; antes, solo arrancar y parar el worker ya costaba unos 2-4 minutos por paso. Del 4 al 19 la run entera tardó 128,7 min, y casi todo fue cálculo de SQX: variantes 35 min, MC Retest 26 min, WFM 18 min, build 10-16 min, SPP 10 min.

Qué ves: la sesión aparece en tu panel como el custodio encendido. **Qué no hace:** con la sesión abierta, las órdenes de `sqcli` al custodio no funcionan (SQX contesta «CLI not ready»); por eso solo el autopiloto la usa. El build termina por **tiempo** (`time-limit` , los minutos de `databank.caps`): si el databank se llena antes, se queda con las mejores y sigue hasta el tiempo.

Qué produce

Una carpeta por cada vez que se corre, `~/Desktop/AlgoData/autopilot/<proyecto>/<fecha-hora>/` :

archivo	qué es
`estado.txt`	una línea: en qué paso va ahora mismo
`resumen.md`	una fila por acción: paso, tipo, segundos y qué hizo; al final, dónde se paró
`fallo.md`	solo si algo falla: el paso, el motivo y las últimas líneas del error
`plan.json`	lo que planeó correr
`tiempos.md`	**dónde se fue el tiempo**: una fila por acción, con los segundos repartidos en columnas (abajo), el total, «SQX x min · todo lo demás y min» y las doce fases más largas. Se reescribe tras cada acción
`fases.csv`	la materia prima de `tiempos.md` : una fila por fase cronometrada (cuándo empezó, segundos, fase, detalle, proceso)
`paso-<n>/hechos.parquet`	**todos los números** de los estudios de ese paso, una fila por estrategia y número
`paso-<n>/verdict.csv`	la nota de cada estrategia (Pasa / Limbo / No pasa), si se quedó y por qué

Cómo se lee `tiempos.md` : `sqx` es SQX calculando de verdad; `arranque` y `parada` son encender y apagar el worker; `export` es sacar los databanks a disco después de parar; `sync` es que SQX sincronice con los archivos; `copias` son las copias de seguridad del proyecto; `resto` es lo demás (Python, esperas). Mide primero `SQX x min · todo lo demás y min` : si «lo demás» es grande, el cuello no es SQX. En una prueba de 10 supervivientes (230 min del paso 6 al 25 de entonces), SQX calculó el 56 %.

Borra estrategias del databank que lee la siguiente tarea de SQX (en el paso 8, `OOS`). Antes de borrar, `curate` deja la lista de lo que había en `reports/<proyecto>/OOS/<día>/curate/` .

Cómo se lee el resultado

`resumen.md` dice, paso a paso, cuántas estrategias entraron y cuántas salieron. Por ejemplo, la línea del juicio del paso 8 será del estilo `OOS: 60 → 4 por medoides` .

Para escribir un criterio necesitas saber qué números hay. Esta orden los lista, con cuántas estrategias lo tienen, su mediana, su mínimo y su máximo:

```
python3 -m pipeline.autopilot.facts --project Test_USDJPY_donchianUpperCrossUp_H1 --step 8 --grep gate.
```

Salida real (2026-10-01):

```
                                    count      median         min         max
key
gate.scorecard.degradacion_value      276      0.7280      0.0070      2.3325
gate.scorecard.estaticas_value        296  15766.8999 -16655.2598  41052.2305
gate.scorecard.familia_value          189      0.1734      0.0170      0.4203
gate.scorecard.forma_value            190      1.7351      0.6008      2.9440
gate.scorecard.mono_p                 190      0.1747      0.0170      0.6407
gate.scorecard.sanidad_value          300    327.0000    250.0000    660.0000
gate.scorecard.survives               300      1.0000      0.0000      1.0000
```

Un criterio en `criteria.yaml` se escribe así:

```
"8":
  cut: OOS
  rules:
    - {fact: gate.scorecard.mono_p, pass: "<= 0.05", limbo: "<= 0.10"}
    - {fact: gate.scorecard.forma_value, near_median: 0.2, limbo_near_median: 0.5}
```

La primera regla es un umbral. La segunda dice: pasa si está a menos del 20 % de la mediana, y queda en limbo hasta el 50 %. La mediana se calcula sobre las estrategias del databank que se va a cortar. La nota de una estrategia es **la peor de sus reglas**. Si a una estrategia le falta el número de una regla, queda en limbo en esa regla.

El paso 12 (cross timeframe) juzga madres, no hermanas. Cada madre tiene sus números por timeframe y por papel: `crossTF.cells.scaled.H4.sharpe` es el Sharpe de su versión escalada a H4, `crossTF.cells.unscaled.H4.pf` el PF sin escalar en H4, `crossTF.cells.control.H4.p` el control de la escalada a H4, y `..._vs_original` ese número dividido por el de la madre en su timeframe. También `crossTF.reading.n_survives`: en cuántos timeframes sobrevivió. Si la madre pasa, **sigue solo ella**, en su timeframe y sin escalar; las versiones escaladas se borran siempre. Por ejemplo:

```
"12": {cut: CrossTF, rules: [{fact: crossTF.reading.n_survives, pass: ">= 1"}]}
```

Sobre las 15 madres del proyecto de prueba, esa regla deja pasar 2 (salida real, 2026-10-01).

Dónde corta cada paso: siempre en el databank del que bebe lo que viene después.

paso	corta	por qué ahí
8	`OOS`	lo lee el retest de mercados (9)
10	`Retest Markets - Family`	de ahí salen las madres que se escalan (10.5)
12	`CrossTF`	de ahí copia el 13 las madres, sin las escaladas
14	`MCR 8 Stress`	es el único de los ocho MCR que lee el SPP
16	`SPP OOS`	lo lee la WFM (19); las que quedan se casan con su gemela de `SPP IS` y las variantes (16.5) se diseñan con el backtest combinado

En el paso 10 el juez lee el P3 sobre la z conjunta de V0 + V2 (1,65 pasa) y cuenta en P8 los mercados `structural` como control negativo según la familia de la idea; el detalle está en `07-otros-mercados-y-timeframes.pdf › 05-retest-mercados`.

Un ejemplo completo

El plan sobre un proyecto donde todo, del 7 en adelante, está por hacer (salida real, simulando ese estado sobre `Test_USDJPY_donchianUpperCrossUp_H1`):

```
7 sqx · 8 python · 8 judge · 9 sqx · 10 python · 10 judge · 11 sqx · 12 python · 12 judge ·
13 sqx · 14 python · 14 judge · 15 sqx · 16 python · 16 judge · 16.5 variants   → se para antes del 19
```

Sobre ese mismo proyecto, que ya está terminado, el plan real no corre nada:

```
{"do": [], "stop": {"n": "21", "why": "no tiene tareas de SQX que el autopiloto sepa lanzar"}}
```

Sobre un proyecto nuevo, el plan sigue ahora hasta el 16.5 (`16 judge · 16.5 variants`) y se para antes del 19 (`la Walk Forward Matrix lee el oos2`).

Qué NO te dice

- Las medoides son representantes de su grupo, no las mejores: **la medoide no mide calidad**.
- El veredicto propio de cada estudio (el MANTENER/DESCARTAR de la puerta, por ejemplo) **no corta por sí solo**. Es un número más (`gate.verdict_csv.drop`) y solo corta si lo pones en una regla.
- Hoy, en el paso 12, **no puede comparar con la original**: en el último run la celda de la madre en su propio timeframe salió con 0 operaciones (OPEN.md #89). Los números `..._vs_original` faltan hasta que eso se arregle.

Si algo falla

- **«el worker no está libre»**: está encendido o lo tiene otra sesión. Espera, no lo pares tú.
- **«paso N: no se lanza»**: el preflight del paso de SQX dice qué falta (una tarea, una entrada vacía).
- **«tiene reglas y criteria.yaml no dice qué databank cortar»**: falta `cut` en ese paso.
- **Un fallo conocido de SQX** («Error while running project» tras «Task finished», un `NullPointerException`): el autopilot lo **reintenta él solo una vez**, antes borrando lo que el intento fallido había dejado en los databanks (si no, todo saldría duplicado). Con la sesión única, para el proyecto y recorta del databank de salida las estrategias de más; con `--restart`, borra los ficheros sobrantes.
- El **paso 26** sigue necesitando sus flags de MT5 (`--mt5-from`, `--mt5-to`, `--mt5-model`, `--mt5-firms`); sin ellos el autopilot se para limpio antes del 26. Ver el capítulo 85.

En todos los casos `fallo.md` dice el paso y el motivo, y el worker queda parado.

105. Workflow de un comando — del paso 5 al 21 con una sola línea

> **Desde el 2026-10-11** la cadena no corre los pasos 22, 23, 24 ni 25. Ningún paso se renumera; el 26 se añade si le das sus cuatro datos.

Qué pregunta responde

¿Puedo lanzar el workflow entero de un activo sin ir paso a paso? Sí: `bin/workflow.sh` crea el proyecto, le pone la paleta de bloques y corre solo todos los pasos de SQX y de Python (del 6 al 21, y el 26 si le das sus datos), juzgando y recortando con tus criterios. Tú das ocho valores; él no decide nada.

Cuándo lo usas, y cuándo no

Úsalo para probar una idea de punta a punta, o para lanzar un `Trade_` con todo decidido de antemano. **No** lo uses si algún valor está sin decidir: una duda se pregunta antes (regla 11), no se rellena por defecto. No hace el paso 26 (MT5) salvo que le des sus cuatro parámetros `--mt5-*`: sin ellos se para justo antes y lo dice en el resumen.

Antes de empezar

- El custodio (`SQX_w2`, puerto 5070) tiene que estar libre: nadie más lo usa.
- **No lances nada pesado en la máquina mientras corre** (tests, estudios, otros cálculos): el 2026-10-05 un test en paralelo hizo las tareas de SQX el doble de lentas.
- Un agente lo ejecuta con las skills `/workflow-simple` (sin plantilla) y `/workflow-template` (con plantilla).

Cómo se ejecuta

Cuatro formas. Ejemplos con Nikkei M30 largos, paleta `tendencia_base_v3`, 1000 estrategias y 20 minutos.

1. **`simple`** — sin plantilla, un solo lado:

```
bin/workflow.sh simple Test_NIKKEI225_M30_simpleL_a NIKKEI225 M30 long trendFollowing tendencia_base_v3
"Tendencia en Nikkei M30, largos" 1000 20
```

2. **`template`** — desde una plantilla ya hecha (la paleta rellena su hueco aleatorio; `-` si no tiene hueco):

```
bin/workflow.sh template Test_NIKKEI225_M30_plantilla_a NIKKEI225 M30 /ruta/a/mi-plantilla.sqx
tendencia_base_v3 "Tendencia en Nikkei M30, largos" 1000 20
```

Sólo hasta un paso — `--until`. Con `simple` o `template`, añadir `--until <paso>` tras los minutos crea el proyecto **sin las tareas de los pasos posteriores**, y el run acaba en el juicio de ese paso en vez de seguir hasta el 25: `oos` (paso 7), `crossmarket` (10), `crosstf` (12), `mcretest` (14) o `spp` (16). Sirve para probar el embudo con muchas estrategias sin pagar la cola.

```
bin/workflow.sh template Test_USDJPY_M30_doubleROCLong_10k USDJPY M30 /ruta/a/template.sqx
doubleROCLong_USDJPY_M30 "Prueba hasta crossTF" 10000 90 --until crosstf
```

El resumen termina con «Se para antes del paso 13: el proyecto no lleva sus tareas: acaba aquí». Ese proyecto no se puede continuar más allá: para el workflow entero se crea otro sin `--until`.

Hasta la Walk Forward Matrix — `--until wfm`. Es el único corte que no quita tareas: el proyecto se crea entero y el run acaba tras el paso 19 (la WFM, su lectura y las filas de 17-19 en el ledger), sin los pasos 20 a 25. El resumen termina con «Se para antes del paso 20: --until wfm: el run acaba tras la WFM». Si el run falla y se reanuda, se repite el corte: `bin/workflow.sh run <PROYECTO> --until wfm --own-log`; sin él, `run` sigue hasta el 25.

3. `run` — continúa un proyecto que ya existe, desde donde se quedó:

```
bin/workflow.sh run Test_NIKKEI225_M30_simpleL_a
```

4. `plan` — enseña lo que haría `run` y no ejecuta nada:

```
bin/workflow.sh plan Test_NIKKEI225_M30_simpleL_a
```

Los ocho valores de `simple` y `template` (`simple` lleva además FAMILIA, obligatoria, justo después del lado):

valor	qué es	ejemplo
PROYECTO	`Test_…` (prueba, se tira) o `Trade_…` (en serio); solo letras, números y guiones bajos	`Test_NIKKEI225_M30_simpleL_a`
ACTIVO	el activo, como se llama en `assets/`	`NIKKEI225`
TF	`M15`, `M30`, `H1` o `H4`	`M30`
LADO / PLANTILLA	`long` o `short` (nunca los dos); en `template`, el `.sqx` de la plantilla	`long`
FAMILIA (sólo `simple`)	la familia de la idea, una de `breakout`, `trendFollowing`, `meanReversion`, `momentum`, `volatility`, `pattern`, `session`; sin valor por defecto, si falta o no es una de ellas el comando se niega y se te pregunta. Se guarda en `projects/registry.csv` y el paso 10 la lee (un proyecto ya creado sin familia se puede reanudar sin ella y se juzga como antes) (en `template` es el arquetipo de la plantilla)	`trendFollowing`
PALETA	la familia de idea: un archivo de `sqx/blocks/palettes/` sin `.yaml`	`tendencia_base_v3`
PROPÓSITO	una frase entre comillas	`"Tendencia en Nikkei M30, largos"`
ESTRATEGIAS	cuántas se generan antes de parar	`1000`
MINUTOS	tiempo máximo de la generación	`20`

Dos opciones al final:

- `--smoke`: **prueba funcional**, solo proyectos `Test_`. Solo juzgan las reglas de legibilidad; sirve para comprobar que la cadena funciona, no para sacar estrategias buenas.
- `--restart`: la ruta antigua, que arranca y para SQX en cada paso. Es más lenta; solo si la sesión única ha fallado.

Cuánto tarda

Medido el 2026-10-05: **160 minutos** para 1.000 estrategias y 10 madres de 500 variantes. Una madre real de 5.000 variantes cuesta unos **32 minutos** cada una, así que con más madres el total crece de forma proporcional.

Qué produce

Todo cuelga de `AlgoData/autopilot/<PROYECTO>/<fecha-hora>/`:

archivo	qué es
`resumen.md`	una línea por paso: qué hizo, cuánto tardó, cuántas estrategias entraban y salían, y dónde se paró
`tiempos.md`	cuánto tardó cada paso
`fallo.md`	solo existe si falló: el paso y el motivo
`estado.txt`	la última línea dice `FIN · …` o `FALLO en el paso N: …`
`paso-<n>/`	los números y el veredicto de cada paso que juzga

Al terminar el comando imprime la última línea de estado y la ruta de `resumen.md`.

Cómo se lee el resultado

Abre `resumen.md`. Tres casos: `FIN` y «Se para antes del paso 26» (completo; el 26 es otra tarea); `FIN` y «no dejó ninguna superviviente» (un recorte se quedó sin nadie: es un resultado, no un error); `FALLO en el paso N` (mira `fallo.md`). Si el comando no imprime ni línea de estado, se negó a empezar y la frase que escribe es el motivo.

Si falla

1. Para el custodio si lo dejó vivo y es tuyo: `bin/sqx-worker.sh --role custodian stop`.
2. Reintenta una vez desde donde se quedó: `bin/workflow.sh run <PROYECTO>` (con `--smoke` si lo llevaba).
3. Si vuelve a fallar en el mismo paso: `bin/workflow.sh run <PROYECTO> --restart`.
4. Si también falla: parar y leer `fallo.md`. No se toca el código ni los criterios para pasar.

Qué NO te dice

No dice si una estrategia es buena: aplica tus criterios (`pipeline/autopilot/criteria.yaml`) y recorta. No hace el paso 26 sin sus parámetros de MT5. No adivina un valor que falte: ante la duda, pregunta.

El uso que imprime el propio comando

```
# workflow — the whole workflow of one project, steps 5 to 25, in one command. Nothing to decide:
# every choice is an argument, a missing one is an error, and the first failure ends it.
#
# Usage:
#   bin/workflow.sh simple   <PROJECT> <SYMBOL> <TF> <long|short> <FAMILY> <PALETTE> "<PURPOSE>"
<STRATEGIES> <MINUTES> [--until STEP] [--smoke]
#   bin/workflow.sh template <PROJECT> <SYMBOL> <TF> <TEMPLATE.sqx> <PALETTE|-> "<PURPOSE>" <STRATEGIES>
<MINUTES> [--until STEP] [--smoke]
#   bin/workflow.sh run      <PROJECT> [--until wfm] [--smoke] [--restart]
#   bin/workflow.sh plan     <PROJECT>
```

106. La campaña — un bucle sin supervisión que recorre plantilla × activo × temporalidad

Qué pregunta responde

«¿Qué pasa si pruebo, una tras otra y sin que yo esté delante, todas las combinaciones de plantilla, activo y temporalidad que el tablero de investigación admite?». La campaña toma una celda, corre su workflow entero (pasos 6 a 25, con `bin/workflow.sh template`), guarda el resultado con sus estrategias y pasa a la siguiente. Solo largos, y solo M30, H1 y H4.

No hay ninguna IA dentro del bucle: es un programa corriente. Programar celdas es mecánico (veredicto del Council, 2026-10-05), el lote se decide antes de ver resultados, y tú tienes el veto con el archivo `STOP`.

Cuándo lo usas, y cuándo no

Lo usas cuando quieres que el custodio trabaje horas o días sin tocarlo. No lo uses para probar una sola idea (eso es `/workflow-template`), ni mientras el máster esté abierto y lo necesites, ni con otra cosa pesada corriendo: duplica los tiempos de SQX.

Antes de empezar

- El tablero (`python3 -m studies.research.board`) tiene que estar al día: de él salen las celdas.
- El custodio (`SQX_w2`) libre: la campaña lo arranca y lo para sola.
- Al menos `min_free_gb` (50 GB) libres en `AlgoData`; con menos no empieza una celda.
- Lo que se corre sale de `pipeline/campaign/config.yaml` y de `pipeline/autopilot/criteria.yaml`.

Cómo se ejecuta

1. Ver qué hay y qué vendría después (no escribe nada):

```
python3 -m pipeline.campaign.plan
```

2. Congelar las N siguientes como un lote (esto sí escribe el registro real):

```
python3 -m pipeline.campaign.plan --queue 5
```

3. Lanzar el bucle, separado de la terminal:

```
setsid nohup python3 -m pipeline.campaign.run >> ~/Desktop/AlgoData/campaign/loop.log 2>&1 &
```

4. Pararlo y reanudarlo:

```
python3 -m pipeline.campaign.run --stop     # pone STOP: para el autopilot y SQX
python3 -m pipeline.campaign.run --go       # quita STOP; se vuelve a lanzar el bucle
```

flag	de qué	qué hace
`--queue N`	`plan`	congela las N siguientes candidatas como un lote
`--show N`	`plan`	cuántas candidatas enseña (15 por defecto)
`--stop`	`run`	pone el archivo STOP; la celda en curso queda `interrupted` y la máquina queda libre en minutos (abajo)
`--go`	`run`	quita el STOP

Un STOP a mitad de un build no lo detiene (SQX no sabe parar un build desde fuera, y al pedirle salir guarda primero los databanks de todos los demás proyectos mientras el build sigue a tope). Lo que hace el bucle desde el 2026-10-06: pide la salida, espera a que SQX haya guardado cada databank en disco de los otros proyectos, y entonces mata el proceso por PID. Lo que el proyecto en curso tenía solo en memoria se pierde; sus salidas en disco y el log de ese arranque se apartan, y al reanudar la celda ese paso se repite desde el principio. En la prueba, de STOP a máquina libre pasaron unos minutos, no el resto del build.

Un paso de SQX que no avanza ya no se espera para siempre: cada tarea tiene un tope, 3 veces lo más largo que esa misma tarea ha tardado en runs sanos (`AlgoData/autopilot/task-durations.json` , mínimo 20 minutos, 60 si nunca se ha visto; el build queda fuera, tiene su propio límite). Al pasarse, el autopilot la aborta, cierra la sesión y cuenta como fallo del paso: entran los tres intentos de la celda, cada uno con SQX recién arrancado. El caso que lo motivó ya no ocurre: «MCR 6 Exits» se colgaba para siempre sobre estrategias que salen solo por condición (sin stop, objetivo ni trailing: nada que perturbar); ahora esa tarea se omite en esas poblaciones, como «MCR 4 MinDist» sin órdenes pendientes.

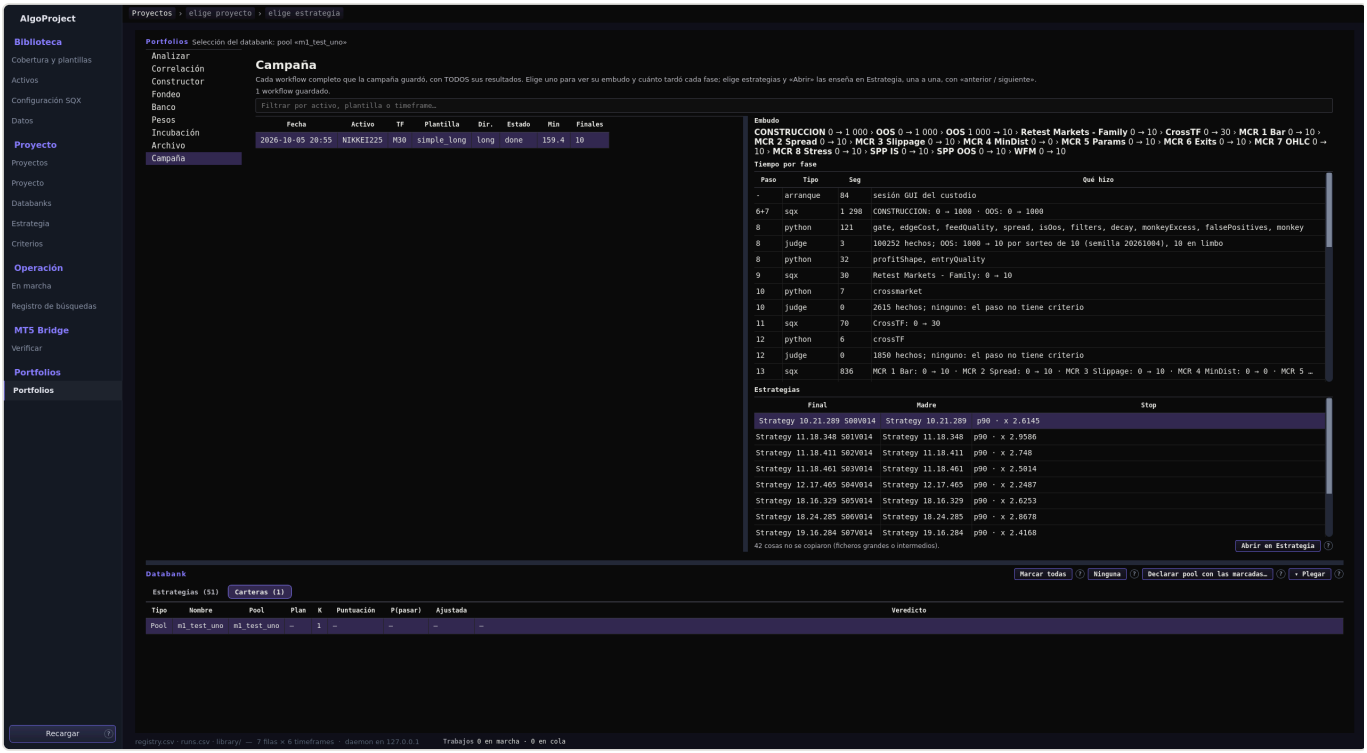
Cada celda tiene tres intentos (crear y correr, correr, correr con reinicio); si falla los tres queda `failed` . Dos celdas fallidas seguidas paran el bucle: el fallo ya no es de la celda. Una celda interrumpida se retoma al arrancar de nuevo.

Qué produce

Todo en `AlgoData/campaign/` :

- `ledger.jsonl` — el registro: una línea por cada cambio de estado de cada celda; no se reescribe.
- `batches/<id>.json` — cada lote tal como se congeló.
- `runs/<plantilla>_<activo>_<tf>_<dirección>_<fecha>/` — un workflow terminado: `workflow.json` (tiempo por paso, recuentos `A → B` de supervivientes, huella de `criteria.yaml` , finalistas), `run/` (la carpeta del autopilot) y `estrategias/` (enlaces a cada finalista congelado en `AlgoData/archive/`).
- `STOP` , `loop.lock` y `loop.log` .

En la ventana, la vista «Campaña» lista los workflows guardados y abre sus estrategias desde el archivo.



La paleta de cada celda la elige el `buildingBlocksExpert` (decisión del dueño, 2026-10-05): antes de crear el proyecto, una sesión de Claude (Opus) escribe la paleta del hueco libre para esa plantilla en ese activo y timeframe (`sqx/blocks/palettes/<plantilla>_<ACTIVO>_<TF>.yaml`). Si el experto tiene una duda, la celda queda apartada con estado `question` y el bucle sigue con la siguiente; `python3 -m pipeline.campaign.plan` la enseña con su pregunta. `indicesIBS` sólo se ofrece en índices.

Dónde quedan las supervivientes

Cada estrategia que termina un workflow se archiva sola en `AlgoData/campaign/supervivientes/<ACTIVO>/<TF>/<dirección>/<familia>/<plantilla>/<fecha>_<estrategia>/`: dentro está `estrategia.sqx` (se puede cargar en SQX tal cual) y tres enlaces — `archivo` (la versión congelada con todos sus resultados), `madre` (la de su estrategia madre, que es la que firma los estudios) y `workflow` (el run del que salió). `supervivientes/indice.csv` tiene una línea por estrategia (fecha, activo, timeframe, dirección, familia, plantilla, madre, stop) para filtrar en una hoja de cálculo. Las de una celda de prueba van aparte, bajo `supervivientes/_pruebas/`.

Una celda de prueba

`python3 -m pipeline.campaign.plan --test <PLANTILLA> <ACTIVO> <TF>` encola una prueba: build corto (3.000 estrategias, 60 minutos; `test:` en `config.yaml`), proyecto `Test_…`, y no cuenta como el run real de esa celda. Cuántas pasan del paso 8 lo decide `dev.sample` de `criteria.yaml` (5 desde el 2026-10-05).

Cómo se lee el resultado

```
$ python3 -m pipeline.campaign.plan
Registro: 0 celdas —

Candidatas sin lanzar: 827 (tablero del 2026-10-02T20:19:47). Las 15 primeras:
  AUDUSD     H1   reversion    cciPercLong                       nota  67.5  prior Alta
  AUDUSD     M30  reversion    cciPercLong_withExit              nota  67.5  prior Alta
  EURUSD     M30  reversion    hurstMeanRevLong                  nota  67.5  prior Alta
  GBPJPY     H1   momentum     directionalMomentumFilters_withExit  nota  67.5  prior Alta
  USDCAD     H1   reversion    indicesIBS                        nota  67.5  prior Alta
  USDCAD     M30  reversion    meanReversionFilters              nota  67.5  prior Alta
  USDJPY     H1   reversion    meanReversionFilters_withExit     nota  67.5  prior Alta
  USDJPY     M30  reversion    cciPercLong                       nota  67.5  prior Alta
  AUDJPY     H1   tendencia    trendRegimeFilters                nota  60.0  prior Alta
  AUDJPY     H1   reversion    hurstMeanRevLong                  nota  60.0  prior Alta
  AUDJPY     H1   momentum     doubleROCLong                     nota  60.0  prior Alta
  AUDJPY     H4   tendencia    aroonLong_withExit                nota  60.0  prior Alta
  AUDJPY     H4   reversion    meanReversionFilters              nota  60.0  prior Alta
  AUDJPY     H4   momentum     bookTriggers_withExit             nota  60.0  prior Alta
  AUDJPY     M30  tendencia    filterMALong                      nota  60.0  prior Alta
```

El registro está vacío (0 celdas) y hay 827 candidatas: celdas admitidas del tablero × plantillas largas de su familia sin marca, con el activo con costes completos. Orden: puntos del tablero, una plantilla por celda y vuelta. Una celda ya en el registro, o ya corrida por un proyecto `Trade_` o `Research_`, no se vuelve a ofrecer.

```
$ python3 -m pipeline.campaign.run --help
usage: python3 -m pipeline.campaign.run [-h] [--stop] [--go]

options:
  -h, --help  show this help message and exit
  --stop      para el bucle y SQX; guarda lo que haya
  --go        quita el STOP para poder arrancar de nuevo
```

Así termina el bucle en la prueba (`python3 tests/test_campaign.py` , con una orden de workflow falsa y una raíz de datos temporal: es la salida de la prueba, no de una campaña real):

```
$ python3 tests/test_campaign.py
RAÍZ DE PRUEBA: /tmp/tmppjp_e21n — lo que salga de aquí no es un resultado
Trade_A: interrupted · 0.0 min · 0 finalista(s) · /tmp/tmppjp_e21n/campaign/runs/tA_XAUUSD_H1_long_20261005-2…
PARADO: STOP
Trade_A: done · 3.0 min · 0 finalista(s) · /tmp/tmppjp_e21n/campaign/runs/tA_XAUUSD_H1_long_20261005-205539b
Trade_B: done · 3.0 min · 0 finalista(s) · /tmp/tmppjp_e21n/campaign/runs/tB_XAUUSD_H1_long_20261005-205539
PARADO: no quedan celdas pendientes
Trade_C: failed · 0.0 min · 0 finalista(s) · /tmp/tmppjp_e21n/campaign/runs/tC_XAUUSD_H1_long_20261005-205540
Trade_D: failed · 0.0 min · 0 finalista(s) · /tmp/tmppjp_e21n/campaign/runs/tD_XAUUSD_H1_long_20261005-205541
PARADO: 2 celdas fallidas seguidas
ok
```

Un ejemplo completo

`python3 -m pipeline.campaign.plan --queue 5` → `python3 -m pipeline.campaign.run` (detached) → al día siguiente, `python3 -m pipeline.campaign.plan` muestra el estado de cada celda → abrir la vista «Campaña».

Qué NO te dice

- Que una estrategia vaya a funcionar en real: es la salida del workflow, con sus límites.
- Un resultado suelto no vale lo que parece: se juzga contra **N = todas las líneas del registro**, fallos incluidos (el registro es el número de pruebas).
- Un lote se congela antes de que existan sus resultados; los resultados solo informan el lote siguiente, nunca se reordena el actual.

Si algo falla (y avisos)

- **El build dura siempre 180 minutos.** De SQX en sesión viva solo vale el límite de tiempo; 10.000 es el tamaño del databank (guarda las 10.000 mejores), no una parada; y un build en marcha no se puede parar antes desde fuera: el STOP lo mata (arriba) y se reconstruye al reanudar.
- Si un reintento se rechaza, el motivo está en `salida.txt` dentro de la carpeta del run del autopilot (`AlgoData/autopilot/<proyecto>/<sello>/`); todos los intentos van con `--own-log` , así que el log reciente del custodio, que es del propio bucle, no cuenta como «ocupado».
- **Los filtros son `pipeline/autopilot/criteria.yaml` tal como está**; cada run guarda su huella. Se pueden apretar después sobre lo guardado, nunca aflojar: cada paso borra lo que rechaza.
- Con `dev.on: true` , el paso 8 se queda con 10 al azar: es tu interruptor.
- `failed` tras tres intentos: mira `runs/<…>/run/fallo.md` ; dos seguidas paran el bucle.
- Disco con menos de `min_free_gb` : el bucle no arranca celda; libera espacio y relánzalo.

55. Retirar proyectos — quitar de SQX los custom projects que ya respondieron

Qué pregunta responde

¿Qué custom projects hay en cada instalación, cuántas tareas tiene cada uno, cuánto ocupa y para qué se creó? Y, cuando uno ya no sirve, lo quita de SQX sin perder su configuración: el `project.cfx` se guarda comprimido en AlgoData y la carpeta desaparece de `user/projects`.

Lo normal: no haces nada

Cada lunes a las 02:50 (la noche del domingo al lunes) el agente `projectJanitor` pasa solo (`bin/weekly-project-cleanup.sh`, en el crontab). Aplica la regla de `sqx/projects/sweep.py`:

se retira	se queda
todo proyecto `Test_` de w1 y w2	todo proyecto `Trade_` o `Research_`
todo proyecto antiguo de **menos de 10 tareas** (un workflow entero tiene 15-18)	lo tocado en las últimas 24 h
lo que tú apuntes en `AlgoData/projects/retire-queue.txt`	todo lo de una instalación encendida
	del máster, todo lo que no esté en la cola

Antes de retirar, el agente busca señales de vida que la regla no ve: un encargo abierto que nombre el proyecto, una línea de `OPEN.md`, o un ledger o un pipeline escritos esa semana. Si encuentra alguna, se queda con la duda y no lo toca. No arranca, no para ni consulta ningún SQX.

Te deja el resumen en `AlgoData/audit/AAAA-MM-DD-proyectos.md`: qué se fue, qué se quedó y por qué, y cuánto disco se liberó. El log está en `AlgoData/logs/weekly-project-cleanup.log`.

Para que se lleve un proyecto concreto, incluido uno del máster, añade una línea a la cola:

```
custodian USDJPY_emaCross_H1
master    XAU_crossmarket_demo
```

Una vez retirado, su línea desaparece de la cola.

Cuándo lo usas a mano, y cuándo no

- **Al terminar una tarea que creó un proyecto `Test_`.** Quien lo crea, lo retira. Es la regla.
- **Cuando `--list` enseña proyectos de pocas tareas** que quedaron de pruebas sueltas.
- **No** para un proyecto `Trade_` ni `Research_`: ese se queda hasta que tú digas.
- **No** en el máster, salvo un proyecto que tú hayas nombrado.
- **No** para "hacer sitio" en un proyecto que se va a volver a correr: vacía sus databanks, no lo borres.

Lo que un run encontró **no vive en el proyecto**. Métricas, trades, curvas y veredictos ya están en parquet en `AlgoData/raw/`, `harvest/` y `reports/`, y retirar el proyecto no los toca. Lo único que se pierde son las estrategias de los databanks. Si quieres conservar las de la última prueba (la WFM), añade `--keep WFM`.

Antes de empezar

- **La instalación del proyecto tiene que estar parada.** Un SQX encendido tiene el proyecto en memoria y lo reescribiría al salir (regla dura 4). El comando lo comprueba y se niega si no lo está: el puerto en los workers, cualquier proceso que corra desde `~/Desktop/SQX` en el máster.
- **Nadie puede estar usándolo.** Si el proyecto cambió hace menos de 60 minutos, el comando se niega, porque puede haber otra sesión cosechándolo.

Cómo se ejecuta

Ver qué haría el pase semanal si corriera ahora (no toca nada):

```
python3 -m sqx.projects.retire --sweep
```

Ver qué hay:

```
python3 -m sqx.projects.retire --list
```

Retirar (sin `--yes` solo dice lo que haría):

```
python3 -m sqx.projects.retire Test_XAU_keltner_humo --role custodian          # prueba en seco
python3 -m sqx.projects.retire Test_XAU_keltner_humo --role custodian --yes    # de verdad
python3 -m sqx.projects.retire Trade_XAU_keltner_H1 --role custodian --keep WFM --yes
```

flag	obligatorio	qué hace
nombres	para retirar	uno o varios proyectos de la misma instalación
`--list`	no	lista los custom projects de las tres instalaciones y no toca nada; es lo que hace sin nombres
`--role`	no	`custodian` por defecto; `conductor` o `master`
`--keep`	no	un databank que se guarda junto al `.cfx`, p. ej. `--keep WFM`; se puede repetir
`--sweep`	no	aplica la regla semanal a las tres instalaciones; con `--yes`, la ejecuta
`--yes`	no	lo hace de verdad. Sin él, prueba en seco
`--own`	no	el proyecto lo creó esta misma tarea: no espera los 60 min sin cambios
`--drop-unsaved`	no	retira aunque queden supervivientes del paso 8 sin guardar en el almacén: sus `.sqx` se pierden. Solo cuando el dueño las da por perdidas; el barrido semanal nunca lo usa

Tarda segundos y no arranca SQX.

Qué produce

- `~/Desktop/AlgoData/projects/retired/<instalación>/<proyecto>-<fecha>.tar.gz`: el `project.cfx` y los databanks de `--keep`.
- La carpeta `<instalación>/user/projects/<proyecto>` **se borra**.
- En `~/Desktop/AlgoData/projects/registry.csv` la fila del proyecto queda con la fecha de retiro y la ruta del archivo. Si el proyecto era anterior al registro, se le añade una fila.

El registro y los nombres

Desde el 2026-09-26, `sqx.projects.builder` solo acepta dos prefijos:

- **`Test_...`**: una prueba de funcionamiento. Se retira en cuanto ha respondido.
- **`Trade_...`**: un proyecto con intención de dar frutos reales. Se queda.
- **`Research_...`**: lo que construye la cola de investigación (desde 2026-10-03). Se queda hasta que tú decidas si se tira o se conserva.

Además pide `--purpose "una frase"`. Cada proyecto que crea deja una fila en `registry.csv`: nombre, tipo, instalación, fecha, propósito, símbolo, timeframe, plantilla y si lleva el workflow entero. En `--list`, la columna `legacy` marca los proyectos anteriores a esta regla.

Cómo se lee el resultado

```
$ python3 -m sqx.projects.retire --list
== master (SQX) — 13 custom
  AUDJPY                          19 tareas       221 MB  2026-09-25 07:07  legacy
  CADJPY_H1                       19 tareas         2 MB  2026-09-02 14:33  legacy
  EURJPY_H1                       18 tareas        23 MB  2026-09-25 07:07  legacy
  EURUSD                          19 tareas       584 MB  2026-09-25 07:04  legacy
  GBPJPY_H1                       20 tareas      1066 MB  2026-09-25 07:07  legacy
  Infinox_SP500ft_H4_HighPrecision     8 tareas        82 MB  2026-09-02 14:33  legacy
  SP500_H1                        18 tareas       197 MB  2026-09-25 07:00  legacy
  TestXAUUSD2                      3 tareas         0 MB  2026-09-23 20:32  legacy
  USDCHF                          23 tareas        10 MB  2026-09-25 06:59  legacy
  USDJPY                          19 tareas      1433 MB  2026-09-25 07:02  legacy
  XAUUSD                          17 tareas       734 MB  2026-09-25 07:10  legacy
  XAUUSD_Breakout_H1               5 tareas         0 MB  2026-09-03 06:50  legacy
  XAU_crossmarket_demo             3 tareas         0 MB  2026-09-23 20:34  legacy
== conductor (SQX_w1) — 2 custom
  XAUUSD_Breakout_H1               5 tareas         0 MB  2026-09-02 21:12  legacy
  XAU_crossmarket_demo             3 tareas         0 MB  2026-09-23 20:48  legacy
== custodian (SQX_w2) — 17 custom
  PerfUSDJPY_Python_v1            15 tareas      1393 MB  2026-09-24 20:58  legacy
  TestUSDJPY_Workflow_v1          15 tareas       207 MB  2026-09-24 20:05  legacy
  TestUSDJPY_crossTF_v1            1 tareas        26 MB  2026-09-24 12:56  legacy
  TestXAUUSD2                      4 tareas         2 MB  2026-09-24 06:45  legacy
  TestXAU_crossTF                  1 tareas         1 MB  2026-09-23 21:23  legacy
  Test_XAUUSD                      1 tareas         0 MB  2026-09-22 13:55  test
  USDJPY_emaCross_H1              18 tareas       386 MB  2026-09-26 07:28  legacy
  USDJPY_structural_v1            18 tareas         9 MB  2026-09-26 08:09  legacy
  USDJPY_variantes                 3 tareas         0 MB  2026-09-25 16:42  legacy
  USDJPY_wfm                       1 tareas        34 MB  2026-09-25 17:50  legacy
  USDJPY_workflow_profiling_v1    18 tareas      2496 MB  2026-09-26 13:38  legacy
  XAU_ISOOS_ejemplo                2 tareas        26 MB  2026-09-23 14:54  legacy
  XAU_crosstf_probe                2 tareas         1 MB  2026-09-23 20:55  legacy
  XAU_mcr_prueba                   8 tareas        33 MB  2026-09-23 23:23  legacy
  algo_XAU_keltner                15 tareas         1 MB  2026-09-23 10:10  legacy
  blocktest_grupoA                 1 tareas         6 MB  2026-09-24 08:49  legacy
  smoke_keltnerUpperCrossUp        1 tareas         6 MB  2026-09-23 07:26  legacy
```

Cada fila es un proyecto: nombre, tareas, tamaño en disco, último cambio, tipo y propósito (si está registrado). Con el workflow entero, un proyecto tiene 15 a 18 tareas. **Uno de 1 a 5 tareas es una prueba suelta**, y es lo primero que conviene retirar.

```
$ python3 -m sqx.projects.retire --role custodian smoke_keltnerUpperCrossUp XAU_mcr_prueba
would retire SQX_w2/smoke_keltnerUpperCrossUp: 1 tareas, 6 MB → /home/sergioguslw/Desktop/AlgoData/projects/retired/SQX_w2/smoke_keltnerUpperCrossUp-2026-09-26.tar.gz
would retire SQX_w2/XAU_mcr_prueba: 8 tareas, 33 MB → /home/sergioguslw/Desktop/AlgoData/projects/retired/SQX_w2/XAU_mcr_prueba-2026-09-26.tar.gz
dry run — add --yes to retire them
```

La prueba en seco dice, por proyecto, cuántas tareas y megas se van y a qué archivo irían.

Un ejemplo completo

1. `python3 -m sqx.projects.retire --list`: ves `smoke_keltnerUpperCrossUp` en el custodio, con 1 tarea y 6 MB.
2. Compruebas que el custodio está parado (`bin/sqx-worker.sh --role custodian status`) y que ninguna otra sesión lo está usando.
3. `python3 -m sqx.projects.retire smoke_keltnerUpperCrossUp --role custodian`: la prueba en seco.

4. Lo mismo con `--yes`. La carpeta desaparece y su `.cfx` queda en `AlgoData/projects/retired/SQX_w2/smoke_keltnerUpperCrossUp-<fecha>.tar.gz`.

Para recuperarlo: `tar xzf <archivo> -C ~/Desktop/SQX_w2/user/projects/`, con el custodio parado. Vuelve con su configuración y sin estrategias (salvo las de `--keep`).

Qué NO te dice

- **Si un proyecto sigue haciendo falta.** El número de tareas y la fecha orientan, pero no saben si una ficha de knowhow cita ese proyecto como ejemplo. La ficha sigue siendo válida porque los datos están en parquet, aunque el proyecto ya no esté en SQX.
- **Si otra sesión lo va a usar mañana.** La guarda de 60 minutos solo protege lo que se está moviendo ahora.

Olvidar un proyecto del todo: `sqx.projects.forget`

`retire.py` deja adrede el parquet y los CSV de `raw/`, `harvest/`, `metrics/`, `reports/`, `strategyPermutations/` — es lo que un run encontró, y normalmente vale la pena conservarlo. Cuando de verdad quieres borrar un proyecto entero, no solo sacarlo de SQX, usa `sqx.projects.forget`: retira lo que siga vivo en el conductor o el custodio (igual que `retire.py`, así que el `.cfx` queda archivado) **y además** borra su subcarpeta en esas seis carpetas de AlgoData.

```
python3 -m sqx.projects.forget USDJPY_wfm               # prueba en seco: qué se borraría
python3 -m sqx.projects.forget USDJPY_wfm --yes          # de verdad
```

Se niega igual que `retire.py` con un proyecto de serie, y con uno que solo exista en el máster — ese install es del dueño (regla dura 3) y nada lo toca sin que él lo nombre. No borra `bars/`, `spread/`, `templates/`, `ledger/`, `projectsBackup/`, `snapshots/`, `logs/`, `profiling/`, `cache/` ni `pipeline/`: eso es dato base o el registro global, nunca de un proyecto suelto.

Si algo falla

- `the custodian is up ...`: el worker está encendido. Páralo con `bin/sqx-worker.sh --role custodian stop`, pero solo si no hay nadie más usándolo.
- `... changed less than 60 min ago`: alguien lo está tocando. Mira `ListAgents` y el log del día.
- `... is a stock project`: `Builder`, `Retester` y los demás de serie no se retiran nunca.

62. Metadatos de una estrategia — qué hace y con qué costes se probó, leído del fichero

Qué pregunta responde

Abre un `.sqx` y te dice, como campos sueltos y sin arrancar SQX: si opera en largo, en corto o en los dos sentidos; qué condiciones tiene la entrada (y la salida, si la hay) con sus valores reales; qué tipo de orden usa y si lleva stop, objetivo, trailing o salida por número de barras; con qué money management se calcularon sus números; si cierra el viernes y a qué hora; y con qué spread, slippage, comisión y swap se hizo el último test. Es lo que pinta el panel de metadatos de la ficha de estrategia de la ventana (encargo 22 §5 y §12.2) y lo que el archivo de estrategias congela.

Cuándo lo usas, y cuándo no

Cuando quieres saber qué es una estrategia antes de gastar CPU en ella, o comprobar que el coste con el que se hizo un backtest es el que crees.

No es un traductor: no te da pseudocódigo ni un backtest en Python (eso quedó fuera de esta ronda, decisión del dueño del 2026-09-27). Da **campos**. Tampoco te dice si la estrategia es buena: no mira ni una métrica.

Antes de empezar

Nada. Solo lee ficheros del disco: el `.sqx` y, si se lo das, el `project.cfx` del proyecto. **No envía ningún comando a SQX**, así que puedes usarlo con el maestro abierto, con un worker corriendo o con todo apagado. Da igual de qué instalación sea el fichero.

Cómo se ejecuta

```
python3 -m sqx.inspect.strategymeta "$HOME/Desktop/SQX/user/projects/XAUUSD/databanks/OOS/Strategy 10.13.25.sqx" \
    --cfx "$HOME/Desktop/SQX/user/projects/XAUUSD/project.cfx"
```

argumento	obligatorio	qué hace
el `.sqx`	sí	la estrategia. Lo normal es que esté dentro de la carpeta de un databank
`--cfx`	no	el `project.cfx` del proyecto al que pertenece ese databank. Con él, además de lo que guarda el `.sqx`, sale lo que dice **hoy** la tarea que escribe en ese databank

Tarda menos de un segundo por estrategia. Imprime un JSON por pantalla y no escribe nada. El código vive en `sqx/inspect/strategymeta/` (`__init__.py`, `rules.py`, `settings.py` y `sqx/inspect/strategymeta/__main__.py`, que es la línea de comandos). Desde Python, lo mismo es `from sqx.inspect.strategymeta import read` y `read(Path(sqx), Path(cfx))`.

Qué produce

Solo la salida por pantalla: un diccionario con estos campos.

campo	qué es
`name`, `feed`, `timeframe`	nombre, feed y timeframe del último test
`direction`	`long`, `short` o `both`: la dirección de las órdenes de entrada
`entries`, `exits`	una fila por condición: la señal, el operador (`AND` / `OR`), el bloque y sus parámetros con el **valor real** (no el nombre de la variable)
`signals`	cada señal en una línea: `Bloque(Param=valor, …) AND Bloque(…)`
`indicators`	el resumen que el propio SQX guarda de indicadores de entrada y de salida
`orders`	cada orden de entrada: tipo, dirección, stop, objetivo, trailing, break-even y salida por barras (`null` = no lleva)
`order_types`	los tipos de orden: `EnterAtMarket`, `EnterAtStop`, `EnterAtLimit`
`money_management`	el método de tamaño con sus parámetros y el capital inicial; `from` dice si es el de la estrategia o el de la tarea
`friday_close`	si cierra el viernes (`on`), a qué hora y la clave de SQX (`ExitOnFriday`)
`trading_options`	fin de semana, cierre al final del día, franja horaria, máximo de operaciones al día, sesión
`last_test`	la configuración con la que SQX calculó los números que guarda el `.sqx`: databanks de entrada y salida, ventana, tramo OOS, precisión, motor y **costes** (spread, slippage, distancia mínima, comisión, swap)
`backtest`	solo con `--cfx`: cada tarea del proyecto que escribe en ese databank, con la misma forma que `last_test` y si está activa
`asset`, `asset_card_sha256`	la ficha de `assets/symbols/` del activo y su huella **de hoy**
`notes`	avisos que hay que leer junto a los números

Cómo se lee el resultado

La primera mitad dice qué hace la estrategia:

```
$ python3 -m sqx.inspect.strategymeta ".../XAUUSD/databanks/OOS/Strategy 10.13.25.sqx" --cfx .../XAUUSD/project.cfx
{
  "name": "Strategy 10.13.25",
  "feed": "XAUUSD_DukasM1_Infinox",
  "timeframe": "M30",
  "direction": "long",
  "signals": {
    "LongEntrySignal": "CBlock_WassersteinShiftLow(Int2=50, Double3=16.8) AND BBBarOpensAboveUp(Period=20, Deviation=1.9,
ComputedFrom=0, Shift=1)"
  },
  "indicators": {
    "entry": [
      "(BB) Bar opens above Upper band",
      "WassersteinShiftLow"
    ],
    "exit": [
      "ExitAfterXBars"
    ]
  },
  "orders": [
    {
      "type": "EnterAtMarket",
      "direction": "long",
      "stop_loss": null,
      "profit_target": null,
      "trailing_stop": null,
      "break_even": null,
      "exit_after_bars": 22
    }
  ],
  "order_types": [
    "EnterAtMarket"
  ],
  "friday_close": {
    "on": true,
    "key": "ExitOnFriday",
    "time": "21:00"
  }
}
```

La Strategy 10.13.25 del proyecto XAUUSD compra (`long`) a mercado cuando se cumplen a la vez dos condiciones: el bloque propio `WassersteinShiftLow` con 50 y 16,8, y que la barra abra por encima de la banda de Bollinger superior (periodo 20, desviación 1,9). No lleva stop ni objetivo: sale a las 22 barras. Cierra los viernes a las 21:00.

La segunda mitad dice con qué se probó (la captura acorta `backtest`, cuyos costes son iguales a los de `last_test`):

```
$ python3 -m sqx.inspect.strategymeta ".../XAUUSD/databanks/OOS/Strategy 10.13.25.sqx" --cfx .../XAUUSD/project.cfx   (segunda
mitad)
{
  "money_management": {
    "from": "task",
    "method": "ATRRiskBasedSizingFixedRisk",
    "params": {
      "Amount": "1000",
      "ATRPeriod": "20",
      "ATRMult": "5",
      "Decimals": "2",
      "LotsIfNoMM": "0.01",
      "MaxLots": "5000"
    },
    "initial_capital": "100000",
    "max_drawdown": "30"
  },
  "last_test": {
    "input": "Results",
    "output": "OOS",
    "window": [
      "2008.01.01",
      "2022.12.31"
    ],
    "oos": [
      [
        "2018.01.01",
        "2022.12.31"
      ]
    ],
    "costs": {
      "spread": "0",
      "slippage": "0",
      "min_distance": "10",
      "commission": [
        {
          "method": "PercentageBased",
          "value": "0.01"
        }
      ],
      "swap": {
        "use": "true",
        "type": "percent",
        "long": "-8",
        "short": "38.76",
        "tripleSwapOn": "WEDNESDAY",
        "rolloutHour": "23:00"
      }
    }
  },
  "backtest": [
    {
      "project": "XAUUSD",
      "task": "Retest strategies",
      "active": true,
      "costs": "... (iguales a last_test)"
    }
  ],
  "asset": "XAUUSD",
  "asset_card_sha256": "6aa3ff6596aceed8cf3f2947b5981e4d962fafec185775bd3cfae7b8819c4667",
  "notes": [
    "asset_card_sha256: ficha de costes actual, no la del backtest (no existe la histórica)"
  ]
```

Tres fuentes, tres significados, y conviene no mezclarlas:

- **`last_test`** es lo que de verdad se usó: SQX lo guarda dentro del `.sqx` al hacer el test. Aquí: spread 0, slippage 0, comisión del 0,01 % y swap en porcentaje.
- **`backtest`** es lo que la tarea del proyecto dice **hoy**. Si alguien cambió la tarea después del test, no coinciden y una nota lo avisa: «los costes de «…» hoy no son los del último test».
- **`asset_card_sha256`** es la huella de la ficha del activo **de hoy**. `assets/` no guarda historia, así que no es la del backtest; la nota lo dice siempre.

Si varias tareas escriben en el mismo databank (una gemela desactivada, por ejemplo), salen todas y una nota lo dice; no se elige ninguna por ti.

Un ejemplo completo

1. Eliges una estrategia de un databank, por ejemplo la carpeta `OOS` del proyecto XAUUSD del maestro.
2. Lanzas el comando de arriba con su `--cfx`.
3. Lees `direction` y `signals` para saber qué hace, `orders` para ver si lleva stop, y `last_test.costs` para el coste con el que se calcularon sus números.
4. Si `notes` trae algo más que el aviso de la ficha, léelo antes de fiarte de los costes.

Para una estrategia construida por el builder genérico (no desde plantilla) las señales del lado que no opera salen como `Boolean(Value=false)`: es SQX desactivando ese lado, no un error.

Qué NO te dice

- No dice si la estrategia es buena ni cuánto gana: no mira métricas.
- No traduce la lógica a pseudocódigo ni la ejecuta.
- No sabe qué ficha de costes estaba vigente el día del backtest: solo la de hoy.
- Los parámetros de un bloque propio salen con el nombre que les dio SQX (`Int2`, `Double3`), no con el significado que tienen en el bloque.
- Las horas de `trading_options.weekend` salen tal cual las guarda SQX. En el maestro aparecen como `00:38`: parece que en la tarea se escribió `2300` en un campo que va en segundos; mientras esa opción esté apagada (`on: false`) no afecta a nada.

Si algo falla

- **Una nota «la tarea «…» no se pudo leer (falta Retest-Task8.xml en el .cfx)»**: el proyecto está roto, le falta el fichero de una tarea, y no se puede saber si esa tarea escribe en el databank. `python3 -m sqx.inspect.project_health` lo confirma. El resto de la salida sigue valiendo.
- **Una tarea de `backtest` con `costs: null`** y la nota «no lleva costes en su XML»: es un Custom analysis, que no hace backtest; valen los costes de `last_test`.
- **La nota «lee sus costes de CustomData»**: es un Automatic retest (MC, SPP, WFM, retest en mercados), que guarda su propio spread y slippage aparte; `setup_from` lo dice. Si no lleva swap, `swap` sale `null`.
- **`backtest: []` y la nota «ninguna tarea del proyecto escribe en este databank»**: el `.sqx` no está en la carpeta de un databank de ese proyecto, o la tarea escribe en el databank por defecto. `last_test` sigue siendo válido.
- **`asset: null`**: ningún fichero de `assets/symbols/` declara ese feed.

82. El perfil de mercado — qué comportamiento tiene cada activo, y si paga sus costes

Qué pregunta responde

Para cada uno de los 19 activos, en M15, M30, H1 y H4, y por separado para largos y cortos: **¿qué familia de comportamiento hay ahí —tendencia, ruptura, reversión, momentum, volatilidad, patrón o sesión—, con qué fuerza, y da para pagar el doble del coste?** Es la pieza 1 del director de investigación: el mapa que dice dónde merece la pena buscar una idea antes de gastar un build.

Cada medida se compara con la misma serie de precios remuestreada (mismas barras, otro orden): si el mercado real no se distingue de sus versiones barajadas, ahí no hay nada que explotar.

Cuándo lo usas, y cuándo no

Lo usas antes de pedir ideas: para saber en qué activo, marco y dirección hay estructura medible, y para darle al `ideaExpert` los parámetros de partida (qué umbral, qué horizonte).

No lo uses como prueba de que una estrategia vaya a sobrevivir. Dice dónde hay estructura en el precio del tramo de construcción; nada más. Tampoco lo relances «a ver si ahora sale»: cada pasada es una mirada más a los mismos datos y queda apuntada.

Antes de empezar

- Las barras M1 de los activos en `AlgoData/bars/` (capítulo `03-datos-costes-y-registro.pdf › 13-barras`). No toca SQX: se puede lanzar con cualquier instalación abierta.
- El tramo `build` de cada activo decidido en `assets/_policy.yaml`. **Sólo se lee ese tramo**: lo que viene después (`oos1`, `oos2`) se corta antes de calcular nada. Un activo sin tramo `build` se salta y se dice.
- Si un fichero de barras se ha modificado en la última media hora (se está reescribiendo), ese activo se salta y se dice; se vuelve a lanzar después sólo para él.

Cómo se ejecuta

```
cd ~/Desktop/AlgoProject
python3 -m studies.research.marketProfile.report                     # los 19 activos que falten
python3 -m studies.research.marketProfile.report --symbols XAUUSD --force   # rehacer uno
python3 -m studies.research.marketProfile.report --judge-only        # sólo rehacer las tablas
python3 -m studies.research.marketProfile.report --favourable        # sólo el ranking de familias por
activo
python3 -m studies.research.marketProfile.report --bibliography      # la bibliografía junto a lo
medido
python3 -m studies.research.marketProfile.report --sweep             # el barrido de salidas y
parámetros (estudio aparte)
python3 -m studies.research.marketProfile.report --sweep --judge-only # rejuzgar el barrido sin remedir
```

flag	obligatorio	qué hace
`--symbols`	no	activos a medir; sin él, todos los de `assets/symbols/`
`--force`	no	vuelve a medir un activo aunque ya esté guardado; sin él, lo ya medido no se repite
`--judge-only`	no	no mide nada: rehace la corrección, los filtros y las tablas con lo guardado
`--favourable`	no	no mide ni rejuzga: lee `scores.csv` e imprime qué familias son favorables en cada activo, ordenadas y con nota
`--bibliography`	no	no mide: imprime cada entrada del fichero de bibliografía (`AlgoData/research/literature/edges-2026-10-02.yaml`) con su fuente, su nota de verificación y lo que el mapa midió de ella
`--sweep`	no	lanza el **barrido de salidas y parámetros**, un estudio aparte que escribe en `sweep/`; con `--judge-only` sólo lo rejuzga
`--set`	no	cambia un ajuste de `config.yaml`, p. ej. `--set nulls.draws=2000`

Cuánto tarda (medido el 2026-10-02 con las 99 medidas y 3.000 sorteos, 19 procesos —uno por activo— y la máquina libre: `--set run.workers=19`): los 19 activos, **unos 40 minutos**; un par de forex solo, M15 ~25 min, M30 ~9, H1 ~5, H4 ~2. Con los 4 procesos por defecto, unas 3 horas. Memoria: 1,7 GB por proceso. El barrido (`--sweep`, 60 procesos, uno por celda): **21 minutos**.

El segundo comando mide la longitud de bloque del remuestreo (sólo hace falta si se quiere revisar esa decisión; tarda unos 25 minutos):

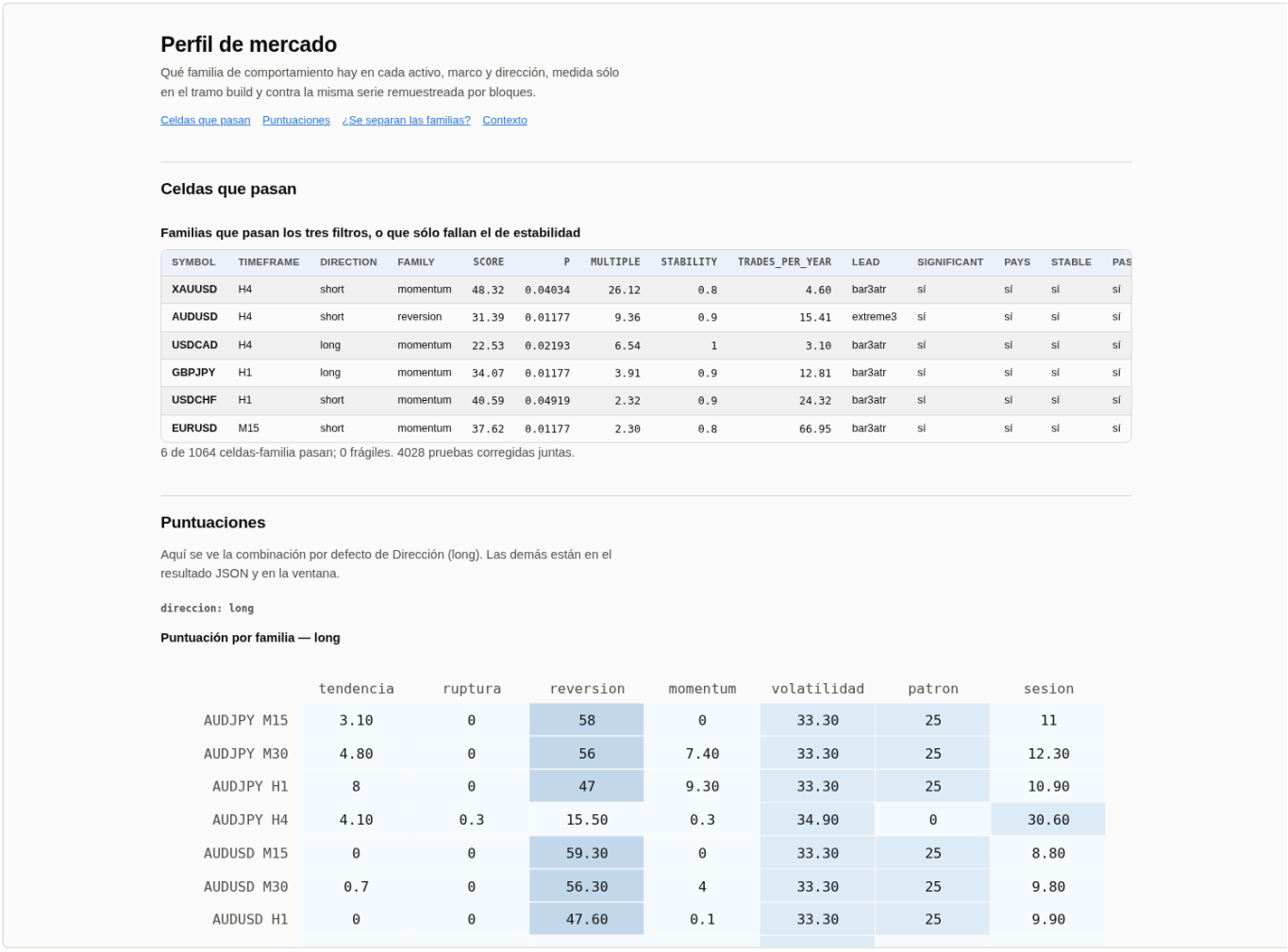
```
python3 -m studies.research.marketProfile.blocklen
```

Qué produce

Todo en `~/Desktop/AlgoData/research/profiles/`. Cada pasada **sobrescribe** las tablas; los ficheros de `cells/` sólo se rehacen con `--force`.

archivo	qué es
`scores.csv`	**el mapa**: una fila por activo × marco × dirección × familia, con su puntuación, su p corregida, el efecto en múltiplos del coste, la estabilidad y si pasa los cuatro filtros
`measures.csv`	las 14.364 medidas en crudo (con `tag`: qué añade cada una a las 31 originales; `needs_clock`; y `q_family`, `passes_family`: la corrección alternativa), cada una con su valor, su nula, su p, sus operaciones, y el detalle (mejor hora, tamaño de la ruptura, vida media…)
`context.csv`	por activo y marco: deriva, agrupamiento de la volatilidad, **coste dividido entre ATR**
`correlation.csv`	cuánto se parecen las puntuaciones de dos familias a lo largo de las celdas
`overlap.csv`	por celda: cuánto coinciden en las mismas barras las entradas de dos familias
`profile.html`, `.md`, `.json`	lo mismo, para leer
`ledger.jsonl`	una fila por medida, en el formato del ledger (aún no volcada al ledger global)
`run.json`	la semilla, qué se saltó y por qué
`blocklen.csv`	la medición de la longitud de bloque
`by_timeframe.csv`, `clock.csv`	de `--favourable`: cada activo × familia en cada marco, y lo medido con reloj que no se propone
`variance_ratio.csv`	cociente de varianzas VR(8) y VR(32) por activo y marco, y en cuántos años de build queda bajo 1
`sweep/variants.parquet`, `best.csv`, `counts.csv`, `ledger.jsonl`	el barrido: cada variante, la mejor por celda-familia con su meseta, los tres recuentos por marco y sus filas de ledger

Cómo se lee el resultado



Perfil de mercado

Qué familia de comportamiento hay en cada activo, marco y dirección, medida sólo en el tramo build y contra la misma serie remuestreada por bloques.

Celdas que pasan · Puntuaciones · ¿Se separan las familias? · Contexto

Celdas que pasan

Familias que pasan los tres filtros, o que sólo fallan el de estabilidad

SYMBOL	TIMEFRAME	DIRECTION	FAMILY	SCORE	P	MULTIPLE	STABILITY	TRADES_PER_YEAR	LEAD	SIGNIFICANT	PAYS	STABLE	PAS
XAUUSD	H4	short	momentum	48.32	0.04034	26.12	0.8	4.60	bar3atr	sí	sí	sí	sí
AUDUSD	H4	short	reversion	31.39	0.01177	9.36	0.9	15.41	extreme3	sí	sí	sí	sí
USDCAD	H4	long	momentum	22.53	0.02193	6.54	1	3.10	bar3atr	sí	sí	sí	sí
GBPJPY	H1	long	momentum	34.07	0.01177	3.91	0.9	12.81	bar3atr	sí	sí	sí	sí
USDCHF	H1	short	momentum	40.59	0.04919	2.32	0.9	24.32	bar3atr	sí	sí	sí	sí
EURUSD	M15	short	momentum	37.62	0.01177	2.30	0.8	66.95	bar3atr	sí	sí	sí	sí

6 de 1064 celdas-familia pasan; 0 frágiles. 4028 pruebas corregidas juntas.

Puntuaciones

Aquí se ve la combinación por defecto de Dirección (long). Las demás están en el resultado JSON y en la ventana.

`direccion: long`

Puntuación por familia — long

	tendencia	ruptura	reversion	momentum	volatilidad	patron	sesion
AUDJPY M15	3.10	0	58	0	33.30	25	11
AUDJPY M30	4.80	0	56	7.40	33.30	25	12.30
AUDJPY H1	8	0	47	9.30	33.30	25	10.90
AUDJPY H4	4.10	0.3	15.50	0.3	34.90	0	30.60
AUDUSD M15	0	0	59.30	0	33.30	25	8.80
AUDUSD M30	0.7	0	56.30	4	33.30	25	9.80
AUDUSD H1	0	0	47.60	0.1	33.30	25	9.90

Cuatro filtros, y una celda sólo cuenta si pasa los cuatro:

1. **Significativa** (`significant`): su p, corregida junto con todas las demás pruebas del mapa (Benjamini-Hochberg al 5 %), sigue por debajo de 0,05. Con 14.364 pruebas, sin corregir saldrían 700 «hallazgos» por puro azar. Al lado va siempre la **corrección alternativa** (`q_family`, `passes_family`: la misma, pero dentro de cada familia); el veredicto es el de cabecera y la elección entre las dos es tuya.
2. **Paga** (`pays`): el efecto medio por operación es al menos **2 veces** el coste de ida y vuelta del activo (spread + slippage en las dos patas + comisión, de `assets/`). La columna `multiple` es ese cociente: 2,3 es justo; 26 es mucho, pero mira cuántas operaciones lo sostienen.
3. **Estable** (`stable`): el efecto tiene el signo bueno en más de la mitad de los años de construcción. `stability` 0,8 = 8 años de cada 10. Si es significativa y paga pero vive de pocos años, sale como `fragile`.
4. **Frecuente** (`frequent`): la medida da al menos **40 operaciones al año** en build (`trades_per_year`; tu decisión del 2026-10-02). Si un activo concreto lo justifica puede bajar a **35**, nunca menos: se anota en `filters.min_trades_per_year_by_asset` de `config.yaml` (hoy vacío) y el perfil se niega a arrancar con un número menor. Un múltiplo enorme con 4 operaciones al año ya no pasa: no llega a una estrategia con significancia estadística.

La **puntuación** (`score`, 0-100) resume lo lejos que está la familia de su azar: un ruido blanco da en torno a 8; por encima de 40 hay algo claro. Una puntuación alta **no** basta: la familia «sesión» puntúa alto en muchas celdas (la hora importa de verdad) y no pasa en ninguna, porque el efecto de una barra no paga el coste.

`trades_per_year` y `frequent` van al lado de los otros tres filtros a propósito: las celdas que fallan sólo ése se pueden ver en `scores.csv` (`significant`, `pays` y `stable` verdaderos, `frequent` falso). Se rehace sin remedir con `--judge-only`.

Lo que mide desde el 2026-10-02 (la ampliación)

Las 31 medidas originales miraban como mucho 55 velas atrás, mantenían 16, no leían ningún marco superior y salían siempre por número de velas. Se añadieron 68:

- **Horizonte largo:** 100 y 200 velas; momentum de D1 a 20, 60, 120 y 252 días; canales de D1 de 20, 55 y 85 días con salida por el canal contrario o trailing.
- **Contexto de D1:** las mismas entradas con filtro de tendencia (cierre de D1 sobre su SMA200) y la compra de caídas de libro (RSI(2) de D1 < 10 sobre SMA200, salida sobre la SMA5).
- **Régimen de volatilidad:** ATR de D1 corto frente al largo, y frente a su mediana de 250 días.
- **Salidas:** vuelta a la media, trailing en ATR, y la misma entrada mantenida más tiempo.
- **Bibliografía:** las reglas del fichero de bibliografía que no estaban cubiertas (33 medidas).

Una vela **sólo ve los días de D1 ya cerrados**, nunca el suyo. Todas las medidas nuevas llevan una posición a la vez, a mercado, y **ninguna usa reloj**. Las cuatro que sí lo usan (mejor hora, mejor franja, mejor día, rango de una franja) se siguen midiendo, pero no lideran familia, no reciben nota y no llegan al tablero (tu decisión): quedan en la nota «medido pero descartado por usar reloj».

El barrido de salidas y parámetros (`--sweep`)

Estudio **aparte**, con su carpeta y su propia corrección. Pregunta: «¿esta estructura pasa el listón con una salida razonable y parámetros cercanos?». Trece entradas × tres parámetros vecinos × nueve salidas (cuatro duraciones, dos trailing, dos salidas por condición, un stop) × largo y corto = 702 variantes por celda, 53.352 pruebas. Para cada celda-familia da la mejor variante y su **meseta**: cuántas vecinas también pagan 2×. Una meseta es evidencia; un pico aislado, no. **Una variante elegida aquí es una hipótesis seleccionada dentro de muestra, no un hallazgo**: sirve para decidir dónde gastar un build de SQX, que es donde está la prueba fuera de muestra. Resultado del 2026-10-02: 3 variantes pasan, ninguna en meseta; por marco, no pagan 2× con ninguna variante 124 de 190 celdas-familia en M15, 108 en M30, 80 en H1 y 39 en H4.

Familias favorables por activo (`--favourable`)

`--favourable` convierte el mapa en una lista por activo, **de la familia más favorable a la menos**, y deja fuera lo que no llega al listón. No mide ni corrige nada: lee el `scores.csv` que ya hay, así que tarda un segundo. Cada familia lleva una nota: **A** pasa los cuatro filtros; **B** es significativa y paga 2× pero tiene pocas operaciones o es frágil (dice cuál); **C** es estable y frecuente con una sola flaqueza (significativa pero paga entre 1× y 2×, o paga 2× con p sin corregir ≤ 0,05 que la corrección no sostiene). Junto a cada una, la paleta de su condición fija y las paletas de las que sortea el hueco libre. Aparte, las familias **medidas como malas** en cada activo: estructura real que no paga ni medio coste, ninguna celda que llegue a medio coste, o signo contrario. Imprime además cada activo × familia **en los cuatro marcos** (para que M30 y H1 no queden escondidos tras la mejor celda), lo que cambia con la corrección alternativa y lo descartado por reloj; escribe `favourable.csv`, `avoid.csv`, `by_timeframe.csv` y `clock.csv` junto al mapa. La lectura comentada, activo por activo, está en `assets/FAMILIAS.md`, que desde el 2026-10-02 **ordena por tu prior** (tu documento de familias por activo) y anota cada entrada con lo medido: «medido a favor», «sin evidencia medida» o «medido en contra». Si rehaces el perfil, pide que se rehaga.

Un ejemplo completo

```
$ python3 -m studies.research.marketProfile.report --force --set run.workers=19
PROGRESS 4 DJ30: ok {'M15': 572.6, 'M30': 272.4, 'H1': 217.3, 'H4': 107.6}
...
PROGRESS 100 hecho

19 activos, 14364 pruebas, 2 de 1064 celdas-familia pasan los cuatro filtros, 0 frágiles; con la
corrección alternativa (por familia) pasan 4
symbol timeframe direction   family   score      p  multiple  stability  trades_per_year            lead
  DJ30        H1      long  ruptura 18.3050 0.0295    3.1434        1.0          71.1942 channel55_up200
EURUSD       M15     short momentum 21.6974 0.0207    2.3030        0.8          66.9458         bar3atr
-> /home/sergioguslw/Desktop/AlgoData/research/profiles
```

Lectura: en DJ30 H1, comprar la ruptura del máximo de 55 velas **sólo con el cierre de D1 sobre su SMA200** y mantener 8 velas ganó 3,1 veces el coste, todos los años, 71 veces al año. En EURUSD M15, vender tras una vela bajista de 3 ATR y mantener 4 velas ganó 2,3 veces el coste. Son pistas para una idea, no estrategias. Con la corrección por familia pasan otras cuatro celdas, todas de reversión (EURUSD M15, M30 y H1 corto; USDCAD H4 largo).

Qué NO te dice

- **No dice que una plantilla vaya a sobrevivir.** Nadie ha comprobado todavía que las celdas con puntuación alta den más supervivientes; eso se sabrá con 20-30 corridas cerradas.
- **El múltiplo no incluye el swap.** Las medidas que mantienen 16 velas H4 pasan dos o tres noches.
- **El efecto de los largos incluye la deriva del activo.** La p ya la descuenta (la nula tiene la misma deriva); el múltiplo no.
- **«Mejor hora», «mejor franja» y «mejor día»** se eligen mirando: su p paga esa elección, su múltiplo no.
- **La familia «volatilidad» puntúa 33 en casi todas las celdas** porque la volatilidad se agrupa en todos los mercados; eso no es una ventaja operable por sí sola.
- **Que no pase casi nada no es un fallo del instrumento.** Auditado el 2026-10-02: la reversión es significativa casi en todas partes y paga 0,2 veces el coste; la tendencia de un solo activo no se distingue del azar a ningún horizonte con 8-10 años; la compra de caídas en índices paga 6-18 veces el coste con 5-8 operaciones al año. Ni las salidas ni los parámetros lo cambian (barrido).
- **El múltiplo de las medidas que mantienen días** es grande casi siempre en índices (deriva): mira la p.
- Hereda la calidad del feed del tramo `build` (en XAUUSD, el spread anterior a 2017 es un modelo).

Si algo falla

- `SALTADO <activo>: feed modificado hace poco o durante la lectura` — otro proceso está reescribiendo sus barras. Espera media hora y lanza `--symbols <activo>`.
- `SALTADO <activo>: sin tramo build` — faltan sus fechas en `assets/_policy.yaml`.
- Va muy lento con la máquina ocupada: baja `--set run.workers=2`; nunca subas de 4 mientras el custodio trabaja.

81. Memoria de resultados — qué se probó, dónde murió y por qué, y qué ideas se gastaron

Qué pregunta responde

Antes de decidir en qué mercado investigar hace falta saber qué ya se intentó. Esta memoria junta en una tabla cada plantilla que se ha corrido en un activo y un marco temporal, cuántas estrategias quedaban tras cada paso, en cuál murió la población y cuántas llegaron al final. Y lleva el índice de todas las ideas que el `ideaExpert` ha propuesto, elegidas o no, con cuántas hipótesis midió para llegar a ellas. Desde el 2026-10-02 dice también **por qué** murió cada corrida cerrada —una causa de una lista cerrada, qué descarta y qué sugiere probar— y compara lo que la idea predijo antes del build (su hoja de hipótesis) con lo que el build hizo. No lanza nada y no toca SQX: solo lee lo que ya está escrito.

Cuándo lo usas, y cuándo no

Lo usas para ver de un vistazo qué celdas (activo × marco × dirección × familia) están sin tocar, qué familias han dado supervivientes y cuántas ideas se llevan gastadas en cada celda. Es la base del tablero del director de investigación.

No lo uses para juzgar una estrategia concreta (para eso están los estudios de los pasos 8 a 19) ni para sacar conclusiones con las corridas de hoy: son pruebas de la cadena, no evidencia.

Antes de empezar

No hace falta nada: ni SQX abierto ni worker. Lee `AlgoData/projects/registry.csv`, `AlgoData/templates/registry.csv`, `AlgoData/templates/runs.csv`, `AlgoData/autopilot/` y `AlgoData/ideas/`. Usa un solo núcleo y tarda un segundo.

Cómo se ejecuta

```
python3 -m studies.research.memory.report
```

flag	obligatorio	qué hace
`--out DIR`	no	escribe en otra carpeta en lugar de `AlgoData/research/memory/`
`--close RUN_DIR`	no	en vez de reconstruir, cierra la fila de `runs.csv` de un run terminado del autopiloto
`--runs-csv FICHERO`	no	con `--close`: actualiza esa tabla en lugar de `templates/runs.csv` (para probar sobre una copia)
`--cell ACTIVO MARCO DIRECCION FAMILIA`	no	imprime la memoria de esa celda y sale: sus fracasos cerrados con causa, las corridas que no son evidencia y las ideas ya propuestas ahí. Es la página que leen el `ideaExpert` y el `ideaReviewer`

Cuando el autopiloto termina un run llama a esa misma función solo; a mano solo hace falta para runs anteriores.

Qué produce

En `AlgoData/research/memory/`, siempre reconstruido desde cero:

fichero	qué es
`attempts.csv`	una fila por proyecto corrido: plantilla, familia, activo, marco, dirección, estrategias tras cada paso (`built`, `oos`, `gate`, `markets`, `mcr`, `spp`), `died_at`, `survivors`, `outcome`, `custodian_hours`, `dev_cut`, `verdict`, y las columnas nuevas: `cause`, `cause_share`, `causes`, `rules_out`, `try_next`, y las de la hoja (`sheet_…`, `built_…`, `trades_ratio`, `holding_ratio`, `hypothesis`)
`failures.csv`	los fracasos cerrados de todas las celdas, con su causa, qué descartan y qué probar
`ideas.csv`	una fila por idea: activo, marco, dirección, hipótesis medidas, si viene de un libro, si ya es plantilla
`ideas_spent.csv`	ideas ya propuestas por celda, con las hipótesis medidas
`survivors_by_family.csv`	por familia y clase de activo: intentos, cerrados, con supervivientes
`untouched_cells.csv`	las celdas de la rejilla (19 activos × M15-H4 × largo/corto × 7 familias) sin ningún intento

Con `--close` escribe en `templates/runs.csv`: rellena `strategies_built`, `strategies_kept` y `verdict`. Un veredicto que escribas tú a mano nunca se sobrescribe.

Cómo se lee el resultado

```
28 intentos, 2 con veredicto en runs.csv, 8 con embudo; 3 ideas en el indice; 1058/1064 celdas sin tocar
project                              template             symbol  timeframe  direction  built  oos  gate  markets  mcr  spp  custodian_hours  outcome
smoke_keltnerUpperCrossUp            keltnerUpperCrossUp  XAUUSD  H1         long       30                                                      unknown
XAU_ISOOS_ejemplo                    keltnerUpperCrossUp  XAUUSD  M30        long       120                                                     unknown
TestUSDJPY_Workflow_v1               emaCloseAbove        USDJPY  H1         long       50                                                      unknown
USDJPY_emaCross_H1                   emaCloseCrossUp      USDJPY  H1         long       100                                                     unknown
Test_USDJPY_builderSessionAuto       donchianUpperCrossUp USDJPY  M30        long                                                               unknown
Test_USDJPY_donchianUpperCrossUp_M30 donchianUpperCrossUp USDJPY  M30        long       200                                                     unknown
Test_USDJPY_mcrRanges                donchianUpperCrossUp USDJPY  M30        long                                                               unknown
Test_USDJPY_crosstfDiag_H1           donchianUpperCrossUp USDJPY  H1         long                                                               unknown
Test_USDJPY_donchianUpperCrossUp_H1  donchianUpperCrossUp USDJPY  H1         long       300                                                     unknown
Test_Calib_USDJPY_H1                 donchianUpperCrossUp USDJPY  H1         long                                                               unknown
Test_Calib_USDJPY_H1_freeL           freeShellLong        USDJPY  H1         long                                                               unknown
Test_Calib_USDJPY_H1_freeL_rnd       freeShellLong        USDJPY  H1         long                                                               unknown
Test_Calib_USDJPY_H1_freeS           freeShellShort       USDJPY  H1         short                                                              unknown
Test_Calib_USDJPY_H1_freeS_rnd       freeShellShort       USDJPY  H1         short                                                              unknown
Test_Calib_XAUUSD_H1                 donchianUpperCrossUp XAUUSD  H1         long                                                               unknown
Test_Calib_XAUUSD_H1_freeL           freeShellLong        XAUUSD  H1         long                                                               unknown
Test_USDJPY_LiveChain_H1             donchianUpperCrossUp USDJPY  H1         long                                                               unknown
Test_XAUUSD_GuiChain_M30             timeRangeBreakout    XAUUSD  M30        long                                                               unknown
Test_XAUUSD_LiveChain_M30            timeRangeBreakout    XAUUSD  M30        long                                                               unknown
Test_XAUUSD_StopTime_H1              donchianUpperCrossUp XAUUSD  H1         long                                                               unknown
Test_XAUUSD_donchianUpperCrossUp2_H1 donchianUpperCrossUp XAUUSD  H1         long                                                0.05           failed@6+7
Test_XAUUSD_donchianUpperCrossUp3_H1 donchianUpperCrossUp XAUUSD  H1         long       60     60   5     5                       0.33           failed@11
Test_XAUUSD_donchianUpperCrossUp_H1  donchianUpperCrossUp XAUUSD  H1         long              128  5     5        5    5     0.82           survivors
Test_XAUUSD_emaCloseCrossUp2_H1      emaCloseCrossUp      XAUUSD  H1         long       300    300  5     5                       0.31           failed@12
Test_XAUUSD_emaCloseCrossUp_H1       emaCloseCrossUp      XAUUSD  H1         long                                                0.03           failed@6+7
Test_XAUUSD_timeRangeBreakout2_M30   timeRangeBreakout    XAUUSD  M30        long       571    571  5     5        5    5     1.07           survivors
Test_XAUUSD_timeRangeBreakout3_M30   timeRangeBreakout    XAUUSD  M30        long                                                0.0            failed@6+7
Test_XAUUSD_timeRangeBreakout_M30    timeRangeBreakout    XAUUSD  M30        long       300                                      0.07           failed@7
```

`outcome` dice cómo acabó cada intento:

- `survivors`: llegó al paso 15 con estrategias.
- `died@<tramo>`: una tarea dejó cero estrategias (ahí murió).
- `failed@<paso>`: el run falló. **No es un veredicto**: no se sabe qué habría pasado.
- `incomplete@<tramo>`: se paró antes de acabar.
- `unknown`: no hay run del autopiloto; solo se sabe lo que dice `runs.csv`.

`dev_cut = yes` marca los runs en modo desarrollo, donde el paso 8 se queda con 5 estrategias al azar. Esas cinco no son evidencia de nada: por eso en la tabla de supervivientes por familia esos runs no cuentan como cerrados.

El veredicto que escribe el cierre automático tiene esta forma:

```
$ python3 -m studies.research.memory.report --close AlgoData/autopilot/Test_XAUUSD_timeRangeBreakout2_M30/20261001-103130 --runs-csv copia.csv

template         timeRangeBreakout
symbol           XAUUSD
timeframe        M30
project          Test_XAUUSD_timeRangeBreakout2_M30
strategies_built 571
strategies_kept  5
verdict          auto: embudo built 571 -> oos 571 -> gate 5 -> markets 5 -> mcr 5 -> spp 5; llegaron 5 al final
                 del paso 15; corte de desarrollo en el paso 8 (sorteo): no es evidencia
```

Por qué murió: la causa

Cada corrida cerrada lleva una **causa**, una sola palabra de una lista cerrada. La lista está en `studies/research/memory/config.yaml` (sección `causes`) y está **aprobada por ti (2026-10-02)**: son 12 causas; se cambia ahí, sin tocar código.

causa	qué quiere decir	qué descarta
`no_population`	el build no da población	esa condición fija tal como está escrita, en ese activo y marco
`too_few_trades`	opera demasiado poco	esa regla en ese marco
`too_many_trades`	opera demasiado (por encima del techo, donde el coste se come el filo)	esa regla en ese marco
`cost_kills`	el coste se come el filo	movimientos de ese tamaño en ese activo, con cualquier regla
`oos_decay`	se degrada fuera de muestra	esa regla
`not_beyond_random`	no bate al azar ni a la deriva del mercado	que la entrada aporte algo
`fragile_parameters`	frágil a parámetros y perturbaciones	esa regla: vive en un punto, no en una meseta
`single_market`	sólo funciona en su mercado o en su marco	que el mecanismo sea general
`funding_rule`	rompe una regla de fondeo (peor día, flotante, drawdown)	esa regla para fondeo
`hypothesis_mismatch`	el build no hizo lo que la hoja predijo	leer el resultado como prueba del mecanismo
`infrastructure`	fallo técnico	nada: **no es evidencia contra la idea**
`unclassified`	murió y ninguna razón escrita dice por qué	nada todavía

De dónde sale. El autopiloto, al juzgar un paso, escribe por cada estrategia qué regla no pasó (`paso-<n>/verdict.csv`, columna `reason`: por ejemplo `fail: gate.scorecard.estaticas_value=-10.0`). La memoria mira el paso donde la población se quedó a cero, traduce cada regla fallada a su causa y se queda con **la causa que tumbó a más estrategias**. `cause_share` dice cuántas («2/4»: tumbó a 2 de las 4 que fallaron) y `causes` las enumera todas. Si la población murió en una tarea de SQX sin que ningún juez escribiera razón, se usa la causa por defecto de ese tramo. Un run que falló (`failed@N`) es siempre `infrastructure`. Los runs en modo desarrollo siguen sin contar.

La hoja de hipótesis: lo que la idea predijo frente a lo que salió

Desde el 2026-10-02 cada idea de una propuesta del director (cap. 83) fija **antes del build** cuatro cosas: cuántas operaciones al año hará la estrategia mediana, cuántas horas durará su operación mediana, dónde **no** debería funcionar y qué resultado la mata. La propuesta la sella (guarda una huella: si alguien cambia la hoja después del build, la memoria lo dice: `altered`).

Al reconstruir la memoria se compara con lo que el build hizo de verdad, leído de las operaciones del tramo de construcción que exportó el databank `Results`:

- `trades_ratio` y `holding_ratio` son **construido ÷ predicho**. 1 es clavado; 0,4 es que opera menos de la mitad de lo que se dijo.
- `hypothesis` vale `pending`, `match` o `mismatch`.

El umbral lo fijaste tú: factor 2 en ambos sentidos (`sheet.tolerance: 2` en `studies/research/memory/config.yaml`), es decir, más del doble o menos de la mitad, en operaciones por año o en horas de tenencia (horas de reloj, fines de semana incluidos). Un desajuste pasa a ser una causa más: `hypothesis_mismatch`. Si la corrida tiene supervivientes, el desajuste solo se anota: no las quita. El revisor también solo anota, en una pasada, con Opus. Si `sheet.tolerance` volviera a `pending`, la memoria enseñaría los cocientes sin tipificar nada.

```
python3 -m studies.research.memory.report --cell XAUUSD H1 long ruptura
# MEMORIA DE LA CELDA XAUUSD H1 long ruptura
# Vocabulario de causas: approved.
#
# Fracasos cerrados (0) — quien proponga aquí adopta o rechaza cada uno por escrito:
#
# Corridas de la celda que no son un fracaso cerrado (6):
# - Test_XAUUSD_donchianUpperCrossUp2_H1 · plantilla donchianUpperCrossUp · failed@6+7 · fallo técnico:
no es evidencia
# - Test_XAUUSD_donchianUpperCrossUp_H1 · plantilla donchianUpperCrossUp · survivors · corte de
desarrollo: no es evidencia
# ...
# Ideas ya propuestas en XAUUSD H1 long, de cualquier familia (1):
# - squeezeReleaseBBUp · 2026-10-01 · no elegida · AlgoData/ideas/XAUUSD/2026-10-01-impulso-h4.md
```

Un ejemplo completo

```
python3 -m studies.research.memory.report
# 28 intentos, 2 con veredicto en runs.csv, 8 con embudo; 3 ideas en el indice; 1058/1064 celdas sin
tocar
```

Hoy hay 28 intentos, 8 con embudo conocido, ninguno cerrado de verdad (los que llegaron al final son de modo desarrollo) y solo 6 de 1.064 celdas tocadas, todas de ruptura y tendencia en XAUUSD y USDJPY.

Qué NO te dice

- Que una familia funcione o no: con pocos intentos cerrados la tasa no vale nada. Mira `closed`.
- Las horas son de reloj del custodio (todos los núcleos), no horas de CPU; y están vacías donde el autopiloto no corrió.
- Un `unknown` no es un fracaso: es que nadie anotó el embudo.
- **Hoy ninguna corrida tiene una causa de verdad**: todas las cerradas son de modo desarrollo (el paso 8 sortea 5) o fallaron por algo técnico. Las causas empezarán a aparecer con la primera corrida juzgada con criterios.
- La causa es la que tumbó a **más** estrategias, no la única: mira `causes`.
- Las operaciones por año de la comparación salen un pelo altas (los años se miden de la primera entrada a la última salida de la población), y las horas son de reloj: un fin de semana cuenta.
- `from_book` en `ideas.csv` vale `unknown` mientras la idea no traiga una línea `Origen:`.

Si algo falla

- **Una fila con `failed@N`**: abre `AlgoData/autopilot/<proyecto>/<fecha>/fallo.md`.

- **`KeyError` o columnas raras en `runs.csv`**: alguien cambió su cabecera; el cierre no la adivina.
- **Una causa `unclassified`**: la regla que falló no está en la lista `causes.facts` de `studies/research/memory/config.yaml`; se añade ahí su fila.
- **`hypothesis` vale `no_data`**: el databank `Results` de ese proyecto no se exportó nunca.

83. Tablero de investigación — en qué celda investigar ahora, y el director que lo lee

Qué pregunta responde

De todas las celdas del mapa (activo × marco × dirección × familia), ¿en cuál merece la pena gastar las tres próximas ideas? El tablero ordena las celdas que tu prior da por Alta o que lo medido en el perfil de mercado admite (cap. 82) con tres factores a la vista —cuánto paga el efecto, lo poco que se ha probado ahí y cómo le ha ido a esa familia en esa clase de activo— y un freno por las ideas ya gastadas. Es **una página**: la que lee el director de investigación (el agente `researchDirector`) antes de proponerte nada.

Cuándo lo usas, y cuándo no

Lo usas antes de pedir ideas nuevas: para ver qué celdas hay y por qué una va delante de otra. Y con `/research-direct`, para que el director elija una, pida tres ideas al `ideaExpert` y te traiga la propuesta con sus bloques.

No lo uses para juzgar una estrategia ni una plantilla: el tablero dice dónde hay estructura medible y sin probar, no que una plantilla vaya a sobrevivir.

Qué entra (tu decisión del 2026-10-02: «usa las dos cosas»; sustituye a «las celdas grises no entran»): una celda-familia entra si **la prior** —tu documento de familias por activo, transcrito en `studies/research/board/prior.yaml` — la da por **Alta** (Alta en la matriz, o familia principal o secundaria de ese marco), **o** pasa los cuatro filtros del perfil («desnuda»), **o** una variante del barrido de salidas y parámetros pasa en meseta. La estadística suma sobre la prior; no es requisito. **Nunca entra una familia cuya regla necesite reloj.** Una celda con prior Baja o sin prior y sin apoyo medido se queda fuera. Donde la prior dice «(largo)», sólo entra el largo.

Cómo se ordena: puntos = 100 × freno × (0,30·prior + 0,30·evidencia + 0,10·señal + 0,15·hueco + 0,15·pasado). La prior vale 1 (Alta), 0,5 (Media) o 0. La evidencia es graduada: pasa desnuda 1 > barrido en meseta 0,8 > significativa y estable pero paga menos de 2× 0,5 > sin evidencia 0,25 > **medido en contra** 0, que además divide los puntos entre dos y sale con su aviso — la celda no se quita, porque te fías de la prior y el conflicto es lo que tienes que ver. Una celda que entra sólo por la prior dice «sin medir» en señal y en operaciones al año: no se inventa una frecuencia. El «pullback» de la prior no es una de las siete familias: sale bajo `reversion`, marcado «pullback: contexto de tendencia + disparo de reversión». La página imprime las 40 primeras; todas están en `board.json`. Lo medido con reloj queda en la nota «medido pero descartado por usar reloj» de `assets/FAMILIAS.md`.

Antes de empezar

Tiene que existir el perfil (`AlgoData/research/profiles/scores.csv`, cap. 82). La memoria (cap. 81) se recalcula sola cada vez. No toca SQX, no necesita ningún worker, usa un núcleo y tarda un segundo.

Cómo se ejecuta

```
python3 -m studies.research.board                      # el tablero
python3 -m studies.research.board.palette momentum     # qué familias de bloques admite el hueco libre
python3 -m studies.research.board.selfcheck            # ¿orienta el perfil? (necesita 20 corridas
cerradas)
python3 -m studies.research.board.proposal scratch/research-director/2026-10-02-draft.json
```

comando	qué hace
`studies.research.board` (`studies/research/board/__main__.py`)	imprime la página y la deja en `AlgoData/research/board/board.json` y `board.txt`
`studies.research.board.palette FAMILIA`	para una idea de esa familia (`tendencia`, `ruptura`, `reversion`, `momentum`, `volatilidad`, `patron`, `sesion`), las familias de bloques que puede usar el hueco aleatorio y con qué peso; `--blocks N` enseña N bloques de ejemplo
`studies.research.board.selfcheck`	el examen del perfil: con 20 corridas cerradas o más compara las celdas de puntuación alta con las de baja y acaba en «mantener» o «reducir» el peso de la señal
`studies.research.board.proposal BORRADOR.json`	lo usa el director: comprueba su borrador (tres ideas, cada una con su hoja de hipótesis y su revisión; los fracasos previos de la celda, adoptados o rechazados), le pone lo que calcula Python —incluido el sello de cada hoja— y lo guarda. `--step N --note "…"` apunta en qué paso va (de 1 a 6), para la ventana

El director, con sus tokens (10-30 $ por propuesta), se lanza con la skill:

```
/research-direct
```

a mano (el botón «Proponer investigación» de la ventana se quitó). La skill refresca la memoria, imprime el tablero, lanza al `researchDirector` y te enseña la propuesta. **No lanza nada en SQX.**

Los pesos y los umbrales están en `studies/research/board/config.yaml`.

Qué produce

fichero	qué es
`AlgoData/research/board/board.json`, `board.txt`	la página de hoy; se sobrescribe cada vez
`AlgoData/research/board/selfcheck.json`	el último examen del perfil
`AlgoData/research/proposals/<id>.json` y `<id>.md`	una propuesta del director: el `.md` es el que lees, el `.json` el que pinta la ventana
`AlgoData/research/proposals/estado.json`	el paso en que va el director mientras trabaja

Cómo se lee el resultado

```
TABLERO DE INVESTIGACION · 2026-10-01T23:46:22
6 celdas-familia pasan los tres filtros del perfil, de 1064 medidas; las otras 1058 no entran, tengan o no hueco de cobertura.

#  celda                    puntos  señal              hueco              pasado                          freno           op/año  avisos
1  AUDUSD H4 short reversion  84.3    9.4x coste (0.94)   0 intentos (1.00)  0/0 cerradas, 0.50 [0.14-0.86]  0 ideas (x1.00)  15.4    pocas operaciones (<40/año)
2  XAUUSD H4 short momentum   75.0    26.1x coste (1.00)  0 intentos (1.00)  0/0 cerradas, 0.50 [0.14-0.86]  1 ideas (x0.86)  4.6     pocas operaciones (<40/año)
3  USDCAD H4 long momentum    70.2    6.5x coste (0.65)   0 intentos (1.00)  0/0 cerradas, 0.50 [0.14-0.86]  0 ideas (x1.00)  3.1     pocas operaciones (<40/año)
4  GBPJPY H1 long momentum    57.1    3.9x coste (0.39)   0 intentos (1.00)  0/0 cerradas, 0.50 [0.14-0.86]  0 ideas (x1.00)  12.8    pocas operaciones (<40/año)
5  USDCHF H1 short momentum   49.1    2.3x coste (0.23)   0 intentos (1.00)  0/0 cerradas, 0.50 [0.14-0.86]  0 ideas (x1.00)  24.3    pocas operaciones (<40/año)
6  EURUSD M15 short momentum  49.0    2.3x coste (0.23)   0 intentos (1.00)  0/0 cerradas, 0.50 [0.14-0.86]  0 ideas (x1.00)  66.9    -

puntos = 100 × freno × (0.50·señal + 0.25·hueco + 0.25·pasado); entre paréntesis, cada factor de 0 a 1.
señal   efecto medio por operación de la medida líder, en múltiplos del coste de ida y vuelta; 10x o más cuenta como 1.
hueco   1/(1 + intentos de esa familia en esa celda): sin probar vale 1.
pasado  tasa de supervivencia de la familia en esa clase de activo, con su intervalo del 90%; con pocas corridas cerradas se queda cerca de la media, y una familia
        sin probar no baja. Ninguna familia tiene corridas cerradas: el factor «pasado» es plano (0.50 en todas).
freno   1/(1 + ideas ya gastadas en la celda / 6): cada idea más sube el listón de las siguientes.
op/año  operaciones al año de la medida líder en build; no es lo que hará una estrategia, es el orden de magnitud.
```

Cada fila es una celda. Junto al puesto van sus factores, el valor real y entre paréntesis el mismo de 0 a 1:

- **señal**: el efecto medio por operación de la medida líder, en múltiplos del coste de ida y vuelta. Hace falta 2 para entrar; 10 o más cuenta como señal completa.

- **hueco**: cuántas veces se ha corrido una plantilla de esa familia en esa celda. Sin probar vale 1; con un intento, 0,5.
- **pasado**: de las corridas cerradas de esa familia en esa clase de activo, cuántas dejaron supervivientes, con su intervalo. Con pocas corridas la tasa se queda cerca de la media y el intervalo es ancho; una familia sin probar no baja. **Hoy no hay ninguna corrida cerrada, así que es 0,50 en todas: el factor es plano y no ordena nada.**
- **freno**: las ideas que ya se han gastado en esa celda. Cada idea más baja los puntos, porque sube el listón de las siguientes.
- **op/año**: las operaciones al año de la medida líder. Ya no es un aviso: el mínimo (40, o 35 en un activo que tú nombres) es el cuarto filtro del perfil, y una celda que no lo alcanza no llega al tablero (tu decisión del 2026-10-02).
- **costes provisionales**: XAGUSD, UKOIL y USOIL, mientras no cierres sus costes.

`puntos = 100 × freno × (0,50·señal + 0,25·hueco + 0,25·pasado)`.

```
Hueco libre para una idea de momentum (momentum):
  peso 3  session          25 bloques  ortogonal: lee otra clase de dato que la condición fija
          BarDayOfWeekIs, BarDayOfWeekIsNot, BarHourIs, BarHourIsBigger
  peso 3  volatility       84 bloques  ortogonal: lee otra clase de dato que la condición fija
          ATRChangesDown, ATRChangesUp, ATRCrossDown, ATRCrossUp
  peso 2  mean_reversion  273 bloques  contratendencia: lo contrario de la condición fija
          BBBarOpensAboveDownAfterOpenBelow, BBBarOpensBelowUpAfterOpenAbove, BHErgodicAboveLevel, BHErgodicBelowLevel
  peso 1  pattern          78 bloques  mismo dato, otra lectura
          BearishEngulfing, BullishEngulfing, CBlock_AdvanceBlock, CBlock_BullishBeltHold
  peso 0  breakout          0 bloques  FUERA: dos iguales, sólo encarece la entrada
  peso 0  momentum          0 bloques  FUERA: la propia familia de la condición fija
  peso 0  trend             0 bloques  FUERA: dos iguales, sólo encarece la entrada

Examen del perfil: hay 0 corridas cerradas y hacen falta 20. Todavía no se puede saber si el perfil orienta; el peso de la señal en el tablero se
       queda en 0.50.
```

La paleta aplica las dos reglas de los libros: el hueco lee **otra clase de dato** que la condición fija (peso 3: tiempo o volatilidad frente a precio) y **tendencia con contratendencia, nunca dos iguales** (peso 2 la contraria; fuera la propia familia y sus parecidas).

Lo que trae cada propuesta desde el 2026-10-02: hoja, revisión y fracasos previos

Una propuesta ya no se acepta sin estas tres cosas:

- **La hoja de hipótesis de cada idea.** Antes del build la idea fija cuántas operaciones al año hará la estrategia mediana, cuántas horas durará su operación mediana, dónde **no** debería funcionar y qué resultado la mata. La propuesta la sella. Cuando la corrida se cierra, la memoria la compara con lo que salió (cap. 81).
- **La revisión.** Un agente distinto del que propone, el `ideaReviewer`, recibe las tres ideas y la memoria de la celda e intenta tumbarlas por cuatro lados: el mecanismo, el coste, el parecido con lo ya probado y las lecturas ambiguas. **No mide nada sobre datos** (no tiene consola): no añade miradas a la cuenta de ensayos. Deja un veredicto por idea: `resiste`, `dudosa` o `tumbada`.
- **Los fracasos previos de la celda.** El `ideaExpert`, antes de proponer, lee la memoria de la celda y por cada fracaso cerrado escribe si lo **adopta** (acepta lo que descarta) o lo **rechaza** (y por qué no aplica).

Qué puede hacer el revisor. Decidiste (2026-10-02) que sólo anota: una pasada, con Opus, y una idea «tumbada» se lanza igual si no la vetas (`review.removes: false` en `studies/research/memory/config.yaml`). El veto es tuyo. Las causas de fracaso son el vocabulario de 12 que aprobaste, y el desajuste de la hoja usa factor 2 en ambos sentidos (cap. 81).

Lo que encarece. El revisor añade entre 1 y 3 $ a una propuesta que cuesta 10-30 $ (estimado por tamaño de lo que lee y escribe, sin medir todavía en una propuesta real).

Un ejemplo completo

```
$ python3 -m studies.research.board
TABLERO DE INVESTIGACIÓN · 2026-10-02T09:08:59
359 celdas-familia entran de 1064 medidas (las otras 705 no): 357 porque la prior las da por Alta, 2
porque pasan los cuatro filtros del perfil, 0 porque una variante del barrido pasa en meseta. Ninguna
familia de reloj entra. Se imprimen las 40 primeras; todas están en board.json.

#   celda                       puntos  prior         evidencia                 señal
hueco              pasado                          freno            op/año     entra por  avisos
1   DJ30 H1 long ruptura        70.6    Media (0.50)  pasa desnuda (1.00)       3.1x coste (0.31)  0
intentos (1.00)  0/0 cerradas, 0.50 [0.14-0.86]  0 ideas (x1.00)  71.2       desnudo    -
2   AUDUSD H1 long reversion    67.5    Alta (1.00)   significativa, <2x (0.50)  sin medir (0.00)   0
intentos (1.00)  0/0 cerradas, 0.50 [0.14-0.86]  0 ideas (x1.00)  sin medir  prior      -
3   AUDUSD H4 short reversion   67.5    Alta (1.00)   significativa, <2x (0.50)  sin medir (0.00)   0
intentos (1.00)  0/0 cerradas, 0.50 [0.14-0.86]  0 ideas (x1.00)  sin medir  prior      pullback:
contexto de tendencia + disparo de reversión
4   AUDUSD M30 long reversion   67.5    Alta (1.00)   significativa, <2x (0.50)  sin medir (0.00)   0
intentos (1.00)  0/0 cerradas, 0.50 [0.14-0.86]  0 ideas (x1.00)  sin medir  prior      -
...
$ python3 -m studies.research.board.selfcheck
Examen del perfil: hay 0 corridas cerradas y hacen falta 20. Todavía no se puede saber si el perfil
orienta; el peso de la señal en el tablero se queda en 0.50.
```

Lectura: primero la única celda que pasa desnuda (DJ30 H1 largo, ruptura; prior Media). Después, las celdas que la prior da por Alta y donde lo medido es significativo aunque pague menos de 2× (reversión en EURUSD, AUDUSD, USDCAD…), y después las de prior Alta sin evidencia. Las 44 «medido en contra» quedan al final, con su aviso.

Qué NO te dice

- Que la celda vaya a dar estrategias: el perfil no está probado como guía (por eso existe el examen).
- Cuántas operaciones dará una estrategia: `op/año` es de la medida, no de la estrategia; el filtro sólo garantiza que la medida no es rara.
- Una celda que entra sólo por la prior no tiene efecto ni frecuencia medidos («sin medir»): la prior es cualitativa.
- El factor «pasado» no dice nada hasta que haya corridas cerradas de verdad (hoy, cero).

Si algo falla

- `FileNotFoundError: …/profiles/scores.csv`: falta correr el perfil (cap. 82).
- `La propuesta no se acepta:` seguido de una lista: el borrador del director rompe una regla (no son tres ideas, una idea en otra dirección que la celda, una paleta con la misma familia que la condición fija, una pregunta sin sus lecturas, una idea sin su hoja de hipótesis o sin su revisión, un revisor que es el mismo que propone, un fracaso previo de la celda sin adoptar ni rechazar). Se corrige el borrador, no el comprobador.
- `la celda … no está en el tablero`: el director eligió una celda que ni la prior da por Alta ni lo medido admite.

90. Explorador de bibliografía — papers que entran solos, reglas ancladas a su cita

Qué pregunta responde

¿Qué reglas de trading se han publicado que no tengamos ya apuntadas, y cuáles de ellas se pueden probar aquí? El explorador busca papers en internet, descarta los que no nos sirven **diciendo por qué y con la frase del paper que lo demuestra**, y de los que sirven saca cada regla como una «entrada»: la regla, el activo, la muestra de los autores, cuántas variantes dicen haber probado y lo que ellos mismos cuentan fuera de muestra. Cada dato de la entrada lleva pegada la frase literal del paper y la página donde está.

Hasta ahora ese fichero (las 62 entradas de `edges-2026-10-02.yaml`) se hizo una vez y a mano. Esto es la parte que faltaba: encontrar, leer y apuntar. De la entrada en adelante todo existía ya: el perfil de mercado la mide (cap. 82), el tablero la ordena (cap. 83), el director propone y tú vetas (cap. 84).

Cuándo lo usas, y cuándo no

Lo usas cuando quieras más cantera publicada, o para ver qué trajo la pasada diaria: qué se rechazó, qué entró, qué está esperando a que tú contestes una pregunta.

No lo uses como fuente de estrategias listas. Una regla publicada es **la que sus autores eligieron enseñar**: trae dentro las que probaron y no contaron, y lo normal es que rinda menos después de publicarse. Por eso la entrada no es un prior: es una hipótesis más, que pasa por el perfil y por toda la cadena como cualquier otra.

Lo que el explorador rechaza antes de leer a fondo (cada rechazo queda guardado con su motivo):

filtro	se rechaza cuando…
`no_rule`	el paper no da una regla con entrada: teoría, un modelo de control óptimo, una red entrenada, un repaso
`cross_sectional`	ordena o mantiene varios activos a la vez, o necesita un universo de acciones
`fundamentals`	necesita algo que no es el precio del propio activo: beneficios, flujos, sentimiento, macro, otro activo
`clock`	la entrada o la salida miran la hora, la sesión, el día de la semana o el calendario (regla 15)
`not_sqx_at_market`	no se puede expresar con bloques de SQX entrando a mercado
`conditions`	la entrada necesita más de dos condiciones (regla 14)
`two_legs`	largo y corto a la vez (un spread, un par). Una regla simétrica sí pasa: cada lado se mide aparte (regla 13)
`unreadable`	no se pudo leer ni el texto completo ni un resumen que diga la regla
`no_backtest`	es una estrategia de un repositorio de código que no trae ningún resultado de backtest propio

Los papers de futuros se quieren (tu decisión del 2026-10-02): no se rechaza nada por operar futuros. Una cartera de futuros pasa si la señal se calcula contrato a contrato; y si el paper aporta un modelo pero declara las reglas clásicas con las que se compara, esas reglas entran.

Nada se deduce. Si el paper no dice un dato, la entrada pone «not stated in source»; no se rellena de memoria ni por el título. Si del paper sólo se pudo leer el resumen, la entrada lo dice. Y si la regla admite dos lecturas, el explorador **no elige**: apunta las dos y te pregunta (regla 11).

Antes de empezar

Nada de SQX: no toca ningún install ni necesita ningún worker. Hace falta internet. Usa dos fuentes gratuitas y sin clave para papers — además de GitHub para dos colecciones de estrategias —: **arXiv** (las categorías de finanzas cuantitativas) y **OpenAlex** (un índice abierto de artículos, que es también por donde se llega a los resúmenes de SSRN, porque SSRN no deja entrar a un programa). No hay nada que instalar: va con `httpx`, que ya estaba, y `pypdf` para leer los PDF.

De dónde lee (`sources` en `studies/research/literature/config.yaml`):

fuente	qué trae	por pasada
arXiv (finanzas cuantitativas)	papers con su PDF	hasta 20 papers entre las dos
OpenAlex	artículos y working papers con PDF abierto; es por donde se llega a SSRN. Sin clave tiene un cupo diario; si se agota, la pasada sigue sin él	
`StockSharp/AlgoTrading` y `rolling-panda-san/notebooks` (GitHub)	estrategias con su código	3 de cada uno
Oxfordstrat (`oxfordstrat.com/resources`)	unas 100 estrategias con reglas y backtest sobre 42 futuros	3
Quantpedia (parte gratuita)	82 fichas de estrategia, cada una resumen de un paper	3
AQR (índice de research)	artículos de la gestora	3

No se pueden leer con un programa (contestan 403 o piden clave): traders.com, SSRN directamente, las páginas del blog de Alpha Architect, Semantic Scholar y Google Scholar. Man Group, Robeco y Research Affiliates cargan sus listados con JavaScript y no están puestos. **No se ponen**: Open Source Asset Pricing, JKP Factors y la biblioteca de Kenneth French, porque son factores que ordenan acciones entre sí y datos, justo lo que rechaza el primer filtro.

Para medir las entradas nuevas tiene que existir el perfil de mercado (cap. 82).

Cómo se ejecuta

Lo normal es la skill, que hace los cuatro pasos y te enseña el resultado:

```
/literature-scout
```

Y sola, una vez al día, con `bin/daily-literature-scout.sh` (ver «La pasada diaria» más abajo).

Los comandos que hay debajo, por si quieres mirar tú:

```
python3 -m studies.research.literature.report                # estado: papers, rechazos, coste,
preguntas, cola
python3 -m studies.research.literature.report --rejections    # cada rechazo con su filtro, motivo y
cita
python3 -m studies.research.literature.report --queue         # sólo la cola de entradas
python3 -m studies.research.literature.report --for XAUUSD --family ruptura   # lo publicado sobre un
activo
python3 -m studies.research.literature.report --answer TR-59-maco-5min-covid 2  # tu respuesta a una
pregunta
python3 -m studies.research.literature.report --register      # rehace REGISTRO.md, el registro de
papers
```

comando	qué hace
`studies.research.literature.report` (`studies/research/literature/report.py`)	sin nada: papers vistos por estado, rechazos por filtro, lo que costó cada pasada, las preguntas abiertas y la cola
`--rejections`	todos los papers rechazados o repetidos, con el filtro, el motivo y la frase del paper
`--queue`	cada entrada del explorador con su estado y, si ya está medida, lo que salió en `build` y qué celdas llegan al tablero
`--for ACTIVO [--family FAMILIA]`	todas las entradas (las 62 a mano y las nuevas) que hablan de ese activo: es la cantera `libros` del `ideaExpert`
`--answer ID N`	escribe tu elección (la lectura número N) en la pregunta de esa entrada
`--register`	rehace `REGISTRO.md`: cada paper visto, si se leyó completo o sólo el resumen, y qué salió de él («útil» con sus entradas, «nada» con el motivo, «repetido»). Un paper que está ahí no se vuelve a buscar ni a leer

Los pasos de una pasada, que usan el agente y la skill (`studies/research/literature/scout.py`):

comando	qué hace	tokens
`python3 -m studies.research.literature.scout search [--papers 20]`	pregunta a arXiv y a OpenAlex por cada consulta de `config.yaml`, quita los papers ya vistos o ya apuntados y deja los nuevos en la bandeja	0
`… scout inbox`	los papers de la bandeja que aún no tienen decisión	0
`… scout fetch CLAVE`	guarda el texto del paper página a página (o sólo el resumen si el PDF no se deja descargar, y dice por qué)	0
`… scout excerpt CLAVE`	enseña el resumen y las seis páginas donde más se habla de la regla, los datos y el resultado: es lo que lee el agente, en vez del paper entero	0
`… scout reject CLAVE --filter F --quote "…" --where "p. 7" --reason "…"`	apunta un rechazo; **lo rehúsa si la cita no está tal cual en el texto guardado**	0
`… scout reopen CLAVE`	devuelve a la bandeja un paper rechazado, para que la siguiente pasada lo lea otra vez	0
`… scout accept BORRADOR.yaml`	comprueba el borrador de una entrada contra el texto guardado, mira que no sea una regla que ya tenemos, le pone número y la guarda	0
`… scout measure ID MEDIDAS.yaml`	añade a una entrada ya guardada las medidas con las que el perfil la probará	0
`… scout cost RUN --tokens N`	cierra la pasada con lo que costó, en tokens	0

Los tokens se gastan sólo al leer, y cada lectura es **una llamada única a Claude, sin herramientas** (tu decisión del 2026-10-02: los PDF los leen agentes, gastando poco, y el coste se dice en tokens): una llamada **criba** todos los resúmenes; luego **una llamada por paper** recibe el extracto y devuelve las entradas, que el programa comprueba y guarda (si rehúsa alguna, un reintento). La criba la hace sonnet y la lectura del paper opus (tu decisión: en leer y entender el paper se gasta lo que haga falta). Los textos de esas llamadas los monta `python3 -m studies.research.literature.calls` (`studies/research/literature/calls.py`: `triage-prompt`, `triage-apply`, `extract-prompt`, `extract-apply`, `under`); no se usa a mano.

Las entradas nuevas esperan en la cola sin medir: el mapa se regenera a mano. Medirlas en `build` (no toca SQX; usa CPU a prioridad baja):

```
nice -n 19 python3 -m studies.research.marketProfile.report --top-up
```

Sólo mide lo que falta en cada activo, no repite el mapa entero. Después vuelve a juzgar todo el mapa.

Qué produce

Todo en `AlgoData/research/literature/`:

fichero	qué es
`papers.jsonl`	un renglón por paper que la búsqueda ha devuelto alguna vez: de dónde salió, su resumen, y qué se hizo con él (`inbox`, `rejected` con filtro, cita y motivo, `accepted`, `duplicate`)
`papers/<clave>.txt`	el texto guardado de cada paper leído, con sus marcas de página: contra esto se comprueba cada cita
`scout-<día>.yaml`	las entradas aceptadas ese día, con el mismo formato que el fichero hecho a mano más las citas (`anchors`), la selección de los autores (`selection`) y la pregunta si la hay
`runs.jsonl`	un renglón por pasada: papers leídos, entradas, llamadas, tokens
`REGISTRO.md`	el registro para leer: un renglón por paper visto y qué salió de él

Y la pasada diaria deja `AlgoData/audit/<fecha>-bibliografia.md` con el estado del día. No sobrescribe el fichero de las 62 entradas: ese no lo toca nunca.

Cómo se lee el resultado

```
$ python3 -m studies.research.literature.report
70 papers vistos: 9 accepted, 3 duplicate, 58 rejected
   31 rechazados por `no_rule` — the paper states no trading rule with an entry (theory, a model, a forecast)
    5 rechazados por `unreadable` — neither the full text nor an abstract stating the rule could be read
    5 rechazados por `fundamentals` — needs data that is not the asset's own price (earnings, flows, macro, text)
    3 rechazados por `not_sqx_at_market` — not expressible with SQX building blocks entering at market
    8 rechazados por `clock` — the entry or the exit reads the hour, the weekday or the calendar (hard rule 15)
    4 rechazados por `two_legs` — long and short held together: not one direction per strategy (hard rule 13)
    1 rechazados por `conditions` — the entry needs more than two conditions (hard rule 14)
    1 rechazados por `cross_sectional` — ranks or holds several assets at once: not a one-asset time-series rule
run 20261002-140144 (opus): 20 papers, 5 entradas, 171,568 tokens · por paper 8,578 · por entrada 34314
run 20261002-143154 (sonnet): 20 papers, 4 entradas, 923,856 tokens · por paper 46,193 · por entrada 230964
run 20261002-143808 (sonnet): 20 papers, 3 entradas, 605,582 tokens · por paper 30,279 · por entrada 201861
run 20261002-144518 (sonnet): 7 papers, 5 entradas, 128,411 tokens · por paper 18,344 · por entrada 25682

PREGUNTA TR-59-maco-5min-covid: The paper calls the rule «the traditional moving average cross-over (MACO) rule», but its Equations (1) and (2) average r,
    defined as «the natural log return at the five-minute interval», not the price.
  1. As the equations are written: long when the mean of the last S log returns is above the mean of the last L log returns; out when it is below. (S, L) =
    (1,50), (1,200), (5,150).
  2. As the name says: long when the S-bar moving average of the price is above its L-bar moving average; out when it is below. (S, L) = (1,50), (1,200),
    (5,150).

PREGUNTA TR-61-price-above-sma-or-cash: «When the MA turns lower (higher) than the current daily closing price, we invest in the risky asset»: and «every fifth
    trading day (proxy for weekly rule)» — the meaning of the frequency is ambiguous.
[… otras 7 preguntas …]

17 entradas del scout: 8 «sin medida: falta una condición», 8 «pregunta al dueño», 1 «medida»

TR-56-tsmom-decaying-weights-china · tendencia · «sin medida: falta una condición»

VOL-57-bollinger-5min-covid · volatilidad · «pregunta al dueño»

TR-58-filter-rule-5min-covid · tendencia · «pregunta al dueño»

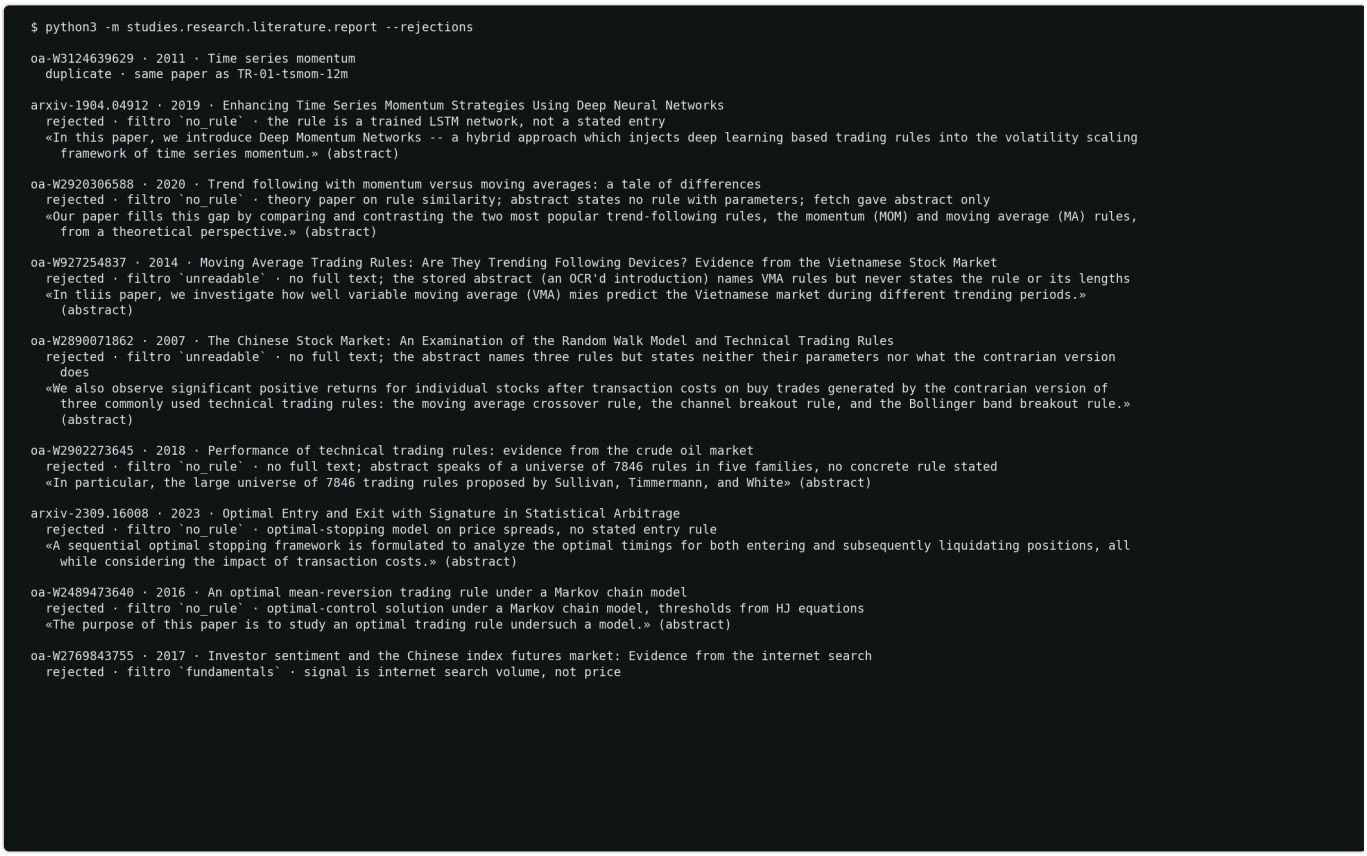
TR-59-maco-5min-covid · tendencia · «pregunta al dueño»

BRK-60-trbo-5min-covid · ruptura · «medida» · medidas trbo_c50_opp, trbo_c150_opp, trbo_c200_opp
  3 medidas, 184 celdas con contraste, 8 con p cruda ≤ 0,05, 0 significativas tras corregir, 0 pasan los cuatro filtros; mejor: `trbo_c200_opp` XAUUSD M15 long
    8.7× el coste, p corr. 0.14, 36 op/año
[…]
```

Arriba, cuántos papers se han visto y qué fue de ellos, y por qué filtro cayeron los rechazados. Luego una línea por pasada con su coste. Luego las **preguntas abiertas**: reglas que el paper escribe de forma que admiten dos lecturas; cada una trae sus lecturas numeradas y espera tu respuesta. Al final, la cola.

Cada entrada está en uno de cuatro estados:

estado	qué significa	qué hay que hacer
«pregunta al dueño»	la regla tiene dos lecturas	contestar con `--answer`; hasta entonces no se mide
«sin medida: falta una condición»	el perfil no tiene la condición que la regla necesita; la entrada dice cuál	añadirla al perfil en una sesión (es código), o dejarla
«en cola»	tiene medida escrita y el perfil aún no la ha corrido	nada: se corre a mano con `--top-up` (la pasada diaria ya no mide)
«medida»	ya está medida en `build`	leer el resultado



Un rechazo siempre trae la frase literal del paper que lo justifica y dónde está (`abstract` o la página). Si te parece mal rechazado, esa frase es lo primero que mirar.

Cuando una entrada está medida, su línea dice en cuántas celdas hubo contraste, cuántas salen con p cruda ≤ 0,05, cuántas siguen siendo significativas tras corregir por todas las pruebas del mapa, cuántas pasan los cuatro filtros del perfil (significativa, paga dos veces su coste, estable por años, al menos 40 operaciones al año) y cuál es la mejor. **Sólo las que pasan los cuatro filtros hacen que su familia entre en el tablero por lo medido.** «Ninguna celda» es un resultado, no un fallo.

Un ejemplo completo

El primer día (2026-10-02), cuatro pasadas, 67 papers leídos y 3 repetidos detectados:

```
$ python3 -m studies.research.literature.report
70 papers vistos: 9 accepted, 3 duplicate, 58 rejected
   31 rechazados por `no_rule` …
    8 rechazados por `clock` …
    5 rechazados por `unreadable` …
    5 rechazados por `fundamentals` …
    4 rechazados por `two_legs` …
    3 rechazados por `not_sqx_at_market` …
    1 rechazados por `conditions` …
    1 rechazados por `cross_sectional` …
run 20261002-140144 (opus): 20 papers, 5 entradas, 171,568 tokens · por paper 8,578 · por entrada 34314
run 20261002-143154 (sonnet): 20 papers, 4 entradas, 923,856 tokens · por paper 46,193 · por entrada
230964
run 20261002-143808 (sonnet): 20 papers, 3 entradas, 605,582 tokens · por paper 30,279 · por entrada
201861
run 20261002-144518 (sonnet): 7 papers, 5 entradas, 128,411 tokens · por paper 18,344 · por entrada
25682
…
17 entradas del scout: 8 «sin medida: falta una condición», 8 «pregunta al dueño», 1 «medida»

BRK-60-trbo-5min-covid · ruptura · «medida» · medidas trbo_c50_opp, trbo_c150_opp, trbo_c200_opp
  3 medidas, 184 celdas con contraste, 8 con p cruda ≤ 0,05, 0 significativas tras corregir, 0 pasan los
  cuatro filtros; mejor: `trbo_c200_opp` XAUUSD M15 long 8.7× el coste, p corr. 0.14, 36 op/año
  al tablero (pasan los cuatro filtros): ninguna celda
```

Lectura:

- De 67 papers leídos, 9 dieron algo: 17 entradas. Los demás no traen una regla copiable (31), usan reloj (8) o caen por otro filtro.
- De las 17 entradas, 8 esperan una respuesta tuya (el paper escribe la regla con dos lecturas), 8 necesitan una condición que el perfil no tiene, y 1 se pudo medir ya.
- La medida: la ruptura de rango sobre cierres a 50, 150 y 200 barras (Lento y Gradojević, 2022), en índices, metales y petróleos. 8 celdas de 184 con p cruda ≤ 0,05 es lo que da el azar (un 4 %); ninguna sobrevive a la corrección y ninguna pasa los cuatro filtros. **Decisión: no se construye sobre ella; el tablero no cambia.** La mejor celda (oro M15 largo) paga mucho pero hace 36 operaciones al año, por debajo del mínimo de 40.
- Las cuatro líneas `run` no son comparables entre sí tal cual. La primera es un agente largo y su cifra la dio el propio agente, sin contar relecturas. La segunda y la tercera son un agente por paper con herramientas: cada uso de herramienta relee unos 41.000 tokens de contexto fijo. La cuarta es el esquema que quedó: una llamada única por lectura, sin herramientas, unos 21.000 tokens por llamada (antes 86.000). Ese lote sólo encontró 7 papers nuevos; con 20 saldría más barato por paper, porque la criba se reparte.

El registro que tú lees, con un renglón por paper, es `AlgoData/research/literature/REGISTRO.md` (se rehace con `report --register`, y sola cada día).

La pasada diaria

`bin/daily-literature-scout.sh` hace lo mismo sin nadie delante: busca (sin tokens), lanza la criba, descarga los que quedan y hace una llamada por paper. Corre cada día a las 07:00 (cron). `run.budget_tokens` en `studies/research/literature/config.yaml` es sólo un seguro contra un desbocamiento (3 millones; una pasada

normal gasta unos cientos de miles): si se llegara, no lanza más y los papers que queden siguen en la bandeja para el día siguiente. Apunta lo que costó, rehace el registro, mide en `build` lo que falte y deja el estado en `AlgoData/audit/<fecha>-bibliografia.md`. El registro va a `AlgoData/logs/daily-literature-scout.log`.

```
bin/daily-literature-scout.sh --dry-run    # enseña rutas, modelo y topes; no lanza nada
```

No hay tope de entradas en cola (tu decisión del 2026-10-02: «si salen más se prueban»).

Qué NO te dice

- **Que la regla funcione.** Lo publicado es lo que sus autores eligieron; la entrada apunta cuántas variantes dicen haber probado y si su muestra pisa nuestro tramo `build`, precisamente para que no se lea como evidencia independiente.
- **Que la entrada diga sólo lo que dice el paper.** El programa comprueba que cada cita está tal cual en el texto guardado y en esa página. No puede comprobar que el dato escrito al lado no vaya más lejos que la cita: eso se mira leyendo. Tampoco exige una frase entera: un trozo de 20 letras pasa.
- **Nada sobre `oos1` ni `oos2`.** La medida es sólo en `build`, como todo el perfil.
- **Que no haya más papers.** Sólo busca en arXiv y OpenAlex, por las consultas de `config.yaml`, y de OpenAlex **sólo los que tienen PDF abierto** (tu decisión del 2026-10-02): un resumen casi nunca dice la regla, y en el primer lote ninguno de los 13 papers leídos sólo por el resumen dio una entrada. El precio es no ver los papers de revistas cerradas.
- **Las estrategias de repositorios de GitHub valen menos que un paper.** Entran (tu decisión del 2026-10-02) con el grado más bajo, «code repository», que les pone el programa: nadie las revisó y casi nunca dicen sobre qué datos se probaron ni cuántas variantes se descartaron. Las que no traen ningún backtest propio se rechazan. Hoy se leen dos colecciones, tres estrategias de cada una por pasada: `StockSharp/AlgoTrading` (3.811 estrategias cortas) y `rolling-panda-san/notebooks` (25 papers replicados).

Si algo falla

- `REFUSED` seguido de una lista, al aceptar: el borrador rompe algo — una cita que no está en el texto, un dato sin cita, una palabra de reloj en la regla, más de dos condiciones. Se corrige el borrador mirando el paper, no el comprobador.
- `the quote is not in the source at …: nothing recorded`, al rechazar: la cita no es literal o la página no es esa.
- `DUPLICATE`: la regla ya es una medida del perfil o de otra entrada, con otro nombre. No se guarda.
- `BIBLIOGRAFÍA sin medir — <entrada>: condición desconocida […]`, al medir: la medida de esa entrada nombra una condición que el perfil no tiene. La entrada se queda en cola hasta que se corrija.
- `abstract only … — <url>: HTTP 403 …`: la editorial no deja descargar el PDF a un programa. Es lo esperado con SSRN y con muchas revistas; el paper se lee por el resumen.

97. Inspeccionar proyectos y archivos de SQX — salud, mapa, índice y copia de logs

Cuatro comandos de **solo lectura** sobre lo que SQX deja en disco. Ninguno arranca, para ni consulta SQX, y ninguno escribe dentro de una instalación (`archive_logs` escribe en `AlgoData/logs/`, `index_sqx` donde tú le digas).

Qué pregunta responde

- `python3 -m sqx.inspect.project_health`: «¿hay algún proyecto roto en esta instalación?» (un proyecto al que le falta un archivo de tarea, y que SQX descarta al cargar sin avisar).
- `python3 -m sqx.inspect.dump_project`: «¿qué hace de verdad este proyecto?» Lo convierte en un mapa legible: bancos de datos, orden de tareas y qué lee y escribe cada una.
- `python3 -m sqx.inspect.index_sqx`: «¿qué estrategias distintas tengo en el disco, y en cuántas copias?» Identifica cada `.sqx` por el hash de su XML interno, no por el nombre.
- `python3 -m sqx.export.archive_logs`: copia comprimidos los logs de SQX a `AlgoData/logs/` antes de que se pierdan.

Cuándo lo usas, y cuándo no

- `project_health`: antes de fiarte de un proyecto, o si uno «desaparece» de SQX al abrirlo.
- `dump_project`: en vez de leer el `project.cfx` a ojo. No existe ningún mapa guardado en `docs/`: se regenera cada vez, así nunca está desfasado.
- `index_sqx`: para saber cuántas estrategias únicas hay entre todas las carpetas (`strategy_pools` de `config/machine.yaml`). No sirve para ver métricas: solo identidad, símbolo y feed.
- `archive_logs`: no lo lanzas tú normalmente; corre solo cada noche a las 03:45 y `bin/sqx-log-prune.sh` borra un log vivo únicamente si ya existe aquí su copia `.gz`.

Antes de empezar

Nada especial: se pueden lanzar con SQX abierto o cerrado, porque solo leen archivos. Un proyecto se busca por nombre en el master y, si no está, en el conductor y el custodio; también vale una ruta a un `.cfx`.

Cómo se ejecuta

```
python3 -m sqx.inspect.project_health --install ~/Desktop/SQX_w2
python3 -m sqx.inspect.dump_project Trade_NIKKEI225_M30_volatilidadL -o /tmp/mapa.md
python3 -m sqx.inspect.index_sqx /tmp/indice.json
python3 -m sqx.export.archive_logs --install ~/Desktop/SQX_w2
```

comando	flag	qué hace
`project_health`	`--install RUTA`	instalación a revisar (por defecto el master)
`project_health`	`--json`	la misma tabla como JSON
`dump_project`	`PROYECTO` (obligatorio)	nombre o ruta de un `.cfx`
`dump_project`	`-o ARCHIVO`	escribe el Markdown ahí en vez de imprimirlo
`index_sqx`	`SALIDA` (obligatorio)	JSON que se escribe
`index_sqx`	`--workers N`	procesos en paralelo (48 por defecto)
`archive_logs`	`--install RUTA`	repetible; sin él, master y todos los workers

Tiempos medidos: `project_health` y `dump_project`, instantáneos; `index_sqx` unos 3 s para 8.382 archivos; `archive_logs`, segundos (salta los logs ya copiados y al día).

Qué produce

- `project_health`, `dump_project` (sin `-o`): solo pantalla.
- `dump_project -o`: el Markdown en la ruta que des.
- `index_sqx`: un JSON con `by_hash` (por estrategia: símbolo, feed y cada copia con su ruta, su carpeta y su fecha) y `unreadable` (archivos truncados o corruptos).
- `archive_logs`: `AlgoData/logs/<instalación>/…log.gz` más un manifiesto. Un log del día se reescribe en la pasada siguiente.

Cómo se lee el resultado

Salud. Una fila por proyecto. `moji` cuenta campos con texto estropeado (un proyecto importado desde Windows); el comando muestra debajo cómo decía el original. «older than the install» solo significa que el proyecto lo guardó una versión anterior de SQX: no es un fallo. Lo grave es `broken: missing …`: SQX no lo cargará.

```
$ python3 -m sqx.inspect.project_health --install ~/Desktop/SQX_w2
/home/sergioguslw/Desktop/SQX_w2 — newest project stamp 144.2953

project                          version   tasks  moji  state
Builder                          144.2953      1     0  ok
Optimizer                        144.2953      1     0  ok
PortfolioComposer                144.2953      0     0  ok
PortfolioMaster                  144.2953      1     0  ok
Retester                         144.2953      1     0  ok
Trade_NIKKEI225_M30_volatilidadL 142.2399     20     1  older than the install

Mangled fields, decoded:
  config.xml:templateFile = C:\Users\Rubén Martínez\OneDrive\Escritorio\FILTROS\Build strategies.cfx
```

Mapa. Cabecera con la ruta de donde se leyó, y luego las secciones `Databanks`, `TL;DR`, `Databank flow`, `Task order` y `Tasks in detail`.

```
$ python3 -m sqx.inspect.dump_project Trade_NIKKEI225_M30_volatilidadL | sed -n 1,16p
# Trade_NIKKEI225_M30_volatilidadL — pipeline map

Generated by `python3 -m sqx.inspect.dump_project` from `/home/sergioguslw/Desktop/SQX_w2/user/projects/Trade_NIKKEI225_M30_volat…
Project declares `version="142.2399"`.

> **Read-only artefact.** Regenerate rather than hand-editing. Never edit a
> `project.cfx` on disk while SQX holds the project: SQX rewrites the whole
> file on save and exit, and the change is silently lost.

## Databanks

| databank | view | sync | position |
|---|---|---|---|
| `Results` | Modo Sergiogus | Auto-sync every 1 hour ⚠ | 0 |
| `Last generation` | Modo Sergiogus | Auto-sync every 1 hour ⚠ | 1 |
| `Initial population` | Default - Main data | Auto-sync every 1 hour ⚠ | 2 |
```

```
$ python3 -m sqx.inspect.dump_project Trade_NIKKEI225_M30_volatilidadL | grep -n "^## "
10:## Databanks
45:## TL;DR
50:## Databank flow
59:## Task order
84:## Tasks in detail
```

Índice. Dos cifras: estrategias únicas y archivos ilegibles. Aquí, 8.382 archivos son 6.119 estrategias distintas: el resto son copias.

```
$ python3 -m sqx.inspect.index_sqx /tmp/indice.json
scanning 8382 .sqx across 7 pools
unique strategies: 6119   unreadable: 5
```

Logs. `archived` = copia nueva escrita; `current` = ya había una al día.

```
$ python3 -m sqx.export.archive_logs --install ~/Desktop/SQX_w2
current   launcher_2026_09_27.log            3,044 bytes
current   launcher_2026_09_28.log              436 bytes
current   launcher_2026_09_29.log            1,742 bytes
current   launcher_2026_09_30.log            4,460 bytes
current   launcher_2026_10_01.log            7,340 bytes
current   launcher_2026_10_02.log           15,449 bytes
current   launcher_2026_10_03.log           18,929 bytes
archived  launcher_2026_10_04.log              966 bytes
archived  worker-daemon.log             52,587,893 bytes
archived  xvfb.log                              0 bytes
SQX_w2: 5 written, archive at /home/sergioguslw/Desktop/AlgoData/logs/SQX_w2
```

Un ejemplo completo

Quieres saber qué hace el proyecto `Trade_NIKKEI225_M30_volatilidadL`, que vive en el custodio: `python3 -m sqx.inspect.dump_project Trade_NIKKEI225_M30_volatilidadL -o /tmp/mapa.md` lo encuentra solo y escribe el mapa. Antes de abrirlo, `project_health --install ~/Desktop/SQX_w2` te confirma que no está roto.

Qué NO te dice

- Que un proyecto esté «ok» no dice que sus tareas estén bien configuradas, solo que su archivo está completo.
- El mapa es lo que hay **en disco**; si SQX tiene el proyecto abierto con cambios sin guardar, no los ve.
- El índice no distingue la misma estrategia reexportada con otra métrica: la identidad es su XML.
- Nunca edites a mano un `project.cfx` que SQX tenga cargado: lo reescribe al guardar y perderías el cambio.

Si algo falla

- `FileNotFoundError … not in user/projects of any of …`: el nombre no existe en ninguna instalación; revisa la ortografía o pasa la ruta del `.cfx`.
- `dump_project` con `KeyError` / `BadZipFile` en un proyecto concreto: le falta un archivo de tarea; `project_health` te lo dirá.
- `index_sqx` con archivos en `unreadable`: están truncados; no detienen el escaneo.

91. Potencia del perfil — qué tamaño de edge es capaz de ver el perfil de mercado

Qué pregunta responde

El perfil de mercado (cap. 82) dice cuántas celdas pasan sus filtros. No dice cuántas celdas **con** edge se le escapan. Este estudio lo mide: planta un edge de tamaño conocido en las barras de `build`, le pasa exactamente la misma prueba del perfil (la misma nula, los mismos sorteos) y cuenta cuántas veces lo detecta. El resultado es el **efecto mínimo detectable**: el edge más pequeño que el perfil ve 8 de cada 10 veces. Sirve para leer un «no pasa»: si el perfil no podía verlo, no es prueba de que no haya nada.

Cuándo lo usas, y cuándo no

Lo usas cuando el mapa del perfil cambia (medidas nuevas, más sorteos, otra corrección) y quieres saber qué puede ver ahora; y antes de dar por muerta una celda porque «el perfil no la nombra».

No lo uses para buscar edge: aquí no se mira ninguna dirección real. Las réplicas se hacen sobre copias de las barras con el signo de cada barra echado a cara o cruz (la volatilidad real, ninguna dirección), y el edge que aparece es el que se ha plantado.

Antes de empezar

Tiene que existir el mapa del perfil (`AlgoData/research/profiles/measures.csv` y `context.csv`, cap. 82): de ahí sale la p que hoy exige la corrección (0,00167 el 2026-10-02). No toca SQX.

Memoria: cada proceso ocupa cerca de 1 GB. El tope está en `run.budget_bytes` (8 GB) y `run.workers` (4). Con un build de SQX en marcha no lo subas: 32 procesos llegaron a 25 GB.

Cómo se ejecuta

```
python3 -m studies.research.profilePower.report                # corre las réplicas que falten y juzga
python3 -m studies.research.profilePower.report --judge-only    # sólo las tablas, con lo ya guardado
python3 -m studies.research.profilePower.report --set "run.cells=[USOIL/H1, USOIL/H4]"   # más celdas
python3 -m studies.research.profilePower.report --set plant.how=random    # el control
```

flag	obligatorio	qué hace
`--judge-only`	no	no corre réplicas; rehace las tablas con las guardadas y con el mapa de hoy
`--force`	no	repite las réplicas aunque ya estén guardadas
`--set clave=valor`	no	cambia un valor de `studies/research/profilePower/config.yaml` para esta corrida

Lo que más se cambia: `run.cells` (las celdas), `plant.effects` (los tamaños, en múltiplos del coste de ida y vuelta), `plant.replications` (copias por celda), `plant.measures` (qué medida lleva el edge de cada familia) y `plant.how` (`condition`: en las barras que siguen a la condición de la medida; `random`: las mismas barras, pero al azar — el control, que no debe detectar nada).

Cuánto tarda. Una réplica cuesta entre 4 segundos y 1 minuto en H4 y entre 20 segundos y 4 minutos en H1 (para en cuanto la p exigida ya es inalcanzable). Las 3 celdas H4 con 10 medidas, 7 tamaños y 8 copias son 1.680 réplicas. Se puede cortar: guarda cada 100 y al relanzar sigue donde iba.

Qué produce

Todo en `AlgoData/research/power/`:

fichero	qué es
`runs.parquet`	una fila por réplica; no se borra, se le añaden las nuevas
`mde.csv`, `mde_class.csv`	el efecto mínimo detectable por familia (o clase de activo), medida y marco
`curve.csv`, `curve_class.csv`	la curva de potencia: tasa de detección por tamaño plantado
`reach.csv`, `reach_class.csv`	lo que el **mapa entero** puede ver, por familia o clase y marco
`widened.csv`	cada prueba del mapa con su efecto mínimo detectable
`calibration.csv`	la simulación frente a la fórmula con la que se extiende al mapa entero
`power.html`, `power.md`, `power.json`	el informe
`ledger.jsonl`, `run.json`	las miradas a los datos y la última corrida

Las tablas se sobrescriben en cada corrida; las réplicas no.

Cómo se lee el resultado

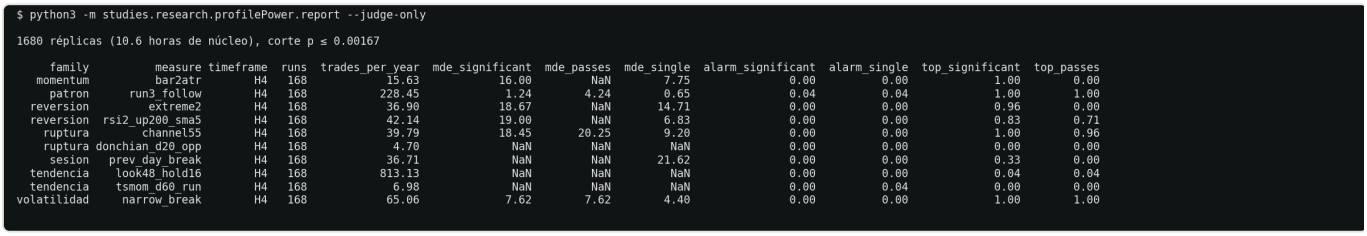
```
$ python3 -m studies.research.profilePower.report --judge-only

Lo que el mapa entero puede ver (aproximación normal):
     family timeframe  tests  mde_median  mde_atr_median  trades_per_year_median  sees_1x  sees_2x  sees_5x
   momentum        H1    418       3.887           0.513                  47.966    0.124    0.230    0.660
   momentum        H4    381       6.998           0.466                  16.159    0.058    0.100    0.381
   momentum       M15    418       1.542           0.422                 107.452    0.330    0.617    0.892
   momentum       M30    418       2.651           0.479                  77.796    0.196    0.366    0.859
  reversion        H1   1056       3.974           0.541                  61.869    0.160    0.312    0.570
  reversion        H4   1050       8.416           0.546                  26.679    0.034    0.081    0.323
  reversion       M15   1056       1.381           0.357                 164.764    0.436    0.545    0.654
  reversion       M30   1056       2.442           0.482                 113.543    0.281    0.448    0.651
    ruptura        H1    395       7.330           1.007                  41.328    0.035    0.139    0.387
    ruptura        H4    354      18.499           1.259                  12.358    0.008    0.034    0.096
    ruptura       M15    400       1.969           0.568                 161.982    0.298    0.502    0.702
    ruptura       M30    399       3.930           0.769                  79.755    0.120    0.291    0.546
     sesion        H1     38       8.599           1.041                  46.930    0.053    0.079    0.184
     sesion        H4     38      11.151           0.709                  38.126    0.000    0.000    0.079
     sesion       M15     38       7.210           2.277                  51.299    0.053    0.079    0.184
     sesion       M30     38       7.240           1.399                  48.733    0.026    0.053    0.184
  tendencia        H1    736       6.872           0.907                  60.263    0.152    0.250    0.444
  tendencia        H4    721      17.834           1.272                  15.911    0.037    0.085    0.268
  tendencia       M15    736       2.911           0.917                 161.110    0.323    0.452    0.567
  tendencia       M30    736       4.000           0.754                 107.373    0.239    0.353    0.530
volatilidad        H1    110       5.662           0.736                  20.396    0.236    0.364    0.455
volatilidad        H4    112       7.519           0.448                  17.114    0.080    0.116    0.321
volatilidad       M15    111       5.016           1.395                  22.215    0.360    0.387    0.495
volatilidad       M30    111       5.056           1.028                  21.315    0.351    0.378    0.468

Por clase de activo:
asset_class timeframe  tests  mde_median  mde_atr_median  trades_per_year_median  sees_1x  sees_2x  sees_5x
     energy        H1    286       1.128           0.729                  54.394    0.448    0.640    0.773
     energy        H4    272       2.367           0.759                  24.231    0.254    0.441    0.684
     energy       M15    286       0.589           0.782                 152.104    0.626    0.727    0.815
     energy       M30    286       0.745           0.720                  92.992    0.570    0.699    0.787
      forex        H1   1447       4.735           0.565                  54.637    0.113    0.230    0.520
      forex        H4   1428      10.847           0.647                  20.514    0.011    0.038    0.263
      forex       M15   1449       1.848           0.437                 152.104    0.358    0.513    0.659
      forex       M30   1449       3.028           0.508                  97.393    0.199    0.364    0.630
      index        H1    722       6.505           0.787                  46.601    0.064    0.158    0.400
      index        H4    663      13.328           0.829                  20.889    0.008    0.032    0.169
      index       M15    726       2.920           0.703                 116.822    0.262    0.426    0.605
      index       M30    725       4.334           0.752                  75.223    0.157    0.268    0.539
      metal        H1    298       4.770           0.696                  48.333    0.124    0.258    0.510
      metal        H4    293      12.145           0.787                  19.513    0.024    0.048    0.263
      metal       M15    298       1.782           0.485                 135.893    0.366    0.513    0.661
      metal       M30    298       3.002           0.535                  85.609    0.262    0.416    0.617
```

- **mde_median**: el efecto mínimo detectable, en múltiplos del coste de ida y vuelta. 4 quiere decir que una regla tiene que ganar de media 4 costes por operación *por encima de lo que da el azar* para que el perfil la nombre 8 de cada 10 veces. El filtro «paga» pide 2: **un valor por encima de 2 significa que el perfil no ve un edge del tamaño que él mismo exige.**

- **mde_atr_median**: lo mismo en ATR de la barra del marco.
- **sees_1x, sees_2x, sees_5x**: qué parte de las pruebas del mapa vería un edge de 1, 2 o 5 costes. 1,00 es todas; 0,03 es casi ninguna.
- En `mde.csv` hay tres lecturas: `mde_significant` (basta con ser significativo al corte del mapa), `mde_passes` (los cuatro filtros) y `mde_single` (la regla juzgada sola, al 5 % sin corregir). La distancia entre la primera y la tercera es lo que cuesta mirar 14.000 cosas a la vez. Vacío: no se alcanza ni con el mayor tamaño plantado.
- `alarm_*` es la detección sin nada plantado: tiene que quedar cerca de 0 al corte del mapa y cerca de 0,05 en la prueba sola.



```
$ python3 -m studies.research.profilePower.report --judge-only

1680 réplicas (10.6 horas de núcleo), corte p ≤ 0.00167

      family          measure timeframe  runs  trades_per_year  mde_significant  mde_passes  mde_single  alarm_significant  alarm_single  top_significant  top_passes
    momentum          bar2atr        H4   168           15.63            16.00         NaN        7.75               0.00          0.00             1.00        0.00
      patron      run3_follow        H4   168          228.45             1.24        4.24        0.65               0.04          0.04             1.00        1.00
   reversion          extreme2        H4   168           36.90            18.67         NaN       14.71               0.00          0.00             0.96        0.00
   reversion  rsi2_up200_sma5        H4   168           42.14            19.00         NaN        6.83               0.00          0.00             0.83        0.71
     ruptura         channel55        H4   168           39.79            18.45       20.25        9.20               0.00          0.00             1.00        0.96
     ruptura donchian_d20_opp        H4   168            4.70              NaN         NaN         NaN               0.00          0.00             0.00        0.00
      sesion    prev_day_break        H4   168           36.71              NaN         NaN       21.62               0.00          0.00             0.33        0.00
   tendencia     look48_hold16        H4   168          813.13              NaN         NaN         NaN               0.00          0.00             0.04        0.04
   tendencia     tsmom_d60_run        H4   168            6.98              NaN         NaN         NaN               0.00          0.04             0.00        0.00
volatilidad      narrow_break        H4   168           65.06             7.62        7.62        4.40               0.00          0.00             1.00        1.00
```

Lectura de la simulación en H4 (1.680 réplicas, 2026-10-02): al corte del mapa una ruptura de canal necesita 18 costes por operación, una reversión 19, una barra de impulso 16; un patrón de muchas operaciones, 1,2. Las reglas de pocas operaciones (`donchian_d20_opp` , `tsmom_d60_run` : 5-7 al año) no se detectan ni con 25 costes. Juzgadas solas al 5 % bastarían 9, 7-15 y 8. La simulación detecta de media 9 puntos **menos** que la fórmula: la tabla del mapa entero es optimista.

Un ejemplo completo

```
$ python3 -m studies.research.profilePower.report --judge-only
Lo que el mapa entero puede ver (aproximación normal):
     family timeframe  tests  mde_median  mde_atr_median  trades_per_year_median  sees_1x  sees_2x
sees_5x
    ruptura        H1    395       7.330           1.007                  41.328    0.035    0.139
0.387
    ruptura        H4    354      18.499           1.259                  12.358    0.008    0.034
0.096
    ruptura       M15    400       1.969           0.568                 161.982    0.298    0.502
0.702
  reversion        H4   1050       8.416           0.546                  26.679    0.034    0.081
0.323
  reversion       M15   1056       1.381           0.357                 164.764    0.436    0.545
0.654
...
```

Lectura (2026-10-02): de 10.926 pruebas con operaciones, la mediana necesita 4,6 costes (0,63 ATR); sólo el 31 % vería un edge de 2 costes. Lo decide el número de operaciones: con menos de 12 al año hacen falta 48 costes; de 40 a 100, 4,1; con más de 300, 0,7. En H4 el perfil está casi ciego (ve 2 costes en el 3-5 % de las pruebas de forex, índices y metales).

Cuánto costó: las 1.680 réplicas de H4, 10,6 horas de núcleo. H1 cuesta unas cuatro veces más por celda y no está simulado; lo cubre la aproximación.

Qué NO te dice

- Que haya edge en ninguna parte: sólo qué tamaño se vería si lo hubiera.

- La tabla del mapa entero es una **aproximación** (la prueba tomada como una t normal); la simulación de `calibration.csv` dice cuánto fiarse. No cubre las medidas de patrón (`inside`, `engulfing`, `run3_*`), que se juzgan por tasa de acierto.
- Un edge muy grande plantado en una regla que opera muchas barras se convierte en la deriva de la serie, y la nula conserva la deriva: el perfil no lo ve. Por eso una regla que está casi siempre dentro (`look48_hold16`) no se detecta a ningún tamaño. Es el diseño del perfil, no un fallo.
- Los tamaños de 10 y 25 costes sólo tienen sentido en reglas de pocas operaciones.

Si algo falla

- `FileNotFoundError: …/profiles/measures.csv`: falta el mapa del perfil (cap. 82).
- La máquina se queda sin memoria: baja `run.workers`. Lo guardado no se pierde.

92. Perfil agrupado — las medidas del perfil juzgadas por grupo de activos

Qué pregunta responde

El perfil de mercado (cap. 82) juzga cada medida en un activo, y en un solo activo casi nunca hay operaciones bastantes para ver un edge pequeño (cap. 91). Este estudio hace la misma pregunta a un **grupo**: ¿esta regla funciona en los pares de forex en H1, tomados juntos? Suma lo que dice cada activo y lo compara con una nula en la que todos los activos del grupo se remuestrean **a la vez en el tiempo**, para no contar diez veces lo mismo cuando los diez se mueven juntos. Y da, para cada activo, su efecto «encogido»: el suyo propio acercado a la media de su grupo tanto más cuanto más ruidoso es.

Cuándo lo usas, y cuándo no

Lo usas después de medir el perfil, para ver qué comportamientos son de una clase de activo entera aunque ningún activo los enseñe solo, y cuánto pagan de verdad una vez quitado el ruido de cada uno.

No lo uses para decidir que **un** activo concreto tiene edge: que el grupo pase dice que la regla funciona de media en el grupo, no en cada miembro. Y no cambia el tablero: sólo cuenta cuántas celdas entrarían si tú decidieras abrir esa vía.

Antes de empezar

Las barras de los 19 activos y sus tramos `build`. Lee sólo `build`, y de cada grupo sólo las barras que tienen **todos** sus activos (en índices, desde octubre de 2013). Para contar celdas nuevas lee `AlgoData/research/board/board.json` (cap. 83). No toca SQX.

Memoria: cada proceso ocupa unos 2 GB. Viene con `run.workers: 4`. Con 20 procesos llegó a 40 GB y puso en riesgo un build de SQX: no lo subas mientras SQX esté construyendo.

Cómo se ejecuta

```
python3 -m studies.research.pooledProfile.report                # mide lo que falte y juzga
python3 -m studies.research.pooledProfile.report --judge-only   # sólo juzga lo ya medido
python3 -m studies.research.pooledProfile.report --force        # mide todo de nuevo, con otra semilla
```

flag	obligatorio	qué hace
`--judge-only`	no	no mide; rehace las tablas con lo guardado
`--force`	no	vuelve a medir todo con una semilla nueva
`--set clave=valor`	no	cambia un valor de `studies/research/pooledProfile/config.yaml` para esta corrida

Cuánto tarda. Es lo mismo que medir el perfil entero: 76 celdas (19 activos × 4 marcos) con 3.000 sorteos. Con 2 procesos en una máquina ocupada, 5 horas (un par de forex: M15 unos 50 minutos, M30 12, H1 6, H4 2). Si se corta, al relanzar sigue con las celdas que faltan y la misma semilla.

Qué produce

Todo en `AlgoData/research/pooled/`:

fichero	qué es
`tests.csv`	una fila por prueba de grupo (clase × marco × dirección × medida): su p, su q y si pasa
`assets.csv`	cada activo de cada prueba: su efecto en bruto y encogido
`board.csv`	las celdas-familia que traería cada grupo que pasa, y cuáles son nuevas en el tablero
`pooled.html`, `pooled.md`, `pooled.json`	el informe
`cells/`	lo medido por activo (se reutiliza al juzgar)
`seed.json`, `ledger.jsonl`, `run.json`	la semilla, las miradas a los datos y la última corrida

Las tablas se sobrescriben en cada corrida.

Cómo se lee el resultado

```
$ python3 -m studies.research.pooledProfile.report --judge-only

142 de 2543 pruebas de grupo pasan (BH al 5%); en mundos sin nada el procedimiento nombra algo el 0.0% de las veces (200 placebos)

(las 14 con operaciones de mayor exceso medio)
group timeframe direction    family           measure  assets   stat     p     q  mean_excess   tau    i2
metal        H4      long  momentum       bar2atr_up200       2  3.654 0.000 0.008        5.512 0.000 0.000
forex       M30      long reversion day_down_volhigh_1d      10  4.433 0.002 0.043        2.124 0.000 0.000
forex       M15      long reversion day_down_volhigh_1d      10  4.264 0.003 0.048        2.014 0.000 0.000
forex        H1      long reversion day_down_volhigh_1d      10  4.284 0.002 0.032        2.011 0.000 0.000
forex        H1      long  momentum            bar3atr      10  4.521 0.002 0.032        1.701 1.240 0.520
metal        H1      long    patron           run3_fade       2 12.868 0.000 0.008        0.764 0.145 0.718
metal        H4     short    patron           run3_fade       2  4.257 0.000 0.008        0.740 0.400 0.689
metal        H4      long    patron           run3_fade       2  3.728 0.001 0.015        0.726 0.000 0.000
forex        H1      long    patron           run3_fade      10 24.249 0.000 0.008        0.657 0.188 0.843
forex        H1     short    patron           run3_fade      10 24.775 0.000 0.008        0.619 0.185 0.863
metal       M15      long  momentum       bar2atr_up200       2  3.094 0.002 0.038        0.613 0.000 0.000
metal       M15     short reversion     extreme2_hold24       2  3.161 0.003 0.048        0.602 0.000 0.000
metal        H1     short    patron           run3_fade       2 12.102 0.000 0.008        0.589 0.178 0.846
metal       M30      long    patron           run3_fade       2 19.369 0.000 0.008        0.587 0.263 0.973

Celdas-familia que traerían: 218 (vía amplia), 3 (vía estricta); nuevas en el tablero: 148 y 0; 164 ya pasaban solas la significancia
```

- **assets**: cuántos activos del grupo tenían operaciones suficientes para esa medida.
- **stat**: la suma de lo que dice cada activo. **p** y **q**: la p del grupo y la misma corregida entre todas las pruebas de grupo; pasa con q ≤ 0,05.
- **mean_excess**: lo que gana de media la regla por operación **por encima del azar**, en costes de ida y vuelta. Es el número que importa: un grupo puede pasar con 0,3 costes — real, pero no paga. Hace falta 2 para el filtro del perfil.
- **tau** e **i2**: cuánto difieren los activos entre sí. tau 0: todos dicen lo mismo. i2 cerca de 1: el grupo no es un grupo (unos lo tienen y otros no).

```
assets.csv — forex H1 long day_down_volhigh_1d

symbol    z    p  multiple  excess   se  shrunk_excess  shrunk_multiple  trades_per_year  significant_alone  strict
AUDJPY 1.51 0.06      1.42    1.70 1.12           2.01             1.73            44.84              False   False
AUDUSD 0.68 0.24      0.65    1.03 1.51           2.01             1.63            45.04              False   False
CADJPY 2.61 0.01      2.90    3.14 1.20           2.01             1.77            44.94              False   False
EURJPY 2.23 0.01      3.72    4.17 1.87           2.01             1.56            41.14              False   False
EURUSD 1.42 0.08      3.75    4.23 2.97           2.01             1.53            43.44              False   False
GBPJPY 1.09 0.13      0.83    1.41 1.30           2.01             1.42            42.64              False   False
GBPUSD 0.47 0.32     -0.13    0.66 1.40           2.01             1.22            46.14              False   False
USDCAD 1.80 0.04      2.94    2.59 1.44           2.01             2.36            42.24              False    True
USDCHF 0.10 0.45      0.02    0.19 1.87           2.01             1.84            43.84              False   False
USDJPY 1.62 0.05      3.10    3.34 2.06           2.01             1.77            45.74              False   False
```

- **multiple**: el efecto del activo solo, bruto, en costes. **shrunk_multiple**: el mismo encogido hacia su grupo. Un activo que solo daba 3,7 puede quedarse en 1,6: lo demás era ruido.
- **significant_alone**: si el activo pasaba la significancia él solo.
- **strict**: el efecto encogido paga 2 costes, es estable por años y da las operaciones mínimas.

La última línea cuenta las celdas para el tablero por las dos vías: **amplia** (todos los activos de un grupo que pasa) y **estricta** (sólo los que cumplen `strict`), y cuántas no están ya.

Un ejemplo completo

```
$ python3 -m studies.research.pooledProfile.report --judge-only
142 de 2543 pruebas de grupo pasan (BH al 5%); en mundos sin nada el procedimiento nombra algo el 0.0%
de las veces (200 placebos)

group timeframe direction    family             measure  assets   stat     p     q  mean_excess   tau
i2
metal        H4      long  momentum       bar2atr_up200       2  3.654 0.000 0.008        5.512 0.000
0.000
forex       M30      long reversion day_down_volhigh_1d      10  4.433 0.002 0.043        2.124 0.000
0.000
forex        H1      long reversion day_down_volhigh_1d      10  4.284 0.002 0.032        2.011 0.000
0.000
forex        H1      long    patron           run3_fade      10 24.249 0.000 0.008        0.657 0.188
0.843
...
Celdas-familia que traerían: 218 (vía amplia), 3 (vía estricta); nuevas en el tablero: 148 y 0; 164 ya
pasaban solas la significancia
```

Lectura (2026-10-02): pasan 142 pruebas de grupo, 99 de ellas con operaciones; en 25 ningún activo pasaba solo. Casi todo es reversión y patrón en forex y metales, y casi todo paga muy poco: la mediana del exceso es 0,37 costes y sólo 4 grupos llegan a 2. Ningún grupo de tendencia ni de ruptura pasa. Por la vía estricta entrarían 3 celdas (USDCAD largo, reversión) y las 3 están ya en el tablero.

Qué NO te dice

- Que cada activo del grupo tenga el efecto: con tau 0 no se distingue «todos lo tienen» de «no hay datos para distinguirlos».
- Metales y energía son dos activos por grupo: agrupar ahí añade poco.
- «Largo» es largo del instrumento: largo USDCHF y largo EURUSD van al mismo grupo aunque sean apuestas contrarias sobre el dólar.
- El encogimiento trata los errores de los activos como independientes; con activos muy correlacionados encoge de más.
- Las pruebas sin operaciones (razón de varianzas, autocorrelación del rango) pasan en casi todos los grupos: dicen que la serie no es ruido, no que haya dinero.

Si algo falla

- `FileNotFoundError: …/cells/forex_M15_EURUSD.parquet`: falta medir esa celda; corre sin `--judge-only`.
- `MemoryError: … bytes of null draws`: los sorteos de un grupo no caben en `run.budget_bytes`.